澳大利亚国立大学

计算机学院

JHQ-GPU

面向表示的 GPU 近似最近邻搜索与工作负载级精排预算校准

Rui Luo

导师：Dr Mengxuan Zhang

中文阅读对照稿

章节、公式与图表编号对应英文稿。图内保留英文标签，便于与原始实验及英文稿核对。

COMP8800 — Advanced Computing Research Project

2026 年 10 月

研究来源与工具使用说明

本论文呈现 Rui Luo 在 Dr Mengxuan Zhang 指导下开展的 COMP8800 JHQ-GPU 项目。

底层 JHQ 表示及主码 / 残差层次归属于原始作者 [15]；本项目研究其 GPU 执行、索引构建与精排预算选择。与 Rui Luo 及 Mengxuan Zhang 共同手稿的关系见 第1.5节与附录 C，共享材料及历史测量保留明确来源。

生成式 AI 协助涉及实现、实验自动化、分析、审阅和论文准备。附录C依据项目记录说明所用工具及活动，附录A和B将实验结论对应至源码、测量及分析材料。提交文本、归属标注与结论解释的责任由论文作者承担。

摘要

紧凑向量表示可以减少存储，但高效 GPU 近似最近邻搜索还取决于评分、候选选择和精排工作量。本文提出 JHQ-GPU，对继承的 JHQ Cartesian 主码 / 残差层次进行 GPU 实现与评估，面向高维平方 L2 搜索。

系统结合 GPU 训练、batch 索引构建及面向表示的查询执行。Cartesian 结构把 256 项 primary 距离表拆成两张 16 项表，不改变实数表示距离。packed 访问、lane 合作精排和工作负载相关分组扫描配合这一设计。选择与共享界论证明确正确性条件；严格 CPU 排序参考与全部 55,296 个被测分组查询实例一致。工作负载策略以无标签输出和大预算参考的一致性选倍率，并区分样本一致性与 GT recall。

RTX 5090 上的归档测量覆盖六份嵌入数据。最大被测索引搜索 17.78 百万向量，常驻 20.44 GiB；两组表在 22/24 配置领先。相对 lane 精排参考，batch 8 集成调度吞吐几何平均为 2.727×，大 batch 收益小得多。0.97 recall、batch 1,000 的插值比较中，JHQ 在四个 E1 数据集中的三个第一，arXiv 与 IVF-RaBitQ 接近平手。历史 OpenAI 运行使用 TF32 GEMM，独立 FP32 敏感性在该目标仍领先归档 IVF-RaBitQ 7.9–8.6%。相同 recall、batch 8 下，四个数据集均改为 CAGRA fp32 领先；batch 32 的排名同样受有限调参网格影响。

全池校准在全部 12 单元满足相对最佳被测预算的 10^{-3} 留出 recall 损失目标，最大实测损失 0.0006；$S = 128$ 最多 40/64 次越阈。这些是固定查询池观察，不是总体 recall 保证。完整轨迹确认全部采样及全池有限网格决策；直接校准复用测量支持合适工作负载上的净收益，距离并列的一般理论仍有条件。负结果进一步说明算术复用和更低重建误差未必改善端到端前沿。研究支持在明确质量、计时和容量条件下联合评价表示、执行及工作量。

致谢

感谢 Mengxuan Zhang 博士指导本研究项目。本工作以 Jiabao Han、Mengxuan Zhang 和 Goce Trajcevski 的 JHQ 研究，以及文中引用的开源 ANN 与 GPU 软件为基础；这些论文和实现提供了研究基础与对照系统。

本项目及论文准备中的生成式 AI 使用记录见附录C。

目录

研究来源与工具使用说明 i

摘要 ii

致谢 iii

Notation and Abbreviations viii

1 引言 1
- 1.1 背景与实际目标 . . . 1
- 1.2 问题定义与范围 . . . 1
- 1.3 三个相互关联的技术挑战 . . . 2
- 1.4 研究问题与可反驳主张 . . . 2
- 1.5 贡献与研究归属 . . . 3
- 1.6 论文结构 . . . 3

2 背景与批判性相关工作 4
- 2.1 ANN 目标与评价指标 . . . 4
- 2.2 IVF、PQ、ADC 与残差精排 . . . 4
- 2.3 继承的 JHQ：旋转、Cartesian 编码与假设 . . . 5
- 2.4 本文使用的 GPU 执行概念 . . . 6
- 2.5 GPU 图搜索：SONG、CAGRA 与 BANG . . . 7
- 2.6 量化 ANN、快速查表与其他方案 . . . 7
- 2.7 批判性综合与研究缺口 . . . 8

3 研究设计与测量方法 9
- 3.1 将研究问题转化为受控比较 . . . 9
- 3.2 数据集、查询、真值与划分 . . . 9
- 3.3 测量边界、波动与显存核算 . . . 10
- 3.4 基线、调参网格与前沿构造 . . . 12
- 3.5 带版本的主张与证据登记 . . . 12
- 3.6 证据强度与有效性控制 . . . 13

4 系统架构与 GPU 索引构建 14
- 4.1 端到端建库与搜索边界 . . . 14
- 4.2 训练、旋转与码本生成 . . . 15
- 4.3 IVF 分配、ID 与物理内存布局 . . . 17
- 4.4 常驻存储、工作区与索引生命周期 . . . 19
- 4.5 构建成本与已测试规模 . . . 20
- 4.6 索引表示与安全契约 . . . 22

5 面向表示的 GPU 搜索执行 24
- 5.1 查询路径与四种不同的排序对象 . . . 24
- 5.2 精确的 Cartesian LUT 分解 . . . 25
- 5.3 打包编码与候选并行的内存访问 . . . 27
- 5.4 通过多 block 扫描实现精确全局主码 Top-c . . . 28
- 5.5 共享完整 Key 阈值与执行调度 . . . 32
- 5.6 Lane 协作的残差评分与最终 Top-k . . . 35

5.7 作为独立原语的 IVF 探针选择 36

6 工作负载级精排预算校准 38
6.1 固定精排预算为何可能浪费 38
6.2 无需 GT 的采样输出一致性 39
6.3 预算搜索、嵌套性与并列安全 41
6.4 预算基线及其回答的问题 42
6.5 留出风险、校准成本与回本 43
6.6 策略失效模式与适用范围 44

7 实验评估与研究发现 45
7.1 最终实验清单与对照实验组 45
7.2 现代 GPU ANN 的召回率—QPS 前沿 46
7.3 笛卡尔查找表、打包加载与 lane 协作精排 51
7.4 全局 top-c、共享界与探针选择的正确性及消融 53
7.5 留出查询上的预算质量、吞吐量与回本 54
7.6 批大小、扫描组调度与工作负载偏斜 61
7.7 冷构建、常驻显存与已验证规模 63
7.8 数值稳健性、异常与召回率归因 66

8 讨论、局限与未来研究 69
8.1 三项跨层认识 69
8.2 两项深入的负结果案例 70
8.3 内部与外部有效性威胁 71
8.4 可检验的后续研究方向 72
8.5 有条件的部署建议 73

9 结论 74
9.1 四个研究问题的答案 74
9.2 系统原则与展望 74

参考文献 76

A Reproducibility and Experiment Manifest 80
A.1 Source and Build Environment 80
A.2 E1 Measurement Details and Review Diagnostics 81
A.3 Budget Harness Contract and Diagnostic Coverage 82
A.4 Data and Trained States 83
A.5 Result-to-Figure Mapping 83
A.6 Supplementary Evidence of 28 October 84
A.7 Strict Selection and Small-Batch Follow-ups 85

B 补充结果与正确性检查 87
B.1 完整参数记录 87
B.2 正确性与数值证据 88
B.3 次要结果与证据状态 88
B.4 材料获取 89

C 贡献与工具使用记录 90
C.1 研究角色与继承工作 90
C.2 生成式 AI 协助 90
C.3 核对、可追溯性与责任 90

插图目录

1.1 问题、机制与证据 2
2.1 候选与精度的层级流水线 5
3.1 主机与设备计时边界 11
4.1 构建与查询架构 14
4.2 按列表排列的物理索引布局 18
5.1 查询阶段与选择边界 24
5.2 打包后的候选并行访问 28
5.3 局部覆盖与截断式两两合并 30
5.4 共享完整键过滤 33
5.5 十六 lane 残差精排 35
6.1 精排预算影响的工作范围 38
6.2 工作负载级校准与评测 40
6.3 嵌套与非嵌套候选池 42
7.1 E1 召回率与吞吐量前沿 47
7.2 TF32 与 FP32 精度前沿 49
7.3 小批次竞争区间 50
7.4 笛卡尔查找表分组 51
7.5 打包访存与精排对照 52
7.6 不同样本大小的留出风险 57
7.7 校准回本估计 59
7.8 有限池风险预测与实测 60
7.9 校准后复用的直接累计时间 61
7.10 批大小与执行区间 62
7.11 已记录的构建成本 64
7.12 常驻显存分解 65
7.13 大数据集搜索前沿 66
8.1 Mailman 与残差界的负面结果 71

表格目录

1.1 技术贡献与证据边界 . . . 3
2.1 设计选择与研究缺口 . . . 8
3.1 研究问题与实验对照 . . . 9
3.2 数据集与训练配置 . . . 10
3.3 E1 主要调参维度 . . . 12
3.4 论点与证据对应 . . . 13
3.5 证据类别与结论范围 . . . 13
4.1 残差训练迭代诊断 . . . 17
4.2 单向量存储模型 . . . 19
4.3 构建与查询对象生命周期 . . . 20
4.4 各构建阶段的行数组显存模型 . . . 21
4.5 已归档的 pilot 诊断 . . . 22
4.6 原始输入 FP64 重编码审计 . . . 23
4.7 表示契约与审计覆盖 . . . 23
5.1 查询阶段的选择契约 . . . 25
5.2 库默认值与评测设置 . . . 25
5.3 笛卡尔查找表的资源权衡 . . . 27
6.1 预算对照及其作用 . . . 42
6.2 预算解释与复用的适用条件 . . . 44
7.1 实验与证据分组 . . . 45
7.2 召回率 0.97 的跨系统比较 . . . 48
7.3 Vogue E1 扫描路径敏感性 . . . 48
7.4 匹配配置下的 FP32 敏感性 . . . 49
7.5 小批次同召回率吞吐比 . . . 50
7.6 128 探针下的精排收益 . . . 52
7.7 严格计算分数选择检查 . . . 53
7.8 探针选择吞吐量比 . . . 54
7.9 全池预算校准 . . . 56
7.10 采样预算的留出质量 . . . 56
7.11 固定预算的质量与吞吐量对照 . . . 57
7.12 采样校准的经济性 . . . 58
7.13 实测最坏留出召回率损失 . . . 59
7.14 500 个复用批次的直接成本 . . . 60
7.15 集成执行与调度收益 . . . 62
7.16 冷构建与常驻显存 . . . 63
7.17 重复空缓存构建 . . . 64
7.18 E1 显存分配范围 . . . 66
7.19 研究问题的答案 . . . 68
8.1 有效性威胁 . . . 72
A.1 Component and run provenance . . . 80
A.2 Result-to-figure mapping . . . 84
B.1 E1 精确配置数量 . . . 87
B.2 校验覆盖范围 . . . 88
C.1 生成式 AI 协助与证据 . . . 90

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Calibration sample size and allowed disagreeing result slots
$C_\alpha, \mathcal{R}_\alpha$	Valid primary survivor set and returned ID set at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Scan buffer capacity and threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	Number of LUT entries per primary subspace
$\kappa(D, p)$	Packed complete-key encoding for bound publication
ℓ_{ref}	Number of cooperating lanes per refined candidate
D_{raw}	Squared distance between original vectors
D_{P}	Distance under the primary reconstruction
D_{H}	Refined ranking score: reconstructed squared distance plus a query-only constant

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation (distinct from the ADC conference)
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE single-precision storage and arithmetic, with multiplication mode stated separately
TF32	Tensor-core multiplication format with reduced input precision and FP32 accumulation
CV	Coefficient of variation of candidate counts

第 1 章

引言

搜索大型嵌入数据库需要同时决定信息与工作：每向量压缩多少、检查哪些向量、为它们投入多少精度。GPU 可以并行执行许多操作，却不会使这些决策免费。本文以 JHQ-GPU 研究这种相互作用，包括 JHQ 主码 / 残差层次的 GPU 实现与实验分析。

1.1 背景与实际目标

高维嵌入支持图片、文档等对象的相似搜索。所测向量有 768–3,072 个坐标，最大数据库含 17.78 百万向量。穷举距离涉及大量算术，完整向量在索引开销之前就占用容量，例如 Stella 的 FP32 数组本身需要 67.81 GiB（第7.7节）。ANN 允许牺牲部分精确近邻恢复，减少访问和评分工作。

现代 GPU 方法提供不同取舍：CAGRA 等图方法通过导航避开很多向量，压缩 IVF 把选择性列表访问与紧凑距离结合 [17, 29, 32]。由此产生系统问题：指定 recall 下，表示、选择与内存移动把成本放在哪里？短码节省存储，但建表、依赖查找、候选保留和重复 launch 设置可能决定是否也节省查询时间。

JHQ 为这一问题提供合适场景 [15]：紧凑 primary 筛选，残差表示精排短名单。架构暴露两个相关决策：如何高效执行广泛筛选，以及精排多少 survivor。它也允许实验隔离：可以保持表示分数不变而改执行，或复用训练索引而改预算。

实际目标是改善 Recall@10、主机可见吞吐 / 延迟、常驻显存和构建成本的可达组合。评估包括对手更快与方案失败的情况，使贡献成为有条件竞争区间及机制的解释，而不是脱离质量与资源约束的单个提速比。

1.2 问题定义与范围

数据库 $\mathcal{X} \subset \mathbb{R}^d$ 和查询 q 的目标是原始平方 L2 top-k。实现先用 IVF 路由，再按 primary 重建分数 D_{P} 选最多 $c = \max\{k, \lceil \alpha k \rceil\}$ 个候选，最后按残差精排分数 D_{H} 返回。两种分数都不同于原始距离，精排值还带 query 常数（第2.3节）。选择过程的精确性针对计算分数与并列契约，最终 ANN 质量仍要对 GT 实测。

表示继承 JHQ 的平方正交旋转、Cartesian 主码和相对 primary 重建的残差。本文研究 GPU 训练 / 编码、物理放置、查询执行及单个工作负载级精排倍率。倍率以无标签查询输出和大预算参考的一致性选出，预期在稳定工作负载上复用，不是逐 query 学习控制器或绝对 recall 证书。

主跨系统比较在一张 RTX 5090、平方 L2 top-10、batch 1,000 和四个数据集上进行。六数据集支持组件与预算研究，大型索引另测。T5 在内部调度消融之外加入跨系统 batch 8/32 对比（第7.2.5节）。压缩索引常驻单卡，CPU 模型准备和主机查询传输仍属于实际架构。多 GPU、在线更新、主机残差分页及十亿规模部署不在已测系统范围。

1.3 三个相互关联的技术挑战

图 1.1把三个挑战连接到机制与证据。

首先，直接 GPU 移植可能忽略 primary 码本的数学结构。每 8-bit 子空间索引 256 个距离，其 Cartesian 坐标允许精确 16+16 分解，但进一步拆分会增加 lookup 和算术。挑战是利用可分性，同时不假定最小表最快。候选并行 word 访问与残差 lane 合作也需要不同映射。

第二，primary 优胜者可能集中在路由全集任何位置。并行组不能安全地各保留全局需求的一小部分。精确局部保留、全局合并与共享完整分数界提供可靠路线，却增加缓冲、staging 和同步。大 batch 可能已有足够 block，不均衡工作负载仍可能受益于分组，因而正确性与执行决策耦合。IVF 列表选择又是独立于向量保留的原语。

第三，固定倍率可能反复精排无关向量，小倍率则可能过早删除真近邻。大预算参考一致性无需 GT，却不能修复参考自身错误或保证未来查询 recall。样本与预算选择也耗时，必须联合评估质量、校准成本及复用。这些问题组织迭代项目记录，不把观察收益包装成预注册预测。

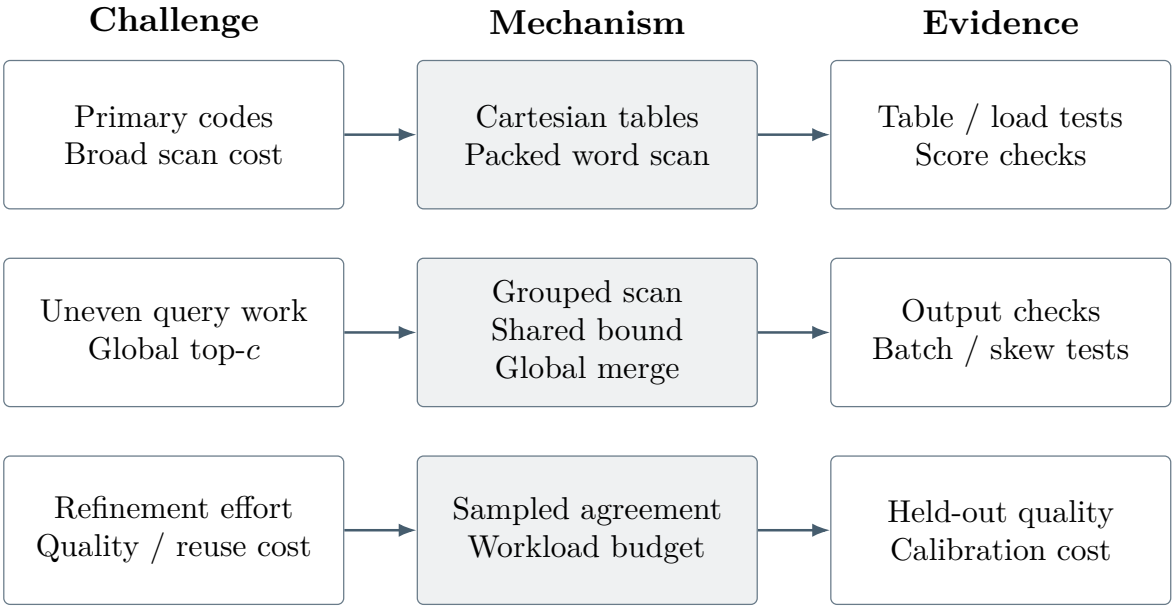


图 1.1. 问题、机制与证据。箭头组织研究问题，不意味着独立测量的提速比可以相乘，也不意味着无标签一致性保证 GT recall。

1.4 研究问题与可反驳主张

RQ1：构建与容量。 如何把 JHQ 层次训练、编码并组织成可搜索的 GPU 常驻索引，实际构建成本和容量是多少？证据为物理布局、train/add 记录和常驻核算（第7.7节）。构建失败或分配超过设备预算会限制容量主张，常驻样本不代表构建峰值。

RQ2：正确高效的执行。 哪些面向表示的评分、保留和调度选择，在满足分数与并列契约时改善 GPU 搜索？LUT、packed load、精排和扫描配对实验连接参考检查与 batch 敏感性（sections 7.3、7.4与7.6）。非并列选择错误挑战正确性，持续减速挑战执行选择。

RQ3：预算质量与总成本。 无标签一致性能否选出有用预算，什么抽样与复用条件下校准值得？互斥查询评价固定预算、抽样 / FULL、枚举及 GT oracle（第7.5节）。越阈挑战可靠性，包含校准后没有正节省则挑战净收益。oracle 是评估对照，不是可部署的无标签决策。

RQ4：竞争区间。 完整系统在共同主机到主机边界下，哪些实测 recall 区间具有竞争力？依据独立调优 E1 前沿和明确显存定义（第7.2节）。int8 图对手、高 recall 反转和未覆盖区间都是答案的一部分，不追求跨硬件、显存预算和请求大小的普遍排名。

第 3 章将这些问题转成受控比较，分别回答建成了什么、执行是否正确、工作负载需要多少工作，以及完整系统有没有有用竞争区间。

1.5 贡献与研究归属

表 1.1列出三项关联项目贡献及继承基础。核心是具体表示–执行–预算组合的适配与评价。JHQ、Cartesian 量化、IVF、bitonic / radix、merge 与 CUDA Graph 仍属于前人工作。

表 1.1. 技术贡献与证据边界。

	继承基础	项目贡献	证据
C1	JHQ 旋转、码本与层次	GPU 训练 / 编码、有界 batch 构建及列表有序 packed 表示	17.78M 向量搜索、常驻 20.44 GiB、重复 train+add 10.54–10.65 s、采样构建峰值 22.58 GiB
C2	Cartesian 可加性与既有 GPU 原语	融合拆表评分、完整键分组保留 / 界、lane 精排与调度	两组 LUT 在 22/24 单元领先；batch 8 调度 2.727×；明确正确性范围
C3	候选扩张和输出一致性思路	工作负载预算规则、有条件选择分析、留出对照与成本 / 风险评估	FULL 的 12/12 单元相对损失达标；小样本揭示风险；四个单元直接测量校准复用

结果使主张具体，但不使之普遍。E1 大 batch 的 0.97 附近插值比较中，JHQ 在三个数据集领先，arXiv 与 RaBitQ 接近平手，其他 recall 区间有对手领先。FULL 最大实测留出损失相对最佳被测臂为 0.0006，$S = 128$ 一个单元有 40/64 次越阈。这些是经验质量结果；10 个 FULL 单元一般嵌套仍有条件，但十二个 FULL 有限网格选择均与枚举一致（第7.5.5节）。OpenAI E1 使用 TF32 GEMM；独立 FP32 敏感性仍保留较小领先（第7.2.3节）。

本 COMP8800 项目由 Rui Luo 在 Dr Mengxuan Zhang 指导下呈现。ADC 合著稿列 Rui Luo 与 Mengxuan Zhang，复用 JHQ-GPU 实现及测量产物 [21]。论文扩展正确性契约、新版实验分析与失败方案讨论，历史稿数字保留各自版本。技术贡献标签界定项目范围，不将合作研究归为单人独有。个人分工和 AI 帮助单列于声明及贡献记录。AI 辅助实现、分析和写作需要明确披露并由作者复核，source ledger 不能替代这一责任。

1.6 论文结构

第 2–3 章建立表示、替代方案与测量设计；第 4 章介绍训练和物理索引构建，第 5 章发展 GPU 评分与选择契约，第 6 章分析工作负载精排校准。第 7 章评估构建、执行、预算质量和竞争区间，第 8 章通过负结果、有效性与可检验扩展解释发现，第 9 章回答研究问题并总结系统原则。附录 A 映射实现和测量产物。

第 2 章

背景与批判性相关工作

ANN 系统结合候选访问、数据表示与选择过程。三者成本互相影响：紧凑编码可能需要昂贵评分器，准确评分也无法恢复此前被排除的向量。本章建立技术词汇，明确继承自 JHQ 的部分，并比较推动 GPU 执行与预算问题的替代方案。

2.1 ANN 目标与评价指标

设带稳定 ID 的数据库 $\mathcal{X} = \{x_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$，对查询 q，精确任务选择原始向量距离最小的 k 项：

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

并列需要指定顺序，或允许多个等价答案。本文使用存储 GT 的 top-10 集合及其固定并列选择。ANN 在允许遗漏该集合成员的同时降低访问与评分成本。Recall@10（式 (3.1)）是经验集合恢复指标，较高实测 recall 不构成最坏情况的乘性距离保证。

本文关注四个成本轴：查询吞吐是在指定 batch 和边界下每秒完成的查询数；batch 延迟是一次调用耗时，吞吐本身不给并发服务尾延迟保证；构建成本包括明确计时的训练和编码；存储区分每向量持久字节与模型、工作区、context、临时分配。实际边界由第 3 章定义。一个方法可以改善某轴而损害另一个轴，同 recall、同显存与同构建投入是不同的比较。

调参产生有限运行点。Pareto 前沿保留 recall 和吞吐不能同时被其他实测配置改进的点。指定 recall 下，最快合格点是实际可用配置，插值则估计邻近点之间的值。未覆盖区间既不支持优势，也不证明不可达。比较系统使用不同码精度与索引大小时，这些区别尤其关键；估计器和端点容差见第3.4节。

2.2 IVF、PQ、ADC 与残差精排

IVF 围绕粗质心分区，列表保存归属于各质心的向量。查询选 n_{probe} 个列表，得到含 n_c 个候选的路由全集 $U(q)$。少访问列表可以省工，却可能在精细评分前排除真近邻。质心用于路由，其距离选择与选择数据库向量是两个不同问题。

PQ 把向量分成 M 个坐标子空间，每个子空间用小型学习码本的标签表示 [16]。若 $\widehat{x}_m = \mathcal{C}_m[z_m]$，ADC 保持查询不量化，计算

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

查询表预计算子空间距离，扫描执行 code load、查表与累加。乘积存在于子空间之间，学习子码本内部的质心未必还有坐标乘积。这决定第 5 章的精确内部拆表是否成立。

这里的“残差”有两种含义。IVF-PQ 常编码 $x - \mu_{\ell(x)}$，粗质心参与重建；JHQ 用 IVF 路由，却相对旋转后的 primary 量化重建定义残差。两者依赖不同；给 JHQ 残差重建再加

IVF 质心，会加入编码器未减去的量。原 PQ/ADC 框架及 GPU 实现也支持扩大候选后进行更精确的 refinement [9, 17]。

在至少有 k 个有效路由候选的分析范围内，JHQ 保留有效 primary 集合 $C_\alpha(q)$，容量上限 $c(\alpha) = \max\{k, \lceil \alpha k \rceil\}$，精排返回 $\mathcal{R}_\alpha(q)$：

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

与 GT 求交得到各阶段覆盖对最终 recall 的上界，却不能定量归因各阶段损失；后者需要固定候选全集的反事实评分。精排可以改善保留向量之间的次序，无法恢复被排除 ID。

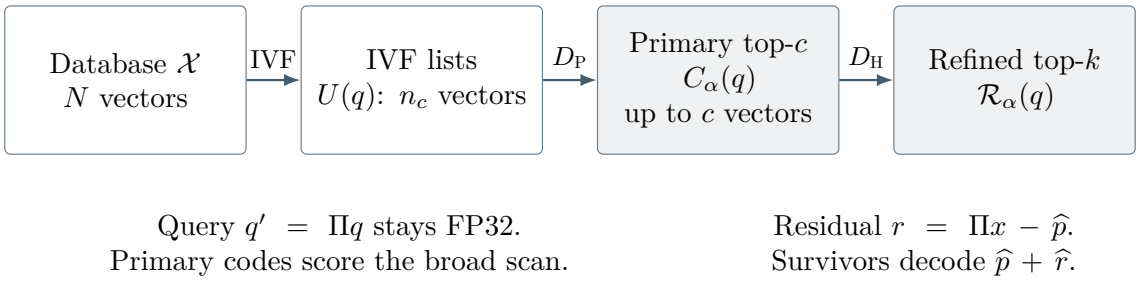


图 2.1. 候选与精度级联。IVF 选列表，primary 选向量，只有 survivor 读残差。层次结构是继承的；图定义逻辑集合与评分精度，不表示实测 recall 损失或 GPU 内核边界。

忽略路由与选择，广泛查表评分需要 $O(n_c M)$ 次 lookup，残差精排需要 $O(cd)$ 坐标工作。这一不对称推动选择性精度，却不能预测延迟，还必须考虑内存、算术、并行及候选选择。增加 probe 改变 U，增加 α 改变该全集内的 survivor；把二者都称搜索深度会掩盖差异。

2.3 继承的 JHQ：旋转、Cartesian 编码与假设

2.3.1 继承部分

JHQ 结合正交变换、解析 Cartesian 主码和较高精度残差层 [15]。本文变换为平方矩阵：$\Pi \in \mathbb{R}^{d \times d}$，$\Pi^\top \Pi = I$，$y = \Pi x$、$q' = \Pi q$，因此

$$\|q - x\|^2 = \|\Pi(q - x)\|^2. \tag{2.4}$$

原工作称其为 orthogonal JL 变换，但此配置不降维；实数距离精确保留，后续量化引入近似。实际 FP32 正交残差与 GEMM 误差需要独立检查。

把 y 分成 M 个 $D_s = d/M$ 维子空间。每子空间 B bit，共 $K = 2^B$ 标签，由 $L = 2^{B/D_s}$ 个标量电平组合；实现要求 Cartesian 参数使 L 为整数。令 Φ 为标准高斯 CDF，构造器使用

$$a_i = \sigma \Phi^{-1}\left(\frac{i - 1/2}{L}\right), \quad i = 1, \ldots, L, \qquad \mathcal{C}_m = \{a_1, \ldots, a_L\}^{D_s}. \tag{2.5}$$

所测 $B = D_s = 8$ 时 $L = 2$，$\mathcal{C}_m = \{-a, +a\}^8$，$a \simeq 0.67449\sigma$，每字节记录 8 个坐标符号。实现从向量能量估计 σ；单位归一化输入对应 $1/\sqrt{d}$ 尺度。该表示来自前作，不是 GPU 执行层的发明。

设 $\widehat{p}$ 拼接 primary 码字，残差 $r = y - \widehat{p}$，量化后为 $\widehat{r}$，层次重建 $\widehat{y} = \widehat{p} + \widehat{r}$。所测残差层每坐标 8 bit，因此主码 / 残差分别占每向量 $d/8$ 与 d 字节，元数据另计。选择性精排改变读多少残差行，不改变存多少行；第 4–5 章把继承的层次搜索实现为常驻 GPU 流水线。

2.3.2 模型假设与分数语义

随机旋转可以分散坐标能量，为共享高斯启发标量模型提供动机，却不能强制一份实际数据库具有独立零均值高斯坐标。固定旋转下，经验均值 μ 变为 $\Pi\mu$，且 $\|\Pi\mu\| = \|\mu\|$。对随机旋转取期望不等于把固定语料居中。即使球面均匀点，各坐标也受固定范数约束而相关。因此等距性是代数保证，分布近似是模型假设；第 8 章的居中编码 pilot 研究其实践后果。

标量最优性的术语也需精确。实现 primary 电平是高斯中分位点；恰好高斯、零阈值的一比特量化器，平方误差最优重建电平应为条件均值 $\pm\sigma\sqrt{2/\pi} \simeq \pm 0.79788\sigma$。因此继承的分位点构造本身不证明 MSE 最优。固定符号及正常数尺度时，a 不改变 primary 排序：

$$D_{\mathrm{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1, +1\}^d. \tag{2.6}$$

它仍影响残差重建。本文依赖可分码本与实测质量，不依赖普遍高斯或最优量化器主张。

最后区分三种距离：D_{raw} 使用原始向量，$D_{\mathrm{P}} = \|q' - \widehat{p}\|^2$ 使用 primary 重建；实现的精排分数满足

$$D_{\mathrm{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.7}$$

query 常数不改变排序，但改变返回数值。对计算分数精确选择仍保留表示与路由误差，不等于原始向量精确重排。

2.4 本文使用的 GPU 执行概念

CUDA SIMT 把线程组织成 block 与 32 线程 warp，寄存器、block 共享内存及 global memory 服务不同复用范围 [26]。Coalescing 把邻 lane 访问组织成合适事务，不要求逻辑码位更少。primary 扫描把 lane 分给相邻向量，精排则让多个 lane 合作处理一个 survivor 的坐标；两者利用不同并行度，一个映射不必同时适合两阶段。

寄存器、共享状态和 block 几何限制 occupancy。更多驻留线程能隐藏延迟，更多单查询 block 也会重复表与局部缓冲。数据复用足以抵消加载与同步时，共享内存才有价值。这些概念推动第 5 章拆表、packed 访问与 batch 调度，吞吐消融测其组合效果。带宽饱和或具体 cache 瓶颈还需计数器及一致的物理字节模型。

选择本身也是重要计算。Faiss GPU 工作把高效距离与 warp 选择结合 [17]。bitonic、radix 和 merge 都是已有构件，对 k、长度和状态成本有不同取舍。AIR top-k 和 GridSelect 进一步展示全局选择如何取决于工作负载及分数是否流式产生 [36]。本文把这些思路适配到 JHQ 的 probe 与 survivor 保留，不把原语称为新发明。

CUDA Graph 回放降低重复 launch 设置；capture、工作区分配与图形状变化属于生命周期，不是数学打分规则。共享阈值发布同样需要排序和内存访问契约；标量分数正确并不足以证明并发准入安全。第 5 章把正确性条件与性能测量分开，第 3 章区分设备 profile 和主机可见搜索区间。

2.5 GPU 图搜索：SONG、CAGRA 与 BANG

图 ANN 通过有希望的邻域导航替代列表扫描，以避免评分大多数向量。GPU 难点在于“发现邻居”与“选择扩展哪个邻域”之间的数据依赖。SONG 把搜索拆成阶段来暴露距离并行，并控制动态内存开销 [37]。这是替代访问策略，不是 JHQ 固定路由全集的另一种实现。

CAGRA 针对 GPU 并行设计建图与搜索，覆盖小 batch 和大 batch[29]。图连通性与 search width 形成不同于 IVF probe 的调参面。边与向量的存储形成容量取舍；量化向量 CAGRA 同样重要，所以只测 fp32 图索引不足以评价压缩索引。E1 包含二者，在共同边界下报告不同工作区间，不拿跨论文提速比相除。

BANG 针对另一容量区间：GPU 保存压缩向量，CPU/GPU 合作访问主机图 [18]。拆分工作结构获得容量，同时引入传输与协调成本。其十亿规模说明完整常驻之外的替代方案，不构成本论文的容量证据。本文 JHQ 压缩搜索索引保留在单卡，没有相同的主机图执行路径。

因此比较取决于访问规则性、表示与驻留方式，而不是图和量化的普遍优劣。少量查询时图导航可能有吸引力，列表扫描则暴露规则编码访问与 batch 并行。二者都可能受选择或移动限制；目标 recall 下谁更快，必须指定设备、显存和工作负载实测。E1 只测试 batch 1,000，JHQ 内部小 batch 收益不能回答相对图基线的问题。

2.6 量化 ANN、快速查表与其他方案

2.6.1 学习表示与利用表示结构

OPQ 通过学习空间分解和码本降低失真 [12]，其旋转为数据优化；JHQ 随机平方旋转支持廉价解析 primary 模型。取舍涉及拟合成本、重建保真和评分结构，更好的通用学习码本未必保留 16+16 精确分解所需的内部 Cartesian 乘积。

ScaNN 的各向异性量化根据最大内积检索的重要性加权误差，说明重建 MSE 不是唯一有用目标 [14]。其目标与实现环境不同于本文平方 L2 层次流水线。可比的是损失选择：面向排序的表示仍须检验实际近邻保留及执行成本。第 8 章停止的排序研究在 JHQ 内部提供了例子。

2.6.2 小查找表已有充分先行工作

PQ Fast Scan 把依赖 cache 的表改造成适于 CPU SIMD 寄存器的表 [1]；Quicker ADC 推广寄存器查找，引入不规则量化器与 split tables[2]。这些工作说明表形状要与执行指令共同设计。本文的范围是继承 Cartesian 坐标结构所支持的精确分解及其 GPU 实测；CPU SIMD 结果提供概念先例，不是本 GPU 的吞吐。FP32 结合顺序和 cutoff 并列仍需独立数值检查。

RaBitQ 也使用随机化量化与紧凑符号信息，具有特定距离估计器及理论误差分析 [10]；多 bit 扩展以存储换估计精度 [11]。其保证属于相应构造和假设，不能因双方都用旋转与二值组件就转移给 JHQ。JHQ 主符号选 survivor，显式存储的残差提供下一精度层；质量同时依赖截断与重建。

GPU IVF-RaBitQ 把该族整合成 GPU 建库与搜索，包括 GPU 距离计算和融合搜索 [32]，因此是直接相关的现代基线，不是“缺少 GPU 量化器”的背景证据。其融合执行说明紧凑编码还需要有竞争力的访问和选择流水线。E1 纳入多个 bit 预算，其高 recall 结果限制 JHQ 的宽泛优势主张。

2.6.3 精排工作量与自适应停止

候选扩张和重排已经存在于 Faiss 的 `k_factor` 及 cuVS refinement 控制 [6, 9]。Faiss autotuning 可用 GT 标签搜索 recall–时间取舍 [8]；学习式自适应终止预测足够搜索量 [19]，DARTH 围绕声明式 recall 目标组织早停 [5]。这些都是均匀工作量之外的相关替代。第 6 章的具体策略用大预算参考的无标签一致性，为整个工作负载选择一个 survivor 倍率，省去预测器训练，却继承参考的质量局限。这是策略范围，不表示自适应搜索或精排预算是新概念。

2.7 批判性综合与研究缺口

表 2.1按所解决约束比较替代方案。已有文献提供 GPU 图搜索、压缩距离、寄存器查找、快速选择和自适应工作量。因此贡献必须明确表示契约、集成决策及证据，仅说“GPU 版本”不足以定位贡献。

表 2.1. 设计替代及本文较窄的研究缺口。表比较机制，实测排名见第 7 章。

替代	可利用结构	主要取舍	JHQ-GPU 问题
IVF-PQ / OPQ	学习子空间编码与 ADC	失真、训练及子码本坐标耦合	内部 Cartesian 主码支持什么 GPU 评分?
Fast Scan / Quicker ADC	小表与 SIMD	表精度、形状、ISA 约束	精确拆分能否抵消额外 GPU 查找?
SONG / CAGRA	选择性图导航	依赖访问、边存储与建图	常驻层次扫描在哪些区间竞争?
BANG	GPU 压缩评分与主机图	容量与通信	完整压缩常驻能达到什么范围?
IVF-RaBitQ	GPU 编码与融合搜索	bit 预算、估计器与 probe	哪些 recall 区间值得残差层次?
自适应工作量	精排倍率、autotuning、学习 / 声明式早停	标签、参考质量、预测或校准成本	无标签工作负载一致性能否以可接受风险和总成本选工作量?

第一项缺口是具体层次的执行契约：可分表与 packed 访问如何配合精确 primary 保留、并发界发布及残差评分。第二项是跨 batch 和偏斜的执行模式选择，因为拆分工作可能加的比省的更多。第三项是工作负载精排选择的质量与成本核算，明确区分参考一致性与 GT recall、选中臂吞吐与净节省。

这些问题推动固定继承表示、改变执行与工作量。更强学习编码和图导航仍是有意义的替代，但同时改动它们会掩盖组件问题。本文以数学契约、配对实验和竞争前沿评估该受约束设计，并保留分布、并列和校准假设，使结论具体而可检验。

第 3 章

研究设计与测量方法

评估需要把系统干预与可观测的成本或质量变化联系起来。本文结合配对执行消融、留出查询上的预算选择，以及各系统独立调优后的前沿比较。这些实验回答不同问题：更快的内核、有效的校准策略和有竞争力的搜索系统，各自需要不同的对照。本章定义这些对照，以及解释结果所需的证据；第 4–6 章介绍机制，第 7 章报告实测结果。

3.1 将研究问题转化为受控比较

表 3.1明确比较单位，以及哪些观察会挑战各项研究主张。一个*实验单元*（*cell*）固定数据集、索引配置和工作负载设置；各实验臂是在该单元内替换的实现或预算。配对执行测量固定索引状态、查询、probe 数、精排预算和打分表示，除非这些正是指定的干预变量。配对比值为 $Q_{\text{treatment}}/Q_{\text{control}}$；几何平均对指定单元的乘性比值作汇总，各单元等权。

表 3.1. 把研究问题转化为可检验的比较。容量观察与配对执行干预需要不同对照。

问题	干预 / 比较	固定或单独核算的条件	指标与反证
RQ1：构建与容量	GPU 构建；缓存与冷构建生命周期；压缩与原始存储	N, d、表示、路由、构建计时边界	train/add 时间与常驻字节；构建失败或模型与分配差额限定可行性
RQ2：执行	LUT 分组、word 访问、精排 lane、分组扫描、共享界与 probe 选择	配对状态、工作负载、分数及参考输出语义	墙钟 QPS、阶段时间和输出检查；变慢或非并列差异挑战机制
RQ3：预算	抽样 / 全池规则，对照固定 64、100、200、枚举与 GT oracle	同一索引与路由；校准和留出查询位置互斥	留出 recall 损失、选中臂 QPS 与校准成本；越阈挑战可靠性
RQ4：工作区间	独立调优的 JHQ、CAGRA、IVF-PQ 和 IVF-RaBitQ	导出向量、GT、k、设备和主机到主机边界；显存另报	同 recall QPS 与覆盖；其他 recall 处更强的竞争者挑战宽泛优势

研究设计区分机制的实现效果与其原因解释。配对 word 访问消融可以确定该访问路径在所测流水线内的吞吐变化；把变化归因于 cache miss 或 occupancy，还需要硬件计数器。跨系统前沿则比较表示、算法与实现的整体选择，无法单独识别某个内核的贡献。第7.6节提供集成调度比较，避免把不同基线上的组件提速比直接相乘。

这些研究问题组织的是迭代开发记录，并非预注册方案：E1 发现 recall 覆盖不足后扩展了网格，工作负载精排需求的解释也来自预算扫描。低 recall 失利、高 recall 反转和失败方案同样进入评估。与量化参考的输出一致性、相对原始向量 GT 的 recall，按定义就是不同的量；预算规则使用前者，留出评估检验后者。

3.2 数据集、查询、真值与划分

表 3.2列出新版预算、LUT 与规模实验使用的六份数据导出；海报 E1 比较使用前四行。Vogue 是时尚图片嵌入，arXiv 是摘要嵌入；两个 OpenAI 数据集来自 Qdrant 的 DBpedia 导出

[4, 30, 31, 33]。BGE 使用意大利语 Wikipedia 配置，Stella 使用 TREC24 BioGen 语料导出[34, 35]。规模来自准备后的实测文件，而不是当前上游数据卡。

表 3.2. 数据与 JHQ 训练清单。n_q 为实际查询数；$n_t = 39n_{\text{list}}$ 是请求的粗聚类训练样本数。E1 使用前四行；全部使用平方 L2 及 $k = 10$。

数据集	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

准备源码规定 L2 归一化，以及带种子的查询抽取与数据库移除。流式 OpenAI/BGE/Stella 转换器先删除非有限行，对范数大于 10^{-12} 的向量归一化，并用 seed 42 无放回选择查询位置。Stella 查询来自 corpus 抽样，没有使用单独的 65 条官方 benchmark 查询；较短的实际导出得到 999 条查询。这些实验评估向量近邻，不评估带相关性标签的检索。两个 OpenAI 导出采用相同位置抽样规则；补证向量核对支持全部查询及 10,299 行 base 样本的按行对应，但未归档原始 source-ID 连接（sections 7.2与7.2.3）。

同一单元的所有方法读取同一查询和 GT 文件。GT 通过对导出 FP32 向量进行穷举平方 L2 搜索得到；流式转换器在数据库分块之间维护 top-k。评估沿用存储的 GT 顺序及其浮点并列选择。设 $\mathcal{T}_k(q)$ 为 GT 前 k 个 ID，$\mathcal{R}_k(q)$ 为返回 ID 集合：

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

Vogue/arXiv 的 GT 行宽为 100，其余四份为 20，评分只比较前 10 项。返回重复 ID 不会重复得分；越界、重复和跨调用变化分别记录。

JHQ 采用 $M = d/8$、$B = B_r = 8$、8 次粗聚类迭代，并请求 100,000 条残差训练样本。搜索扫描复用 `arms6_cache` 训练状态，重新编码数据库；冷构建从没有该状态开始。源码从数据库抽取训练样本，与校准查询分开。预算实验以偶数查询位置构成 500 条校准池，奇数位置构成 500 条留出池，Stella 留出 499 条。每个单元在 $S = 32, 64, 128$ 下各抽 64 次，FULL 使用全部 500 条。GT 只参与离线 oracle 和留出质量评估，可部署的一致性规则使用搜索输出（sections A.3与6.2）。

准备源码和形状日志支持上述协议，但历史向量文件与训练状态的完整哈希缺失。证据登记区分已检查的准备流程与每次运行数据的不可变身份。

3.3 测量边界、波动与显存核算

主比较从主机内存中的 FP32 查询开始，到流完成后 top-10 ID 与分数回到主机为止（图 3.1）。区间包含 pinned staging、H2D、查询预处理、搜索和 D2H。CAGRA int8 查询量化计入其中，量化器训练和索引构建在区间外。资源句柄与输出缓冲复用。JHQ 回传到 pageable 数组，基线回传到 pinned 数组；CAGRA/PQ 经 Python API，JHQ/RaBitQ 经 C++。解释微小差距时保留这些接口差异。

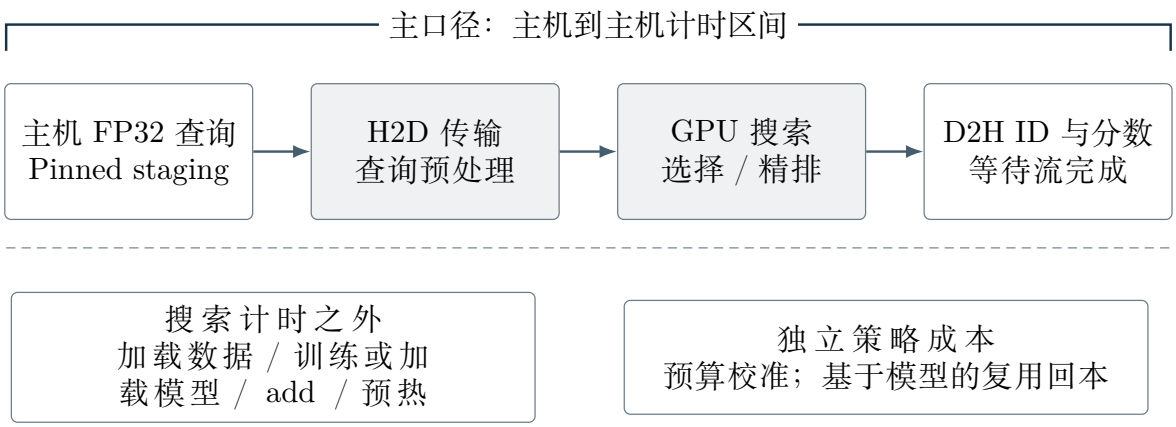


图 3.1. 计时边界。主机区间包含传输和查询相关计算；设备阶段 profile 解释其组成。单独 profile 与同步可能改变执行，因此阶段时间之和不能替代端到端延迟。

第 r 轮有 m_r 次计时调用，每次回答 b 条查询：

$$Q_r = \frac{m_r b}{\sum_{i=1}^{m_r} t_{ri}}, \qquad Q = \operatorname{median}_r Q_r, \tag{3.2}$$

其中 t 以秒计。E1 采用 $b = 1{,}000$ 和 5 轮测量。JHQ 每个变体每轮先做 1 次不计时调用，再通常计时 24 次；基线先做 3 次初始预热，每轮通常计时 24 次，对判定较慢的点减少次数，并保留实际次数。预热使 JIT 和 graph capture 不进入稳态计时。batch 调用的中位 / p95 延迟描述重复 batch 调用，不是逐查询或并发请求的延迟分布。

各轮吞吐的中位数与单次调用延迟中位数的倒数不同。在可得时同时保留二者、各轮最小 / 最大值及基线逐调用轨迹。有效慢调用保留在主估计中。各轮共享训练索引，预算抽样共享查询池：其次数衡量计时波动或抽样敏感性，不等于独立训练重复数。微小配对差异结合离散度和一致性解释，不对扫描单元附加依赖独立性假设的显著性主张。

主环境为 RTX 5090，可报告显存 32,607 MiB，驱动 595.71.05，CUDA toolkit 13.0 和 cuVS 26.08.01。运行获取协作 GPU 锁并保存环境快照；25 核容器配额内使用 16 个 OpenMP 线程。E1 记录功率上限 575 W，快照并不证明锁频或恒定宿主负载。JHQ Release 的 compute_80 PTX 由驱动 JIT[28]，库基线使用安装的构建。辅助设备常驻计时与阶段 profile 分别标注，因为 JHQ 主 API 接受主机查询。

显存核算区分持久编码数组、模型 / IVF 元数据、查询工作区和运行时 / context。进程 used 样本包含索引以外的分配；构建前后差值不包含之后创建的内存池。E1 中减 context 后的 JHQ 总量与 CAGRA 索引差值保留各自定义。组件模型解释常驻总量并显示未归因余量（sections 4.4与7.7），不能代替等显存性能比较；补证以 20 ms 设备采样另测构建峰值。冷 train/add 计时同样不含输入加载、此前的对象构造与主机模型设置。

补证的测量边界。 10 月 28 日实验保留归档 E1 / 预算记录，新增空缓存重复构建、提取 PTX 及 toolkit 反汇编、OpenAI 向量行对应、完整预算轨迹和配对精度干预。GPU 型号及报告驱动 / toolkit 版本一致，但属于另一计时会话。v69 的 OpenAI GEMM 使用 TF32；v72 敏感性在相同训练缓存上改变 add / 查询算术。跨会话基线保留历史标记，补证来源及具体边界见第A.6节。

3.4 基线、调参网格与前沿构造

竞争方法涵盖图遍历（CAGRA fp32/int8）、乘积量化 IVF 与 GPU IVF-RaBitQ [16, 29, 32]，FP32 GPU 暴力搜索提供穷举参考。各方法扫描自己的实现参数（表 3.3），索引配置的存储量和重建精度可以不同。因此主前沿衡量这些网格内指定质量下的可达速度，另行报告显存。

表 3.3. E1 主要调参维度。逐点参数和覆盖扩展保留在原始记录中；各方法使用不同的有限网格。

方法	记录的扫描维度
JHQ	probe 与 α；初始 probe 为 64/128/256，预算为工作负载基准的 0.5/1/2 倍；覆盖扩展最高至 probe 1,024，并增加更大 / 更小预算
CAGRA fp32/int8	图度 32/64；search width 为 1/2/4/8；中间 top-k 容量 32–512
IVF-PQ	8-bit 子码；$d = 768$ 时码长 192/384/768 字节，更高维度时 384/768/1,536；fp16/fp32 查表设置；probe 16–512
IVF-RaBitQ	每维 4/8/9 bit、支持的搜索模式、probe 16–1,024；OpenAI 另有 4,096-list 索引
GPU 暴力搜索	穷举 FP32 平方 L2，采用相同主机计时边界

前沿保留 recall 与 QPS 上不被支配的点：若 $R_a \geq R_b$ 且 $Q_a \geq Q_b$，至少一项严格更好，则 a 支配 b。包络可以由不同索引配置组成。对目标 recall r，实测估计为

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

没有合格样点时无定义。插值估计使用相邻前沿点 (R_a, Q_a) 与 (R_b, Q_b)：

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

归档 parser 先把与目标相差不超过 5×10^{-4} 的前沿点视为直接匹配，包括接近端点的情形；否则只在实测范围内部插值，不对线段外推。差距接近时报告两种估计：前者选实际达到阈值的配置，后者降低网格间距不等的影响；二者都不构成等存储比较。

parser 把失败与非法输出诊断保留在有效点前沿之外。仅有跨调用 ID 变化不会导致删点，原始记录区分集合变化与顺序变化。隔离后的大型基线重跑替代受污染构建，Stella 因 batch 1,000 被跳过的运行不贡献搜索点，决策见第A.2节。论文从全部明确列出的基线族重新构造前沿，包含 CAGRA int8 及不利工作区间（图 7.1）。历史 ADC 与海报图保留各自版本及测量边界。

T5 还保留目标低于某方法前沿最小 recall 时已经达标的最快实测端点；插值未定义不表示更高 recall 的配置不符合目标。

3.5 带版本的主张与证据登记

每项主要实证主张都关联实验族、指标和实际读取记录。机器可读登记为 `data/ch3/audit.json`；第 4–7 章 audit 文件提供详细数值重算。表 3.4规定身份与缺失字段的解释：文件哈希验证分析读取的输入，无法恢复从未归档的可执行文件或训练状态。

表 3.4. 主张与证据记录及现有来源。附录 A 映射实现名称、可执行程序和结果到图路径。

字段组	记录的身份 / 条件	解释
主张与干预	RQ、机制、对照、指标、结果位置、图 / 表	范围明确比较所检验的问题
执行	run-head 或报告版本、harness、编译选项、GPU / 驱动 / toolkit	分析 checkout 与运行源码分开；完整可执行哈希缺失
索引与查询	形状、距离、训练请求、cache 路径、划分 / seed、k、probe、α、batch	记录 cache 名称和形状；完整历史数据 / 状态指纹缺失
测量	计时范围、预热、轮数 / 调用数、参考语义、诊断	每个实验族保留自身协议
分析	原始相对路径与 SHA-256、parser、生成产物、有效性状态	支持离线重新生成报告数值

E1 系统前沿、新版表分组、预算及大型数据扩展与 ADC 论文的旧测量分开。历史 packing 和精排记录回答其原有配对问题。海报在视觉上连接这些机制，但不能替代运行元数据或扩展比较范围。版本名说明来源，机制名说明科学干预。

`scripts/build_ch7.py` 读取归档测量并重新生成实证图和主要数字表，包括图 7.1、7.4与7.12。source ledger 与实验族 audit 保留原始路径及哈希；`scripts/audit_ch3.py` 对照这些记录检查统一协议。源码检查与离线重算建立归档记录和报告数值之间的对应关系。重复 GPU 测量还需要相应 binary、数据和训练状态；各项结果注明缺失的诊断覆盖。附录 A 提供实现与产物细节，并明确可获得的复现实验输入。

3.6 证据强度与有效性控制

表 3.5明确各证据类别能够支持的推断。精确 primary 选择针对实际算出的 primary 分数，原始向量 recall 还受路由和重建影响。

表 3.5. 证据类别及可支持的结论。

类别	支持的推断与范围
形式证明	在明确分数、排序、嵌套和内存前提下的选择或覆盖性质
参考正确性	测试样例及指定键上的一致；区分并列、ID 集合相同与逐位相同
配对 benchmark	固定状态 / 工作负载内的吞吐效果；微架构归因需要额外测量
端到端测量	所测设备、batch 和调参网格内的实际工作区间
模型估计	在明确核算及复用假设下的字节分解或校准回本
未验证主张	解释候选或待补测量；提供研究方向而非实证结果

主要威胁包括共享索引依赖、有限查询池、网格选择效应、接口不对称和单设备范围。相应控制采用配对输入、留出标签、两种同 recall 估计、明确计时边界及保留不利结果。E1 测试 batch 1,000；T5 加入跨系统 batch 8/32，保留达标端点并报告两种 recall 估计（第7.2.5节）。量化参考一致性只给出样本内陈述，推广到总体需要额外假设（第6.2节）。仅按距离处理的 primary 并列仍限制一般预算嵌套结论，涉及 10 个 FULL 单元；十二个 FULL 有限网格枚举均已一致（第6.3节）。这些条件界定第 7 章证据的支持范围，以及第 8 章进一步解释的边界。

第 4 章

系统架构与 GPU 索引构建

本章回答 RQ1：如何从高维向量构造可搜索的 GPU 常驻 JHQ 索引。变换与主码 / 残差表示继承自 JHQ[15]。系统问题在于训练这一表示、编码完整 FP32 数据超出可用设备空间的数据库，并为查询路径提供一致的列表顺序数组。本章依次讨论训练、放置、分配与验证；查询 kernel 细节见第5章，实测比较见第7.7节。

4.1 端到端建库与搜索边界

索引有三个生命周期阶段：训练生成可复用模型状态，添加编码并排列数据库行，搜索使用完成后的数组。实验必须区分这些阶段。训练状态缓存避免重新学习模型，但仍需要数据库编码与列表构建。冷训练加添加，与复用状态后的查询，测量的是不同操作。

主机持有输入向量并提供训练样本，构造方形变换和解析主码码本。GPU 矩阵乘法应用变换；设备训练路径拟合粗粒度 IVF 中心及标量残差码本。添加阶段传输有界批次，编码两级表示，确定 IVF 成员关系，再写成统一的列表顺序表示。因此，主机编排与设备计算共同组成 GPU 常驻搜索系统。

模型状态包含变换、主码码本、残差码本和 IVF 中心。数据库状态进一步包含列表偏移、原始行 ID、主码、残差码及修正项。搜索工作区的逻辑寿命更短：旋转查询、中心分数、探

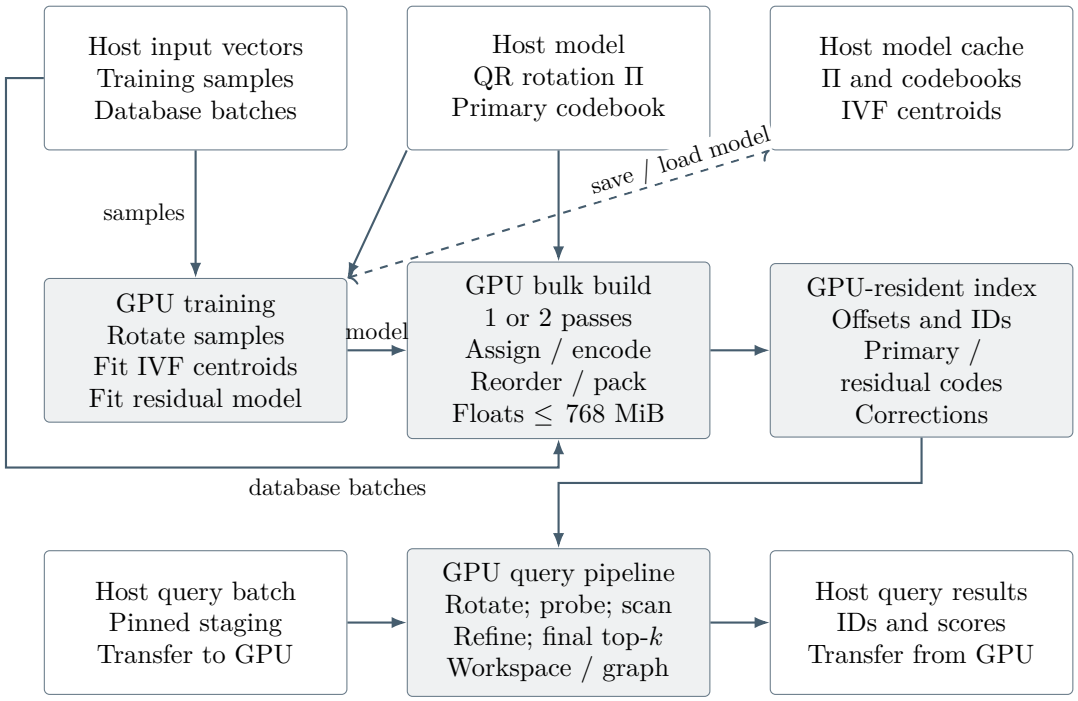


图 4.1. 已实现的建库与查询边界。主机负责采样、正交矩阵构造及解析主码码本生成；评测配置在 GPU 上拟合 IVF 与残差模型。缓存保存训练状态，完成后的数据库表示常驻设备。虚线表示训练状态复用。暂存标注仅约束源码默认的原始 / 旋转浮点缓冲区，其他流水线 scratch 另计。

针偏移、LUT 和保留候选缓冲区在下一批被覆盖，但分配的内存保留复用。捕获的 CUDA graph 属于执行状态，引用这些现有数组。

已核对的 API 通过一次成功的 `add` 构造固定快照，并拒绝后续 add。搜索该快照时物理位置保持固定。数据变更通过重建替代快照处理，形成简单的批量重建生命周期；可变插入 / 删除及快照的事务发布还需要额外索引管理。

搜索接收主机查询并返回主机 ID 和计算排序分数。返回的 D_H 等于重建平方距离加查询常数 $\|q'\|^2$（式 (5.20)），因此保持单条查询内的顺序。用于 range 阈值或跨查询比较前，应先去掉该常数，并继续考虑重建误差。它通过 pinned memory 暂存输入，逐批传输，执行设备查询流水线，再返回结果。主要吞吐实验使用这一 host-to-host 边界；调用 / 重复协议与预热处理见第七章。原始文件加载、训练、添加及调度校准位于稳态搜索 QPS 之外。这些边界使同一索引可供不同查询预算使用，无需将模型构建成本计入每次搜索调用。

该设计从主机内存分批传输构建输入，同时将压缩数据库保留在 GPU。它没有提供依赖磁盘的搜索路径，也不意味着任意大的压缩索引都能放入单个设备。

4.2 训练、旋转与码本生成

4.2.1 变换与采样

实现在主机生成带 seed 的高斯矩阵，通过 LAPACK QR 构造方形正交矩阵 Π。对行优先训练数据 X_t，设备矩阵乘法计算

$$Y_t = X_t \Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

输出保留维数 d。这是正交坐标变换；QR 构造及设备乘法存在有限精度误差，其测量属于第4.6节的 C0。

测试程序按固定步长选择数据库行。主码 / IVF 样本为 Vogue 159,744 行，arXiv 及两个 OpenAI 各 319,488 行，BGE 与 Stella 各 1,277,952 行，均为 $39n_{\text{list}}$。这与 FAISS 默认的每质心 39 训练点警告阈值一致 [7]，是采样规则，不是八轮 IVF 拟合的统计保证。残差训练另用 100,000 行等距样本，成本不随较大粗训练样本增长。

历史受控扫描固定主模型和 IVF 中心，仅改变 Vogue 与 OpenAI-3072 的残差训练样本量，覆盖 $B_r \in \{4, 8\}$ 和 32/128/256 探针。12 个配置中，100K 相对全量的召回率差值最大为 0.0018：11 项较低、1 项相等。增加到中间样本量没有带来单调提升，残差码本身也发生较大变化。这支持有界训练成本的取舍，同时保留可测量的质量损失。该扫描使用 25 轮残差 Lloyd，不能替代之后六个数据集的评测。历史诊断及输入哈希见 `data/ch4/review-audit.json`。

4.2.2 为何保留解析主码模型

主码构造器由样本估计共同尺度：

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

在 $B = D_s = 8$ 时，笛卡尔码本使用 $\{-a, +a\}$ 两级，$a = 0.674\sigma$；每字节表示八个符号位。这保留 JHQ 的解析构造 [15]，支持逐坐标 $O(d)$ 编码，以及第5.2节的小 LUT 精确因子化。对每子空间 256 个码字自由执行 Lloyd 优化，通常会破坏这种可分离性。

固定符号重建 $\widehat{p} = as$，$s_j \in \{-1, +1\}$ 时，

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

当 $a > 0$，这些固定符号码的实数算术主码排序与尺度无关。这一代数性质无需高斯坐标假设。但尺度仍改变残差 $r = \Pi x - \widehat{p}$、拟合残差码本和精排顺序；$a = 0$ 是退化情况，浮点并列仍须处理。

早期前沿诊断说明了取舍。在 Vogue、$M = 96$、$B_r = 4$、128 探针下，早期构建与解析主码构建的 primary-only recall 分别为 0.6618 和 0.6342，而完整系统为 0.9452 和 0.9409。残差精排缩小了观测差距。这是包含残差训练变化的历史跨版本比较，不能作为仅替换主码码本的单因素因果估计。本设计选择的是 GPU 执行成本较低的结构化表示；第七章评价其完整质量与成本前沿。

4.2.3 粗分配的过滤与精算

IVF 中心从变换样本的等距行初始化，执行配置的八轮更新。分配按 $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$ 排序。源码提供 FP32 与加速分配路径；满足通常的维数约束时，默认用 int8 乘积过滤，再用 FP32 重算有歧义的选择。

引理 4.1 (具有认证分数区间的分配)**.** *令 $\widetilde{y}, \widetilde{\mu}_\ell$ 为反量化近似，$e = y - \widetilde{y}$，$f_\ell = \mu_\ell - \widetilde{\mu}_\ell$。保持中心范数精确时，近似分数 $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ 的实数算术误差满足*

$$E_\ell = 2\big(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|\big). \tag{4.4}$$

若每个目标分数均处于有效区间 $[L_\ell, U_\ell]$，仅丢弃 $L_\ell > \min_j U_j$ 的中心，并精确处理剩余分数，则保留最小值及约定的并列规则。

证明. 将内积误差展开为 $\langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle$，应用 Cauchy–Schwarz。被丢弃分数大于某个保留分数的上界，因此不能取得最小值；保留区间相等情形则不会丢失可能的并列者。 □

实现用保守范数界与 FP32 余量常数实现这一过滤与精算模式。引理说明有效界为何允许精确排除；若要求与每种 FP32 累加顺序逐位一致，还需要覆盖相应算术误差的界。已归档诊断对比 FP16 路径，在 Vogue、BGE、Stella 尾批次分别记录 0/14,838、6/31,748、3/16,359 个分配差异。这是有限实现证据，不是完整 FP32 参考认证；冷建日志也未追踪实际执行的分配精度。

4.2.4 残差拟合与迭代预算

IVF 拟合先于残差拟合。后者用固定主模型编码样本，形成 $r = \Pi x - \widehat{p}(x)$，独立于分配的 IVF 中心。每个子空间拟合一个含 $K_r = 2^{B_r}$ 项的有序标量码本，由其 D_s 个坐标共享。GPU 估计器排序残差值，再执行一维 Lloyd 更新；评测配置的上限为 2,000 轮。

表 4.1支持质量收益递减下的固定预算选择：2,000 相对 25 轮提升 0.0063–0.0076 recall。后续变化可能超过 BGE 与 Stella 已报告的两次运行波动，因此该诊断未确立统计上严格的

表 4.1. 历史残差迭代诊断（100K 样本）。Recall@10 从 25 到 2,000 轮明显提升，此后变化较小且不单调；这些不是第七章最终基准点。

数据集	上限 25	上限 2,000	上限 5,000	上限 40,000
Vogue	0.9770	0.9846	0.9843	0.9845
BGE	0.9514	0.9585	0.9592	0.9586
Stella	0.9852	0.9915	0.9921	0.9916

平台。诊断还观察到稀疏尾部 bin 持续变化；这是对所实现浮点更新的观察，不是精确 Lloyd 下降定理失效。包括 Stella 在内的冷日志达到上限，因此将训练描述为有预算的迭代拟合。早期协议在上限 20,000 时，Stella 全量与 100K 样本的结果分别为 0.9921 和 0.9920。

4.2.5 构建工作量与观测不均衡

稠密 QR 一次成本为 $O(d^3)$，旋转为 $O(n_t d^2)$。直接粗分配每轮为 $O(n_t n_{\text{list}} d)$。解析主码构造写入 $O(MKD_s)$ 个值；残差提取访问 $O(n_r d)$ 个坐标，再执行排序与标量更新。这些描述工作量，而非预测延迟。补证正常运行的 Vogue 训练为 298–303 ms，归档 1.06 s 不能代表这些重复。同步阶段诊断中，BGE / Stella 训练约 83% 在 IVF k-means（约 2.97 s）。诊断强制同步且 add 仅用一个缓冲区，其阶段合计须与正常重叠构建分开（第4.5节）。

最终列表不均衡直接影响扫描调度：第五章使用实测候选数 CV，Vogue 已归档的 0.34–0.62 触发分组。八轮 IVF、等距种子和有限采样描述建库策略，但各自对不均衡的因果贡献尚未单独隔离。

被测实现的算术精度。 评测实现 v69 使用 FP32 矩阵存储，但在 $d \geq 1{,}536$ 时启用 cuBLAS TF32 tensor-core 算术：编译期 faithful 分支将覆盖值置空，随后落入按维度选择的默认分支。这影响两个 OpenAI 数据集在训练、add、查询时的旋转及质心 GEMM，环境变量无法覆盖该编译分支。因此 FP32 存储不代表完整 FP32 乘法。FP32 变体 v72 修正了这一开关；敏感性实验复用同一 TF32 训练缓存，只改变 add / 查询算术，没有以 FP32 重新训练模型（第7.2.3节）。

4.3 IVF 分配、ID 与物理内存布局

4.3.1 从数据库 ID 到列表顺序位置

输入行 ID 为 $i \in \{0, \ldots, N-1\}$。粗分配根据计算所得中心分数，将变换后的向量放入某个列表。数学分配目标是

$$a(i) = \operatorname*{arg\,min}_{0 \leq \ell < n_{\text{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

具体实现使用选定的数值分配路径。主码与残差表示编码不减去 $\mu_{a(i)}$；列表成员关系独立于重建，限制之后的搜索集合。

构建器将分配结果与输入 ID 序列共同排序。令 $\rho(p)$ 为排序后物理位置 p 对应的输入 ID，o_ℓ 为列表 ℓ 的起点。完成后的数组满足

$$0 = o_0 \leq o_1 \leq \cdots \leq o_{n_{\text{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

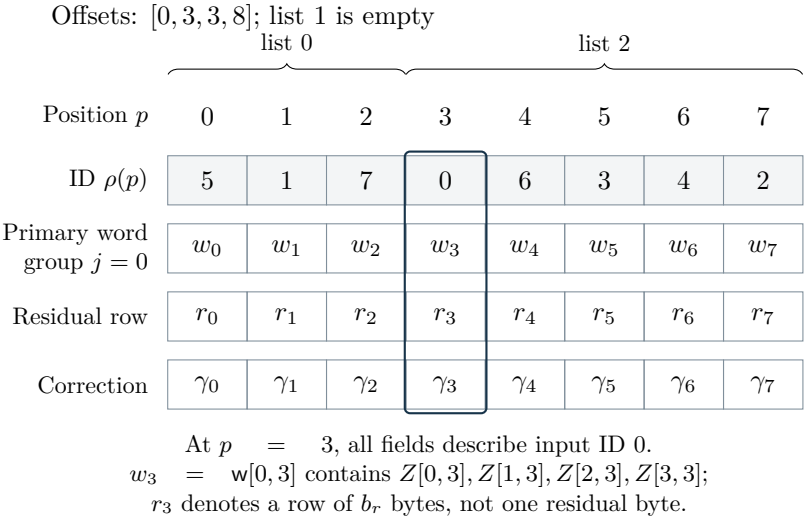


图 4.2. 同一物理位置连接 ID、打包主码 word、残差行与修正项。示意的八行索引偏移为 $[0,3,3,8]$，其中有一个空列表。主码 word 打包同一向量的四个子空间。数值与列表大小均为示意。

偏移通过已排序分配值上的 lower-bound 查找获得。相邻偏移相等表示空列表。按分配结果排序不保证不同独立构建中，同一列表内的行具有稳定顺序。位置在完成后的快照中稳定，其含义是全部关联数组共同描述的行 $\rho(p)$。

常驻 ID 数组实现 $p \mapsto \rho(p)$。逆映射 $\rho^{-1}(i)$ 在数学上存在，但搜索无需再常驻一份逆数组。两遍构建器临时构造逆映射，将输入批次散写到最终位置。查询结果把物理位置翻译回输入 ID；第五章在主码并列时使用位置，在最终并列时使用数据库 ID。

4.3.2 精确地址与表示宽度

令 $Z[m,p]$ 表示物理位置 p 在子空间 m 的主码字节。中间的 subspace-major 副本使用字节偏移 $mN+p$。最终打包为

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t,p], \qquad \mathrm{addr}_{\mathrm{byte}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{4.7}$$

这里 $j = 0,\ldots,M/4-1$。word 布局要求 $M \bmod 4 = 0$，启用该布局的构建对其他值抛出异常；另行编译的字节布局保留原表示。每份打包数组占 NM 字节，但打包期间新旧副本同时存活，占 $2NM$。一遍构建器临时 gather 成 vector-major 的 $[N,M]$ 主码数组，再转置和打包，每份旧副本用完后释放。两遍构建器直接 scatter 到 subspace-major 数组，然后执行相同的最终打包。搜索时的 lane 映射见第5.3节。

残差码保持 row-major，每向量占 $b_r = \lceil dB_r/8 \rceil$ 字节。八比特配置中，行 p 的坐标 t 位于字节偏移 $pb_r + t$。四比特路径中，它位于 $pb_r + \lfloor t/2 \rfloor$，偶数坐标在低 nibble，奇数坐标在高 nibble。每物理行另存一个四字节修正项：

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

gather 或 scatter 时，其值必须与对应行的编码共同移动。因此，即使两个编码数组采用不同布局，残差访问仍重建主码筛选识别的同一对象。

表 4.2. 每向量字节数。JHQ 含编码、ID 与修正项；SQ8 仅计向量编码；CAGRA int8 模型计向量编码及 32 个四字节邻居。均排除偏移、模型参数、工作区及其他运行时分配，不代表完整进程总量。

d	主码	残差	ID + 修正	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

这一分离适应工作负载的两种访问模式：主码在列表顺序中广泛扫描，完整残差行仅在选定位置读取。设备保留原始 FP32 向量可提供精确重排，但要额外占用 $4Nd$ 字节，并改变所报告的搜索设计。这里精排压缩重建，质量影响由第七章评估。

4.4 常驻存储、工作区与索引生命周期

4.4.1 存储字节与分配字节

八比特残差配置具有 int8 量级的常驻表示。主码每子空间一字节时，行载荷为

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

额外八字节保存四字节 ID 与 FP32 修正项。在 $M = d/8$、$B_r = 8$ 时，编码占 $9d/8$ 字节，比 SQ8 向量的 d 字节多 12.5%。因此，无图层次结构是一种设计选择，不能据此声称独有的 int8 压缩优势。

CAGRA 比较使用图度数 $d_g = 32$ 时的 $d + 4d_g$ 字节，与测试程序的 32-bit 邻居 ID 一致。在 Stella 上，JHQ 行载荷为 19.20 GiB，成功的度数 32 CAGRA int8 记录报告索引分配 19,656 MiB（19.20 GiB）。尽管计量边界不同，两者接近仍是必要背景。第七章同时报告 int8、fp32 分配与搜索质量及成本。JHQ 的特点是无图构建和查询可调的残差访问预算；降低 α 不释放常驻残差码。

基本 FP32 模型数组中，变换占 $4d^2$ 字节，物化主码码本占 $4MKD_s$，残差码本占 $4MK_r$，IVF 中心与范数占 $4n_{\text{list}}(d+1)$。选定路径还分配派生中心副本和编码级值。查询工作区包括两个 $[b_{\text{cap}}, d]$ FP32 查询矩阵、$[b_{\text{cap}}, n_{\text{list}}]$ 中心分数、LUT、探针及候选缓冲区。可选残差 LUT 另占 $4b_{\text{cap}}dK_r$ 字节；默认精排直接计算残差分数。分组工作区遵循式 (5.13)，其容量可大于当前扫描所需：已记录的 128 探针样本即使走 query-centric 路径，也保留约 93.8 MiB。

4.4.2 复用、恢复与失效

缓存保存旋转、码本和中心。key 混合模型参数及训练样本的等距校验，但不含代码版本，也不覆盖全部数据库字节。因此，复现需要模型来源，并在实现或训练数据变化时显式使缓存失效。搜索实验复用训练状态，冷建记录则构造独立实例；实例间 recall 接近不能证明模型逐字节一致。

缓存命中仍先执行构造器 QR 与库预热，之后训练调用才检查缓存；再加载 / 上传模型、编码数据并构建列表。Stella 的 6.67 s add 记录只是该恢复路径的一个组件，不是实测进程重启时间。add 调用包括初始工作区分配；调度校准与搜索预热在这一孤立 add 数字之外。

表 4.3. 对象生命周期。大小描述具名数组，不是 CUDA 进程的全部分配；构建阶段见第4.5节。

对象	生产者	位置	生命周期	字节 / 规模
输入 / 样本	加载器 / 测试程序	主机	构建输入；外部持有	原始 $4Nd$；采样子集
变换 / 模型	QR / 训练	主机、GPU	模型及索引寿命	基本 GPU 数组 $4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1)$
训练缓存	训练	主机文件	跨进程；仍需建库	序列化模型状态
输入暂存	添加流水线	Pinned 主机、GPU	add 期间	主机 $4n_b a_b d$；GPU 浮点数组 $8n_b a_b d$，另加 scratch
ID / 偏移	排序 / lower bound	GPU	完成后的快照	$4N + 4(n_{\text{list}}+1)$
编码 / 修正项	编码 / 重排	GPU	完成后的快照	$N(M + b_r + 4)$
顺序 / 逆映射	排序 / 求逆	GPU	仅构建期间	每数组 $4N$，依阶段而定
查询工作区 / graph	搜索 / 捕获	GPU、主机	复用分配，逐批替换内容	依容量而定；graph 引用常驻数组

固定 k 时，改变 α 会改变候选预算，无需重新训练或添加向量。搜索在捕获前扩展不足的工作区，并在实际批大小、候选数、探针数、精排 lane 数、扫描组数或共享界设置变化时重新捕获。校准尝试预算时可绕过捕获。释放索引会销毁 graph 和设备分配；缓冲区由索引持有，在本文评测的顺序调用之间复用。

4.5 构建成本与已测试规模

4.5.1 有界输入批次与两遍构建

一次旋转全部 N 个原始向量需要两个 FP32 设备数组，共 $8Nd$ 字节。构建器改用 pinned 主机内存和设备原始 / 旋转缓冲区分批暂存。若有 n_b 个流水缓冲区，每个最多 a_b 行，则两个设备数组共占 $8n_b a_b d$ 字节。源码默认三个缓冲区、每个浮点数组上限 128 MiB，这一组件至多占 768 MiB。分配乘积及主码编码 scratch 还需额外空间；暂存界不是整个流水线的内存上界。

一遍路径先编码输入顺序的主码 / 残差数组，再 gather 成列表顺序，临时与最终压缩副本共存。容量规则预测内存不足时，两遍构建先分配列表、排序 ID、构建偏移，再次读取输入，旋转、编码并直接 scatter 到最终位置。它用额外一次输入扫描和旋转换取较少的同时存活压缩副本。两遍路径的默认分配可在原始输入与反变换中心之间比较；正交性给出相应的实数算术恒等式。

4.5.2 阶段占用与进程峰值

源码的一遍路径分支估计为

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes.} \tag{4.10}$$

其中依赖 N 的项，是 gather / transpose 时同时存活的十个数组的精确载荷：输入顺序的主码、残差和修正项；分配结果及排序顺序；最终 ID、残差和修正项；vector-major 与 subspace-major 主码副本。共计 $3NM + 2Nb_r + 20N$ 字节。额外 1 GiB 是容量启发式的预留，不是其他全部分配的精确核算。

表 4.4. 行数组的解析占用（GiB），$d = 1{,}024$、$M = 128$、$B_r = 8$。BGE 使用一遍，Stella 使用两遍；Stella 一遍列为反事实容量分析。这些阶段值不含流水线、模型、偏移和运行时成本。

数据集	最终行载荷	一遍 gather	两遍 repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

两遍构建在最终 repack 时，同时保留 ID、残差、修正项和新旧主码数组：

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.11}$$

这一表达式对该阶段的这些数组精确成立。更早阶段还保留流水线 scratch，scatter 时另有临时逆置换。因此，repack 表达式本身不是两遍构建的整体峰值。取整个生命周期最大值时，还需计入模型、排序 / 分配器 scratch、运行时上下文及之后的工作区分配。

表 4.4量化了压缩副本的节省。Stella 冷日志选择两遍，报告式 (4.10) 为 41.6 GiB，尾部可用为 30.6 GiB。另行采样的常驻进程总量包括上下文及查询工作区，BGE 为 11.93 GiB，Stella 为 20.44 GiB。搜索工作集依批 / 探针 / 候选容量而变，见第4.4节。静态行、构建阶段与常驻显存有不同核算边界。补证以 20 ms 间隔采样，BGE 在 add 开头观测 26.08 GiB，Stella 在 repack 阶段观测 22.58 GiB。这是含上下文的设备采样最大值，可能漏掉更短暂分配。

add 早期还有此前遗漏的分配乘积项：

$$S_{\text{dots}} = 4n_b n_{\text{list}} a_b \quad \text{bytes}. \tag{4.12}$$

三个缓冲区、32,768 个列表、每缓冲区 32,768 行时，为三个各 4 GiB 的数组，总计 12 GiB；它们在 add 入口附近与其他流水及数据库数组共存。23.05 GiB 的 gather 载荷属于后续阶段，不能直接再加 12 GiB 作为真实峰值。峰值模型应对各阶段实际存活数组（含流水 scratch）取最大值。Stella 两遍 repack 的较小载荷加模型 / 上下文，解释了其数据库更大却观测峰值更低。

4.5.3 现有测量确立的范围

表 7.16与7.17与图 7.11纳入补证空缓存构建：每个数据集三次输入页已热的正常运行、一次本会话首次读取及一次同步阶段诊断。正常 train+add 范围为 Vogue 0.558–0.566 s、Stella 10.543–10.653 s。Vogue 归档的 1.056 s 训练未复现，三次正常重复为 298–303 ms。阶段诊断强制同步且只用一个流水缓冲区，其合计不混入正常计时范围。

train+add 从构造和样本准备后开始。进程启动到 add 结束的时间戳还覆盖文件打开、主机 QR 与库初始化：Stella 热输入页时 18.2–18.6 s，本会话首次读取为 129.9 s。这一边界结束于调度校准和首次查询之前，并非通用 disk-cold 延迟。六个固定工作点的五次构建召回范围均不超过 0.0012；未重复整条重新训练前沿。

成功的隔离 Stella CAGRA int8 构建在图度数 32 / 64 时，含数据量化为 74.55 / 82.45 s。JHQ 重复构建区间在各自计时边界下更短；完整部署延迟还依赖准备、校准及序列化。

表 4.5. 已归档 pilot 诊断。修正项误差覆盖全部索引行；返回分数误差覆盖采样查询的 GPU top-10 比较。相对分数误差为 $|s_{\mathrm{GPU}} - s_{64}|/s_{64}$，参考包含查询范数常数。

数据集	索引行数	最大修正项绝对误差	最大返回分数相对误差
Vogue	932,328	1.28×10^{-7}	2.66×10^{-7}
arXiv	2,253,000	1.35×10^{-7}	2.38×10^{-7}
OpenAI-3072	999,000	2.77×10^{-7}	4.85×10^{-5}

完成的大规模搜索包含 BGE 10,091,524 和 Stella 17,776,615 个向量。仅原始 FP32 数组就分别需要 38.50 和 67.81 GiB。JHQ 与成功的 CAGRA int8 构建都确立了该 Stella 工作负载的单设备可运行性。部署比较应综合质量、查询成本、重建频率和可用内存，见第7.7节；多种压缩设计均具备放得下的能力。

4.6 索引表示与安全契约

表示契约同时保护数值几何和编码字段之间的对应。一个历史大样本故障说明其实际意义：32-bit 的 $n_t d$ 乘积在 2^{31} 处溢出，导致主码尺度为零、码本坍缩。之后一次运行又因 cache key 不含代码版本，先加载了旧缓存中的错误模型。粗路由正确并不能发现损坏的主表示。修正后的实现分块计算范数并用 double 累积。因此，需要分别检查模型、编码行与放置关系。

4.6.1 已完成的表示证据

10 月 13 日的残差下界 pilot 导出训练状态和编码数据库数组，包括实际 II、主码 / 残差码本、ID、偏移与修正项。导出器将打包 word 解码为逻辑 vector-major 编码。pilot 报告复用 arms6 训练缓存，没有重新训练。三个完整数据库分别为 Vogue（932,328 行）、arXiv（2,253,000 行）和 OpenAI-3072（999,000 行）。

码本检查发现可分离性不匹配、标量级值不匹配、残差码本无序对均为零。独立解码导出的主码 / 残差后，FP64 循环遍历每一行，核对 $\gamma = 2\langle \hat{p}, \hat{r} \rangle$。表 4.5报告各数据集工作负载中的最坏记录误差。这些是导出表示的检查，不是从原始输入向量重新编码。

在七个工作负载、896 个查询实例中，pilot 还对 72,800,590 个路由候选实例，将所提残差下界与 FP64 重建距离比较，未报告违反。该候选集合也支持 FP64 组合分数恒等式检查；它与 GPU 返回分数比较不同，后者仅考察 top-10 槽位。历史程序会跳过路由候选映射中不存在的返回 ID，因此仅看最大误差并不是完整的输出成员关系认证。

离线比较发现，pilot 与评测实现的编码、主码训练、残差训练和索引构建编译单元，在规范化命名空间并去掉 pilot 专用导出方法后完全一致。这支持将 pilot 检查用于共同表示路径，但不能证明之后每个进程采用逐字节一致的训练状态，也未验证大数据集两遍路径：已审计数据库规模较小。逐工作负载数值、原始汇总核对与源码对应可用 `scripts/audit_ch4_review.py` 复算。

4.6.2 独立输入与放置审计

补证导出器在 Vogue、arXiv、OpenAI-3072 与 Stella 上使用共同构建路径。四者全行偏移均单调、端点为 0 和 N，ID 均构成数据库行的置换。对存储旋转矩阵做 FP64 乘法，$\|\Pi\Pi^{\mathsf{T}} - I\|$ 最大逐项误差为 3.96–4.34 $\times 10^{-7}$。采样范数检查使用 FP64 旋转而非 GPU GEMM，因此验证的是存储矩阵的几何性质。表 4.6汇总采样重编码。

表 4.6. 原始输入 FP64 重编码审计。差异数按坐标决策计数，最后一列为残差不一致坐标离决策边界的最大距离。结构检查覆盖全行，重编码采用采样。

数据集	重编码行数	正交误差	主码差异	残差差异	最大边界距离
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

768 / 1,024 维上的差异均在决策边界 1.12×10^{-7} 内。OpenAI-3072 则明显不同：残差不一致占所查坐标的 2.68%，最远离边界 2.67×10^{-5}，与 TF32 旋转一致，因此不能将所有数据集称为 FP32 等价。残差比较以已存主码位为条件，避免重复计入主码翻转。

三个较小数据库在同一模型下强制一遍 / 两遍构建，按 ID 关联的主码和残差字节完全相同，correction 差至多 8.94×10^{-8}；列表分配分别有 1、1、40 行不同。Stella 只验证自然两遍路径。导出器不含 IVF 质心，因此列表成员决策本身仍未独立验证，结构正确不等于已证明分配决策正确。

4.6.3 契约覆盖与剩余检查

构造器要求 M 整除 d、D_s 整除 B；word 打包要求 $M \bmod 4 = 0$。查询侧 LUT 验证器依据实际码字切片检查因子化。位置和 ID 使用 signed 32-bit，较大计数与字节地址乘积需要更宽算术。输入契约假定向量有限、计数可表示，并保留无效哨兵位置。

审计使用导出的旋转与码本，而非凭 seed 重新生成矩阵。剩余工作收窄为：导出 IVF 质心独立验证成员关系，以及构造空列表与对抗尾批次。计算分数选择的严格 CPU 参考已在 T4 网格完成（第7.4.1节），它与 FP64 输入重编码验证不同。第五章以所得对象为前提论证选择，第七章评测质量与成本。

表 4.7. 纳入 pilot 及补证输入审计后的表示契约，区分存储几何、采样算术与行放置的覆盖。

边界	所需性质	证据 / 剩余检查
C0：几何	共同变换，浮点等距误差有界	四数据集已查存储矩阵正交性；区分 TF32 GPU 算术
C1：码本 / 修正	笛卡尔级值、有序残差码本，修正项匹配解码编码	三个数据库 pilot 已完成；全行修正误差见表 4.5
C1：输入编码	存储编码对应预期原始输入行	四份原始输入采样已做 FP64 重编码；量化边界与 TF32 差异已量化
C1：放置	偏移覆盖行，ID 为置换，字段遵循同一置换	全行偏移 / ID 核对及三份按 ID 关联的路径比较；质心成员决策未验证
执行	索引、同步和分数顺序契约有效	已有查询与 sanitizer 检查，有限覆盖见第7.4节

第 5 章

面向表示的 GPU 搜索执行

本章回答 RQ2：如何使 JHQ 的压缩表示支持正确且高效的 GPU 执行。变换、Cartesian 主码码本及残差层次继承自 JHQ[15]；这里关注它们的执行方式。论证沿查询路径，从路由到主码选择，再到残差评分。对每种机制，分别讨论数学目标、实现条件和资源权衡，对应测量见第7章。工作负载级精排预算选择则在第6章单独展开。

5.1 查询路径与四种不同的排序对象

对固定索引，查询先变换为 $q' = \Pi q$，再路由到选中的倒排列表，使用主码筛选，仅对保留候选做精排。最终选择器返回 k 个数据库 ID 及计算所得分数。图 5.1区分运行时的三个选择决策；ground-truth 排序属于外部评价对象。

需要分清四种量：

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

其中 μ_i 是变换坐标中的 IVF 中心，$\widehat{p}(x)$ 为主码重建，$\widehat{r}(x)$ 为解码后的残差。运行时精排分数包含一个仅与查询有关的常数；第5.6节依据实际修正项推导这一点。对固定查询，它保持重建距离的排序，但精排分数和主码分数都不等于原始向量距离。

路由计算中心分数并选择 n_{probe} 个列表（第5.7节）。拼接这些列表得到大小为 n_c 的已访问集合 U。精确探针原语只是按已计算路由分数选择所需中心，不证明 U 包含全部原空间近邻。前缀偏移还给出扫描与调度所需的实际候选数。

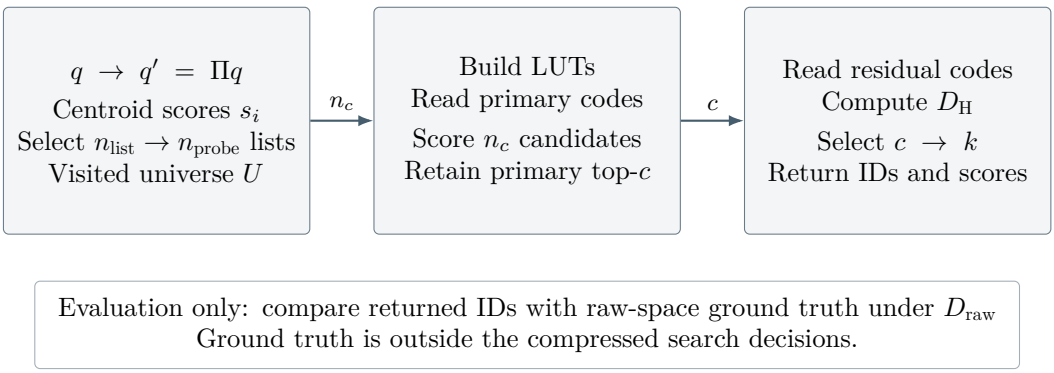


图 5.1. 查询执行及其独立选择边界。路由选择列表，主码筛选选择向量候选，最终选择对精排后的保留候选排序。原空间 ground truth 用于评价，不作为这些搜索决策的输入。

表 5.1. 逐阶段契约。$c_v = \min(c, n_c)$ 为有效主码保留数。“精确”针对约定的计算分数；路由与量化重建决定原空间近似性。

阶段	输入 → 输出	读取数据	主要工作	排序 / 精确性
变换	$q \to q'$	查询；变换	应用 Π	变换（精度见第7.2.3节）
探针	$n_{\text{list}} \to n_{\text{probe}}$ 个列表	FP32 中心；列表长度	评分；radix；名次计数	(s_i, i)；精确探针选择
LUT	$q' \to$ 表	主码本	$MD_s 2^{8/g_{\text{LUT}}}$ 坐标项	实数分解精确；存在 FP32 舍入
主码，$g_{\text{scan}} = 0$	$n_c \to c_v$	打包主码；LUT	$O(n_c M)$ 评分；压缩	仅按距离；分数选择精确，截断并列任取
主码，$g_{\text{scan}} \geq 1$	$n_c \to c_v$	相同主码表示	评分；压缩；合并	(D_{P}, p)；在明确前提下精确
精排	$c_v \to c_v$ 个分数	残差码；修正项	$O(c_v d)$	计算所得 D_{H}；FP32 结合顺序依 lane 而变
最终选择	$c_v \to k$	精排分数；ID	$O(kc)$ 槽位扫描	$(D_{\text{H}}, \text{ID})$；有 k 个有限有效候选时精确

表 5.2. 库默认值与被测流水线配置。环境变量或 setter 可以覆盖默认值。策略参数 -1 请求调度，并不强制执行分组 kernel。

控制项	未配置的库	E1 / 预算流水线
精排 lane	0：标量逐字节路径	16：协作 word 路径
扫描策略	0：查询级	-1：调度器选择 0/2/4/8
共享界	开启，$g_{\text{scan}} = 0$ 时不使用	开启，仅在 $g_{\text{scan}} > 1$ 时使用
调度统计	不自动校准	512 个数据库伪查询

主码筛选构造每查询 LUT，读取已访问向量的主码，再按 D_{P} 保留至多 $c = \max\{k, \lceil \alpha k \rceil\}$ 个候选。固定表分组时，分数计算成本为 $O(n_c M)$，此外还有局部选择和合并。Sections 5.2与5.3讨论表与编码访问，Sections 5.4与5.5建立选择不变量，并分析增加并行度的成本。

残差评分只对有效保留候选读取残差码，进行 $O(cd)$ 坐标工作，并向最终选择器提供 D_{H}（第5.6节）。这一路径使用压缩重建替代搜索时的原始向量读取。实现中的最终 repeated-minimum 选择器为每个输出扫描 c 个槽位，共输出 k 个，另有 block 归约工作。

确定性排序 key 为各类分数附加并列规则：中心选择使用列表 ID，完整 key 主码选择使用物理行位置，最终选择使用数据库 ID。旧的查询级主码路径具有不同的截断并列规则，见Sections 5.4与5.5的限定。因此，分数保持、保留集合一致、有序输出相等及原空间 recall 是不同主张。第七章在各自边界上检查它们，而不把单个 recall 不变当作整条流水线的证明。

表 5.1与5.2明确阶段边界和被测配置。预算 harness 显式设置 16 lane、调度扫描及共享界；仅链接新版本库仍会保留标量 / 查询级默认值。组件消融会按各自协议覆盖相应开关。原始元数据和表 A.1中的机制 / 版本对应表标识这些变体。

5.2 精确的 Cartesian LUT 分解

广泛扫描通过非对称距离计算评估主码重建：查询保持 FP32，存储编码用于选择重建坐标 [15, 16]。对变换后的查询 q'，主码子空间 m 的普通查找表为

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

八比特码要求每个子空间存储 256 项。这里的优化机会来自码本内部的 Cartesian 结构，而不只是向量被划分成 PQ 子空间。

当 $K = 256$、$D_s = 8$ 时，解析码本构造为每维两个标量水平：

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\mathrm{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a\sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

其中 Φ 为标准正态分布 CDF，$s_j(x) \in \{-1, +1\}$。每字节编码八个坐标的符号，高位对应靠前的坐标。当 $a > 0$ 时，主码排序等价于查询与符号向量的内积排序。逐维二水平 Cartesian 积也解释了四组和八组分解在代数上仍然精确。这描述的是解析构造；若替换成非 Cartesian 的学习码本，需要重新检查兼容性。

5.2.1 在不改变量化器的情况下分解

对被测 $D_s = 8$ Cartesian 码本，高半字节 h 选择前四个重建坐标，低半字节 l 选择后四个。因此 $\mathcal{C}_m[16h + l] = (A_m[h], B_m[l])$，两个坐标组之间不存在交叉依赖。定义

$$T_m^{\mathrm{hi}}[h] = \|q'_{m,\mathrm{hi}} - A_m[h]\|_2^2, \qquad T_m^{\mathrm{lo}}[l] = \|q'_{m,\mathrm{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

平方 L2 对互不相交坐标可加，得到

$$T_m[z] = T_m^{\mathrm{hi}}[z \gg 4] + T_m^{\mathrm{lo}}[z \,\&\, 15]. \tag{5.5}$$

只需存储 32 项，而不是 256 项。重建表示和实数运算下的分数保持不变，无需重训练编码、增加量化或改变候选集合。

建表程序从现有码本读取代表码字 $16h$ 和 l，只计算相应坐标片段；另一片段的零编码位与此计算无关。这要求所有共享同一半字节的码字也共享对应坐标片段。`verify_split_lut` 将码字片段与代表码字比较，拒绝不兼容码本。任意学习得到的 256 中心 PQ 子码本不一定满足该条件；仅将编码标签拆成半字节，不会自动分解中心坐标。

T4 已直接测量 FP32 影响：Vogue、arXiv、OpenAI-3072 在 128 探针、$\alpha = 100$ 下比较同一批候选。完整表与 $16 + 16$ 表的 22.6–26.1% 分数比特不同，最大相对误差 4.53×10^{-7}。纳入四 / 八路拆分后，范围为 20.5–29.4% 和 4.64×10^{-7}。256 条查询中，arXiv / OpenAI 的最终 ID 均相同，Vogue 有 2–4 条不同。由于舍入后的主码分数参与精排，即使 survivor 集合相同，末端 ID 也可能变化。因此因子化在实数域精确，在本次 FP32 审计中数值接近但存在实测排名差异（sections A.7与7.4.1）。

5.2.2 分组、建表工作量与资源成本

表分组消融将分解推广到 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ 个连续坐标组，每组位字段与 Cartesian 数字对应。当编码为八比特、$D_s = 8$ 时，每组有 $2^{8/g_{\mathrm{LUT}}}$ 项；总表项和每查询 FP32 存储为

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes}. \tag{5.6}$$

表 5.3. 八比特、八坐标主码子空间的 Cartesian 表权衡。合并加法为组合查表结果所需的代数最小次数，不含子空间之间的累加，也不是实测指令数。

g_{LUT}	表项	FP32 字节	查表次数	合并加法	建表坐标项
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

独立构造表项时，每个子空间计算 $D_s 2^{8/g_{\mathrm{LUT}}}$ 个坐标贡献。相比独立计算完整表的每一项，两组设计将表存储减少八倍、坐标工作量减少十六倍。这是表级成本比，不是应用性能加速预测。

表在全局工作区构造；当完整共享内存占用不超过查询所得的每 block opt-in 上限 $S_{\max}$ 时，由 block 合作搬入共享内存。候选缓冲区容量为 C_{buf} 时，普通精确扫描与分组扫描分别需要

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

这些公式适用于被测扫描，不包含可选融合精排或提前终止的额外工作区。代码恰在对应占用不超过 $S_{\max}$ 时选择共享内存查表。

分组表实验中 $c = 1{,}000$, block 为 512 或 1,024 线程, $C_{\mathrm{buf}} = 2{,}048$。完整表为 96–384 KiB，两组表为 12–48 KiB。NVIDIA 对 compute capability 12.0 给出的每 block 上限为 99 KiB[25]。按该上限，即使 $M = 96$，完整表加查询级缓冲区也需要 114,696 字节，超过容量；最大的两组表加分组缓冲区仅需 65,560 字节。因此，按源码放置规则，这些配置的完整表走全局内存、两组表走共享内存。这是依据启动规则与公开上限作出的推导；本次检查的记录未保留设备属性打印。

四组和八组都只存每子空间 16 项，却增加查表和合并。位提取、查表依赖、搬运、同步和驻留共同决定实测权衡。不同扫描组可能各自搬运同一查询的表。

这里的 $16 + 16$ 表就是八比特符号码的 nibble LUT。PQ Fast Scan、Quicker ADC 及 IVF-RaBitQ 中均有相关小表执行技术 [1, 2, 32]。这一代数联系并不意味着 JHQ 两级分数与 RaBitQ 距离估计器相同。本文贡献在于 JHQ 主码 / 残差结构上的 GPU 执行，包括扫描、选择与精排，以及实测支持的表分组选择。第7.3节中，两组在 24 个单元中领先 22 个，其余差距不足 1%。

5.3 打包编码与候选并行的内存访问

LUT 分解减少每查询的查找表；编码布局决定各 lane 如何取得候选对应的表索引。这是两项独立优化。扫描将相邻 lane 分配给相邻候选出现位置，因此广泛扫描适合按子空间跨候选存储编码，而不是将每个候选的完整主码连续存放。

5.3.1 地址映射与四字节打包

令 o_ℓ 为倒排列表 ℓ 的起点，r 为列表内行号，$p = o_\ell + r$ 为物理位置。若 $Z[m, p]$ 表示主码子空间 m 的八比特编码，逐字节扫描副本将其存于字节偏移 $mN + p$，而候选优先布局使用 $pM + m$。当各 lane 访问同一列表内连续位置时，前者使固定 m 的访问相邻。跨列表边界时，位置可能跳跃，因此不保证每个 warp 都对应一次连续事务。

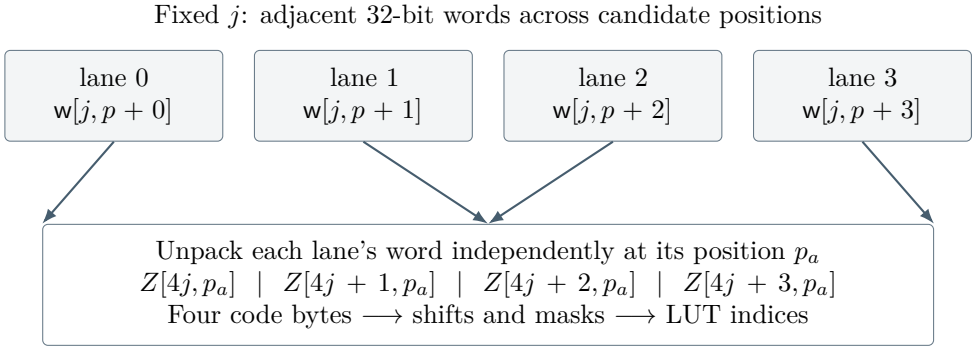


图 5.2. 打包后的候选并行访问，图中仅画出部分 warp lane。固定字组 j 时，连续候选位置访问相邻四字节 word。每个 word 包含同一候选的四个子空间编码，解包保持编码值，随后再进行 LUT 拆分。此图说明地址关系，不表示实测内存事务数量。

打包表示将同一候选的四个子空间编码组成一个无符号字：

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t, p], \qquad \mathrm{offset}_{\mathrm{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

通过 $(\mathsf{w}[j,p] \gg (8t)) \,\&\, 255$ 精确恢复四个编码。每个编码再提供式 (5.5) 中的高、低半字节索引。打包四个子空间，与拆分一个子空间的八比特标签不同，也不等于打包四个候选的编码。

建库路径明确将 $[M, N]$ 字节数组重新打包为 $[M/4, N]$ word 数组。仅把原字节数组重新解释为 `uint32_t`，会合并相邻候选的字节，而不是需要的子空间。实现要求 M 能被四整除，并拒绝不完整的最后一个 word。重排在建库时读写一次 NM 编码字节，旧存储和新存储短暂共存；它位于搜索计时边界之外。

5.3.2 打包访问省下什么、保持什么

在源码层面，一个候选用一次 word load 取得四个编码，替代四次 byte load，然后在寄存器中移位和掩码解包。同一列表连续区间内，32 个 lane 请求逻辑上共 128 字节的 word 窗口。实际事务依赖对齐、活跃 lane 和内存层次。源码 load 操作减少四倍，不代表事务数或耗时也减少四倍。

逻辑载荷仍是每候选 M 字节；重排没有进一步压缩索引。逐字节和 word 两种方案本来就按子空间或子空间组组织扫描副本。因此，历史开关对照测量的是打包布局及其加载 / 解包路径的净影响，不是从候选优先布局进行物理转置的独立贡献，也不能唯一分离指令数、寻址和 cache 的效果。

第7.3节报告 48 个匹配的 byte / word 配置，吞吐比为 1.012–1.658×。每个 arm 只有一个汇总速率，其波动证据弱于较新的重复 LUT 实验。整数表示恒等式是精确的，而最多 10^{-4} 的实测 recall 差异属于整条被测执行路径。方法主张是在保持编码值的情况下减少加载 / 寻址操作，性能解释则受实验测量范围限制。

5.4 通过多 block 扫描实现精确全局主码 Top-c

主码筛选既需要暴露足够的 GPU 并行工作，又不能丢掉本应进入残差精排的候选。以查询为单位的扫描为每条查询启动一个 block；当 batch 很小时，即使每条查询访问大量向量，也只有少量 block 可供执行。GPU 上的选择操作本身就是一个重要的设计问题 [17]。分组扫描

将已访问候选划分给多个 block，再合并各组保留的候选。它保证的是基于已计算主码分数的精确选择；IVF 路由和量化表示仍然具有近似性。

5.4.1 候选划分与确定性顺序

对固定查询，按探针顺序拼接选中的倒排列表，得到 n_c 个候选出现位置。假设每个被访问数据库行只出现一次，搜索期间索引保持不变。对 $g_{\text{scan}} \in \{1, 2, 4, 8\}$，block (q, j) 负责区间

$$U_j = \left[\left\lfloor \frac{j n_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1) n_c}{g_{\text{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

这里的区间表示相应候选，而不是它们的数据库 ID。设备端依据路由得到的候选数计算区间端点。各组候选数最多相差一个；一个较长的列表可以跨越组边界。这种划分平衡的是候选数量，而平均分配列表数量不一定能达到同样效果。每个线程通过探针前缀偏移找到自己的第一个列表，在扫描区间时向前移动列表游标。此阶段不读取残差码。

选择使用的 key 为

$$K_q(x) = \big(D_{\text{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

其中 $p(x)$ 是固定倒排存储中的物理行位置。它在该查询的候选中唯一，不依赖原子追加的先后顺序，也不是最终结果处理并列时使用的数据库 ID。当分数不含 NaN 时，这一定义为不同候选建立全序。局部比较和合并决策都使用相同顺序。要使选择不受划分方式影响，还要求各条比较路径为同一候选计算出相同的分数；排序规则本身不能消除浮点评分差异。

5.4.2 精确局部保留与全局覆盖

用 $\text{Top}_c(V)$ 表示将 V 按 K_q 排序后最前面的 $\min(c, |V|)$ 个候选，其中 $c = \max\{k, \lceil \alpha k \rceil\}$。每个独立组保留完整的局部 top-$c$，而不是分得一个 c/g_{scan} 配额。这一区别不可省略：当 $c = 4$、组数为二时，四个全局优胜候选可能全在同一组。如果每组只保留两个，即使总容量为四，仍会丢掉全局优胜候选。

引理 5.1 (局部缓冲区保留)**.** *设 V_t 为一个 block 截至 chunk t 已评分的候选，B_t 为该时刻完成压缩或过滤后的有效缓冲区。假设完整 key 固定、追加容量充足、压缩精确保留 top-c。令 β_t 为截至该时刻读到的全部完整共享界的最小值，包含初始顶哨兵，则*

$$\text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

关闭共享界时，界始终为哨兵，缓冲区包含整个已扫描前缀的 top-c。

证明. 空前缀满足不变量。新候选若未通过局部准入阈值，则已有 c 个保留 key 排在它之前；这些前驱在之后的前缀中仍存在。共享界拒绝的候选均在 β_t 之后。旧候选若已不属于前缀 top-c，前缀增长也不会改善其名次。因此，每个符合界的新前缀优胜候选，要么已经保留，要么获准追加。精确压缩不会丢弃它：在整个前缀中排在它之前的成员少于 c，缓冲区内也如此。过滤仅删除读到的界之后的 key。对追加、压缩及过滤归纳即得结论。若读取可能返回不同的旧发布值，必须使用历史读取的最小值表述不变量。 □

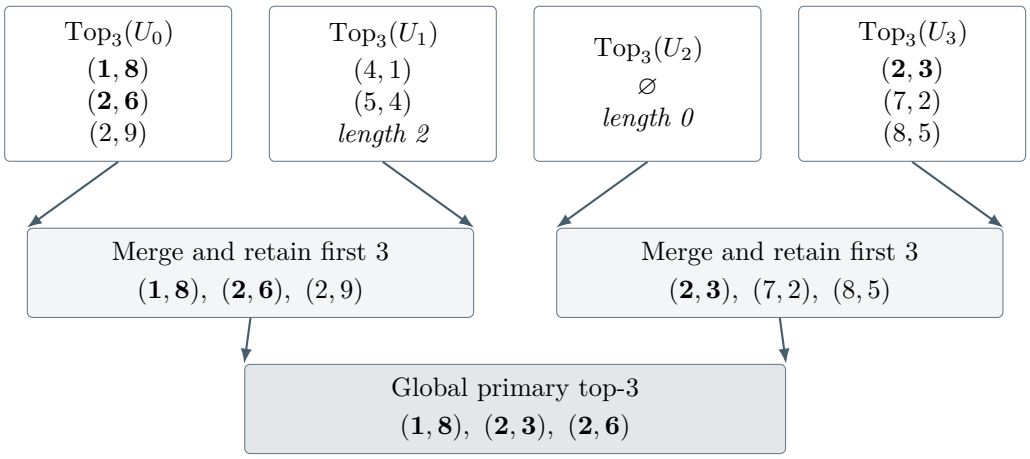


图 5.3. 局部覆盖与截断式两两合并，示例中 $c=3$、$g_{\mathrm{scan}}=4$。条目是 (D_{P},p) key，属于示意而非测量数据。粗体条目是最终全局优胜候选。短组仅贡献有效前缀，空组不贡献条目。每次合并使用同一 key 顺序，并保留至多 c 项。

命题 5.1 (覆盖与截断合并)**.** *在固定 key 和共同全序下，设 U 为已访问候选全集，V 为任一保留子集。若 $\mathrm{Top}_c(U)\subseteq V\subseteq U$，则 $\mathrm{Top}_c(V)=\mathrm{Top}_c(U)$。有序列表两两合并且每轮截取前 c 项保持这一结论。特别地，对互不相交的分组，有*

$$\mathrm{Top}_c\left(\bigcup_j U_j\right)=\mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

候选全集不足 c 时，结论同样成立。

证明. 每个全局优胜候选都在 V 内，其余 V 中的成员均排在全局优胜候选之后，因此从 V 选择可恢复全局结果。对独立分组这一特例，设 x 同时属于全局 top-c 和组 U_j。全局排在 x 之前的候选少于 c 个，因此组内也少于 c 个。于是 $x\in\mathrm{Top}_c(U_j)$，局部结果并集包含全部全局优胜候选。整个全集不足 c 时，所有成员均保留。

对于合并，两个有序列表的前 c 项就是二者并集的 top-c。被截去的候选在该并集中已经有至少 c 个前驱，不可能再进入任何更大并集的 top-c。对合并树的各轮归纳即可得到结论。 □

CUDA 实现在共享内存中维护容量为 C_{buf} 的 key 对缓冲区。扫描 block 含 T_{blk} 个线程，C_{buf} 为二的幂且满足 $C_{\mathrm{buf}}\geq c+T_{\mathrm{blk}}$。压缩操作通过 bitonic 网络排序，保留至多 c 个有效条目；当有效条目达到 c 个时，将最后一个保留 key 记为局部阈值。在此之前，阈值是排在所有有效 key 之后的哨兵。两次压缩之间，仅追加排在阈值之前的候选。即使后续候选继续到来，阈值对应的 c 个见证候选仍使它成为已扫描前缀中最优 c 项的安全上界。

每个 chunk 最多追加 T_{blk} 项。处理完一个 chunk 后，如果下一个 chunk 可能使缓冲区越界，就先压缩。压缩后至多剩下 c 项，$C_{\mathrm{buf}}\geq c+T_{\mathrm{blk}}$ 保证容得下下一个 chunk；不压缩时，显式容量检查提供同样保证。该不变量防止溢出，也不限制一个线程可以遇到多少个有竞争力的候选。固定容量的逐线程保留缓冲区并不具有相同的覆盖保证。

容量论证和精确 bitonic 压缩一起满足引理 5.1的实现前提。启用共享时，命题 5.2保证每个已读界不小于全局第 c 个 key；因此，即使完整局部 top-c 被过滤，引理仍保证该 block 保留自己负责的全部全局优胜候选，再由一般覆盖命题保证合并正确。

另一种方案是先将全部 n_c 个分数 / 位置对写出，再调用独立选择器。这保持相同目标，但增加一个 $O(n_c)$ 中间数组及其写入。这里将打包主码的评分直接送入有界局部缓冲区，以重复局部排序换取 $O(g_{\text{scan}}c)$ 规模的中间输出。基于寄存器的选择也是已有的 GPU 设计 [17]；本轮实验以归档参考路径评估选定的扫描变体。

5.4.3 合并实现、存储与边界情况

当 $g_{\text{scan}} > 1$ 时，每条查询由一个 256 线程的 block 执行 $\log_2 g_{\text{scan}}$ 轮两两合并。有效长度为 a, b 的一对列表输出 $\min(c, a+b)$ 个 key。Merge Path 将输出划分为连续的线程片段：先二分查找每个片段对应的输入偏移，再由该线程顺序合并自己的片段 [13]。所有比较都遵循式 (5.10)，各轮之间通过 block 屏障同步。图 5.3 展示了截断不变量。除划分查找与同步工作外，合并树全部内部节点合计最多输出 $c(g_{\text{scan}} - 1)$ 个 key 对。

两个全局分数 / 位置工作区的容量分别为 $[b_{\text{cap}}, g_{\text{scan}}, c]$ 和 $[b_{\text{cap}}, g_{\text{scan}}/2, c]$，在各轮间交替使用。分数和位置各占四字节，因此二者的总载荷为

$$S_{\text{pairs}} = 8b_{\text{cap}}c\left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2}\right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

各组计数还需要 $4b_{\text{cap}}g_{\text{scan}}$ 字节；配置共享界存储时另需 $8b_{\text{cap}}$ 字节。这些空间位于最终保留数组及其他搜索工作区之外。分配容量在 graph capture 之前增长，之后保留复用，因此以后选择较小的 g_{scan} 不一定释放预留显存。合并只在共享内存中保存列表长度，而不存放全部 $g_{\text{scan}}c$ 个 key。

局部有效前缀由计数界定，而不是仅依靠哨兵分数。压缩时，排序缓冲区填充 $(+\infty, \text{INT_MAX})$；有效位置必须小于这个保留位置。空组计数为零。最终保留数组将未使用槽位填为 $(+\infty, -1)$，残差精排跳过负位置。这两种哨兵用途不同：负位置填充项不能作为有效 key 参与合并。当 $g_{\text{scan}} = 1$ 时，直接写最终保留数组，无需合并。

较早的查询级路径由实现标签 $g_{\text{scan}} = 0$ 选择，表示每条查询一个 block，而不是零个 block。其默认主码选择按分数精确，但仅比较距离的截断规则可能在并列处保留与完整 key 路径不同的成员。有损的 `K_LOCAL=4` 替代路径需要在允许该路径的构建中通过 `JHQ_EXACT_TOPCK=0` 开启；paper-faithful 构建强制使用精确选择。它不是被测默认路径。对仅比较距离的路径，缓冲区论证使用全预序：最终压缩后，保留所有严格小于前缀第 c 小距离的候选，并在该距离处保留足够成员以填满 $\min(c, |V_t|)$ 个槽位。被拒绝的距离已有 c 个距离不更大的保留见证；精确距离排序在每轮压缩保持这一名次性质。并列成员身份不要求唯一。第7.4.1节 的严格 CPU 参考已检查相同计算分数 / 位置目标：55,296 个分组查询实例全部符合完整键顺序；13,824 个单 block 实例全部满足距离全预序，其中 56 个选择了不同截断并列成员。这以有限样例的实现证据补充有条件的缓冲区 / 合并论证。参考在 CPU 上排序 GPU 生成的分数，并未在 CPU 上重新计算原空间距离。

5.5 共享完整 Key 阈值与执行调度

保留完整局部 top-c 建立了正确性基础，但为了得到 c 个全局输出，可能生成 $g_{\text{scan}}c$ 个中间候选。共享界组件减少这部分多余的选择工作，调度器再决定暴露多少扫描并行度。二者均在给定精排预算下工作；选择预算是第6章讨论的独立问题。

5.5.1 带明确见证的完整 Key 上界

当 $|U| \geq c$ 时，用 $K_{(c)}$ 表示全局第 c 小的 key。若某组持有 c 个互不相同且已完整评分的候选，则这些见证中的最大 key τ_j 满足 $K_{(c)} \preceq \tau_j$：全局集合中已经存在 c 个不晚于 τ_j 的候选。共享界初始化为最大哨兵，并按以下方式更新：

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

不足 c 个有效候选时不得发布界。如果整个候选集合不足 c，任何组都无法发布这样的界，所有有效候选保持可接纳。

命题 5.2 (由见证界安全排除候选)**.** *假设 key 固定，每次发布都有 c 个互不相同的候选作为见证，每次读取返回初始哨兵或某个完整的已发布 key。仅在 $K_q(x) \succ \tau$ 时排除候选，不会删除任何全局 top-c 成员，与发布时序无关。*

证明. 每个已发布界都不小于 $K_{(c)}$，这些界的最小值也不小于 $K_{(c)}$。全局优胜候选满足 $K_q(x) \preceq K_{(c)} \preceq \tau$，因此不会被排除。较旧的界仍是上界，只会允许更多工作。此前的安全排除也不破坏后续发布的有效性：任意保留的 c 个互不相同候选仍提供 c 个见证，无论它们是否是该组最好的 c 项。 □

最后这一点很重要。启用共享后，一个组不必返回自己的完整局部 top-c：它可以丢掉局部有竞争力、但已被证明不可能进入全局结果的候选。正确性只要求各组输出的并集仍包含全部全局优胜候选，之后命题 5.1的截断合并论证仍适用。与共享界相等的候选继续有资格被接纳，包括建立该界的见证候选。实际接纳条件为

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

其中 λ_j 是局部压缩阈值，τ_{seen} 是该组最近一次观察到的共享界。

5.5.2 发布、过滤与实现边界

对于由平方和生成的非负 FP32 分数，代码将距离位模式和非负位置打包为无符号 64-bit key：

$$\kappa(D, p) = 2^{32}\,\text{uintbits}_{32}(D) + p. \tag{5.16}$$

此时无符号整数比较与字典序一致。这依赖正零编码，并排除 NaN 和负分数；它不是通用的有符号浮点编码。正无穷排在有限分数之后，全一位模式的初始字又排在有效编码之后。无效位置不得用于发布界。这些是分数定义域与表示的前提，不能从 recall 不变推断出来。

每个 block 的一个线程在每个 chunk 读取该查询的共享界。压缩后，如果有效条目足够，就通过 64-bit `atomicMin` 发布局部第 c 小 key。扫描结束时，再进行一次发布，并通过二分

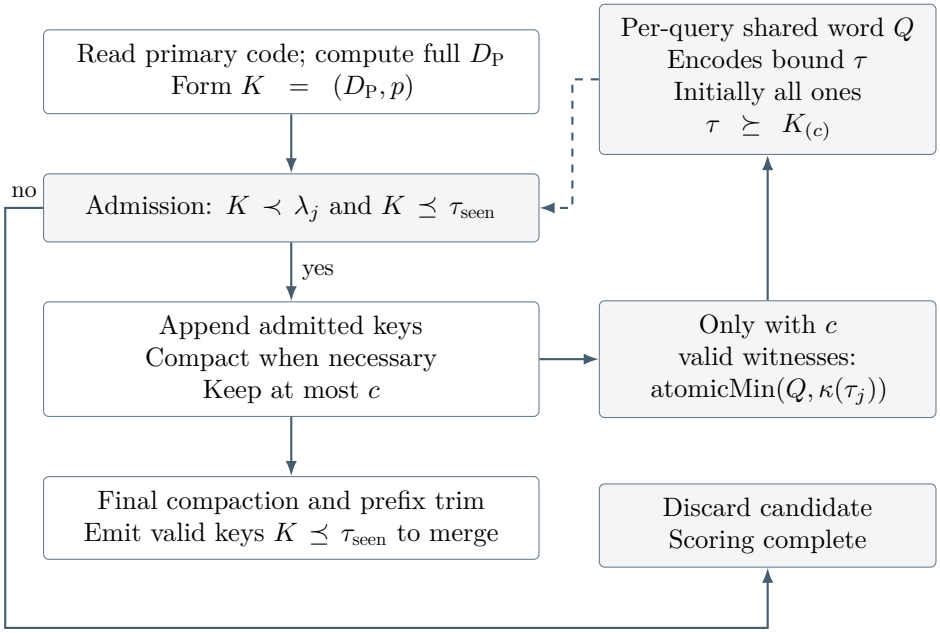


图 5.4. 单个扫描组的共享完整 key 过滤。候选的完整主码分数在接纳之前计算。有效的 c 候选见证集合可以收紧查询级共享界；陈旧但完整的界只会削弱过滤。最终前缀裁剪减少合并的有效输入，但不改变已分配工作区的容量。完整 key 读取这一前提的实现边界在正文中另行说明。

查找只保留不晚于观察到的共享界的有序前缀。各 block 不进行跨 block 屏障同步，也不等待其他组。较晚的更新可能使另一组多输出一些条目，精确合并会安全删除它们。每次搜索前在同一 stream 中重置共享界，扫描 kernel 完成后才执行合并。

归档二进制提取出的 PTX 包含标量`ld.volatile.global.u64` 与 `atom.global.min.u64`。PTX 对作用域兼容、尺寸相同的强访问给出 single-copy atomicity[27]，这些对齐 64 位操作在 PTX 层提供完整 key 读取前提。用 CUDA 13.0 的 `ptxas` 按 `sm_120` 重编同份 PTX，得到单条 64 位强系统读取及 GPU 作用域 atomic-min 归约，归档见第A.6节。toolkit 反汇编支持这一契约，但未捕获驱动 JIT 实际 SASS；论证不单凭 CUDA C++ `volatile`[26]。

图 5.4也明确了优化省下什么。每个被扫描候选仍读取主码，并计算完整主码分数。共享界避免一部分缓冲区追加、局部排序和中间写出，并缩短合并输入列表。它没有实现基于部分距离的提前终止，也没有减少完整主码评分的候选数量。收益还要抵消轮询和原子更新的开销；第7.4节报告的配对加速范围中也包含轻微变慢的情况。

5.5.3 在不改变预算的情况下选择并行度

对 batch 大小 b，分组扫描启动 bg_{scan} 个 block，而不是 b 个。这可以暴露原本不足的并行度，但也重复 LUT 搬运和局部选择，并增加合并步骤，因此固定组数的设计不可能在所有情形下都更好。较小静态配额可以减少中间存储，但要保持覆盖，需要额外证书或回退；pilot 观察到的优胜候选集中现象促使系统选择带见证的阈值方案（第7.4节）。

实际调度器结合并行填充的近似指标和工作负载偏斜统计。令 T_{blk} 为扫描 block 的线程数，H 为 SM 数乘以每个 SM 的最大驻留线程数，$v = \mathrm{sd}(n_c)/\,\mathrm{mean}(n_c)$ 为候选数的变异系

数。规则为

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

偏斜规则可能覆盖填充限制。实验程序以 512 个数据库向量作为伪查询，在固定路由配置下校准 v，不使用 ground-truth 标签。这与第 6 章的采样输出一致性校准不同。校准之后，调度只依赖 batch 大小和已保存统计，不增加每次搜索的主机读回。两个常数在四个 pilot 配置上拟合，之后再到其他配置上评估。一次性伪查询校准不计入稳态搜索时间。补证分别计得主机收集 0.81–7.04 ms、首次校准调用 6.00–27.77 ms、重复中位 2.72–18.13 ms。此启动成本与精排预算校准分开核算（第A.6节）。

当 $g_{\text{chosen}} = 1$ 时，实现选择更快的查询级路径，标签为 $g_{\text{scan}} = 0$。它不改变索引、探针数或精排预算，但继承该路径仅按距离截断时的并列行为。因此，在声称所有调度选择产生相同保留集合之前，必须检查共同全序前提。第7.6节报告独立的集成输出检查，以及调度器的实测收益和 regret。

5.5.4 实际执行路径

对 170 个 SM 的 RTX 5090，公开的每 SM 48 warp 上限给出 $0.6H = 156{,}672$ 线程 [25]。batch 为 1,000、$T_{\text{blk}} = 512$ 时，仅两组的填充代理量已达 1,024,000。因此填充规则选择一组，此时只有足够大的偏斜才强制四组。

主吞吐实验实际使用两种实现，并非全部执行分组扫描。E1 的 38 份运行记录中，九份 $v \ge 0.3$，均为 Vogue，范围约为 0.338–0.622。这些记录共有 132 个运行 / 预算单元：27 个 Vogue 单元调度为四组，105 个 arXiv / OpenAI 单元使用查询级路径。因此，非 Vogue 的大 batch QPS 并未执行分组保留、共享界过滤或合并，但仍包含 LUT、打包访问、lane 精排和探针组件。

Vogue 的调度路径与查询级路径比较中，每 1,000 条查询有 3–15 条仅主码并列成员不同，非并列差异为零，所有记录单元的最终输出均逐位一致。7–11 的较窄范围仅适用于 $n_{\text{probe}} = 128$。

预算实验进一步明确了校准与评测的区别：12 个持出配置中，十个在 batch 500（Stella 为 499）使用查询级扫描，两个 Vogue 配置使用四组。校准样本量 $S = 128$ 时，八个配置使用两组，两个使用四组，两个仍为查询级路径，即 8/12 配置的校准与评测执行路径不同。分组选择主要服务较小 batch 和候选数偏斜的负载，并不存在普遍适用的 batch 256 分界。离线重计数与输入哈希记录在 `data/ch5/review-audit.json`。

该规则是启发式策略，不是 occupancy 计算，也不是延迟 oracle：它没有建模寄存器压力、共享内存驻留限制、与预算相关的压缩成本和合并成本。冻结的伪查询统计也可能无法代表未来查询。记录中的最大 regret 出现在 batch 256，此时某个固定组数方案约快 1.217×。该设计组合候选并行扫描、带见证过滤和工作负载相关执行，并在固定精排预算下测量收益和 regret。后续残差精排与最终 top-k 按第5.6节消费这些有效候选。

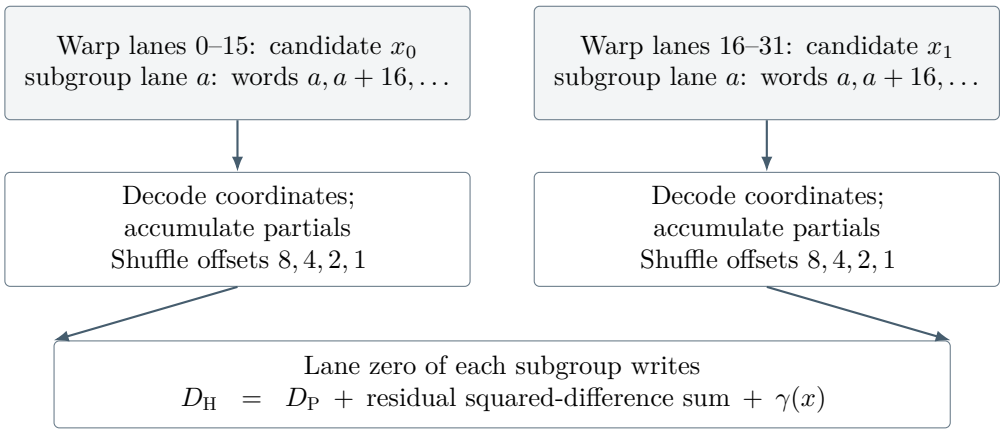


图 5.5. 十六 lane 残差精排。一个 warp 处理两个候选；候选内的 lane a 读取残差 word $a, a+16, \ldots$。残差编码为八比特时，每个 32-bit word 解码四个坐标。子组归约合并部分和，再加上保留的主码分数和修正项。即使子组内 word 访问连续，候选物理位置仍可能分散。

5.6 Lane 协作的残差评分与最终 Top-k

只有主码保留候选才访问残差码。这使系统先对 n_c 个候选进行广泛而便宜的扫描，再执行较窄的 $O(cd)$ 评分阶段。如果对每个访问候选读取残差，坐标计算量将趋向 $O(n_c d)$。lane 协作加速已选候选，不负责决定精排预算。

5.6.1 重建评分与查询相关常数

令 $\widehat{p}(x)$ 为主码重建，$\widehat{r}(x)$ 为变换坐标中的残差解码。编码器存储 $\gamma(x) = 2\langle \widehat{p}(x), \widehat{r}(x) \rangle$。实现中的精排排序分数为

$$D_{\mathrm{H}}(q, x) = D_{\mathrm{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

第二个等式由展开平方范数得到。因此，该分数按平方距离对层次重建排序，但返回值还包含一个仅与查询有关的常数；它既不是直接计算的原始向量距离，也不是未平移的重建距离。该常数不影响固定查询的排序。即使按此分数精确选择，重建误差仍然存在 [15]。

残差坐标使用子空间内共享的标量码本项。标量路径遍历每个候选的各维，累加与 q' 的平方差。查询坐标及在容量允许时的残差码本会搬入共享内存，最后再加上已存的修正项和主码分数。

5.6.2 Lane 到 word 的映射与部分和归约

对 $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$，独立的精排 block 为每个候选分配 L 个相邻 lane。一个 32-bit 残差 word 包含 $d_{\mathrm{word}} = 32/B_R$ 个坐标编码，其中 B_R 为残差位宽。被测八比特路径中 $d_{\mathrm{word}} = 4$，四比特路径中 $d_{\mathrm{word}} = 8$。lane a 读取 word 索引 $a, a+L, a+2L, \ldots$，解码坐标并累加部分和。butterfly shuffle 归约合并 L 个部分和，lane zero 写出精排分数。

启动条件要求残差行按 word 对齐，且 D_s 能被 d_{word} 整除，从而一个 word 不跨越两个子空间的码本边界。所有 lane 都执行 shuffle 序列，包括分配到无效或超出范围槽位的 lane；

它们在输出标为无穷前贡献零。这避免让 warp 参与情况取决于某个候选是否存在。$L = 16$ 时，一个 warp 处理两个候选，而不是每个 lane 各处理一个候选。

实验将两种改变分开。L0 是原来的标量逐字节加载路径，L1 使用 word load，但仍每候选一个 lane，保持其累加顺序。L8/L16/L32 进一步将坐标工作分配给多个 lane，缩短每个 lane 的串行遍历，并在 c 较小时提供有用工作；代价是增加 shuffle，并减少同时处理的独立候选数量。因此 L 并非越大越好。保留候选位置不规则、码本处于共享还是全局内存，也仍影响成本。

5.6.3 最终选择与数值边界

当至少有 k 个有效候选具有有限精排分数、且数据库 ID 唯一时，最终 kernel 按 $(D_{\mathrm{H}}, \text{database ID})$ 精确选择 top-k。每条查询一个 block，读入 c 个分数后反复归约最小值，输出其 ID 和分数，再移除该分数进入下一轮。每轮扫描全部 c 个槽位，没有固定的逐线程候选配额。该契约要求至少 k 个有限有效条目。退化池中，选择器将已选分数设为无穷，但保留有效位置：有限条目用完后，最小的有效 ID 可能以无穷分数重复输出。全无效池则返回无效 ID 和无穷分数。这些情形超出本文评测使用的有限候选池契约。

最终 ID 并列规则与主码保留使用的物理位置规则不同。精确的最终选择器不能找回路由或主码筛选丢掉的候选，也不能消除量化误差。多 lane 归约在选择前改变浮点结合顺序，因此即使实数公式不变，近似并列候选也可能发生排序变化。

第7.3节区分这些影响：L1 在被测用例中逐位一致，多 lane 的距离相对误差最高约为 1.6×10^{-6}。普通预算下 L16 端到端收益范围为 1.015–1.789×；内核级收益更大，不能替代端到端数字。受控的 load-only 与协作变体支持设计结论，而数值等价性需要检查，不能预先假定。

5.7 作为独立原语的 IVF 探针选择

探针选择发生在主码扫描之前，选择的是倒排列表，而不是数据库向量保留候选。对 IVF 中心 μ_i，选择器对以下计算所得 FP32 路由分数排序：

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i \rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

省略共同的查询范数不改变中心排序。与主码平方距离表不同，这些分数可以为负。所需输出是按分数排序的前 $n_{\mathrm{sel}} = \min(n_{\mathrm{probe}}, n_{\mathrm{list}})$ 个中心 ID，并列时较小列表 ID 优先，以及它们列表长度的前缀偏移。

点积继承 cuBLAS 算术模式：被测 v69 的 OpenAI 旋转与质心 GEMM 使用 TF32 输入和 FP32 累积。选择器的精确性针对这些已计算分数；之后的 FP32 `fma` 与确定 key 顺序不会恢复 GEMM 已引入的舍入（第7.2.3节）。

5.7.1 Radix 选择与确定性排序

较早的 bitonic 选择器对逐渐填充的候选缓冲区反复执行 bitonic 排序。radix 替代方案仍为每条查询使用一个 block，但通过四轮从最高有效字节开始的 radix pass 确定截断值。每轮只统计匹配此前已选前缀的 key，再找到包含剩余顺序统计量的 bin。四轮之后，得到 cutoff key 及仍需选择的同 cutoff 条目数。

浮点映射先将正负零均规范化为正零，再对 32-bit 表示 u 作变换：

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

对非 NaN 浮点数，它保持数值顺序，包括负分数和无穷；规范化使相等的零具有同一 key。这与式 (5.16) 的非负分数编码不同。NaN 不属于选择器的排序契约。每轮使用相同、明确舍入的 `fma` 表达式重新计算分数，因此轮次改变不会改变 key。

直方图有 256 个 bin 和 32 份 lane 副本，仅计数器就占 32 KiB。副本将同一 warp 内对热门 bin 的原子更新分开，不同 warp 之间仍可能竞争。这是以共享存储和归约工作换取更分散的直方图更新，不是消除全部竞争。

收集阶段先取得所有小于 cutoff 的 key。如果等于 cutoff 的 key 过多，就按 ID 顺序重新扫描，通过 ballot 和前缀计数选取恰好所需数量、且列表 ID 最小的成员。因此最终并列子集不由原子到达顺序决定。随后，对 n_{sel} 个已选条目分别统计有多少其他已选 key 在 (s_i, i) 顺序中排在它之前，再写入相应名次。这无需排序全部中心，就恢复了确定性的有序输出。

最后，block 前缀扫描生成所选列表长度的偏移和总候选数。当 $n_{\mathrm{probe}} > n_{\mathrm{list}}$ 时，实现通过重复最后一个已选 ID 填充 ID 数组，但将多余条目的长度置为零，不会再次扫描同一列表。这里假设中心集合非空，且请求探针数为正。

5.7.2 成本与独立正确性契约

直方图和收集各轮会重复访问中心分数行；名次统计需要 $O(n_{\mathrm{sel}}^2)$ 次比较。因此该算法不是对任意探针数都具有统一线性成本的替代品。较大的 n_{sel} 同时增加排序工作与已选条目的共享存储，而中心集合较小时也仍需预留直方图空间。Radix selection 是已有选择技术；这里的贡献是它在该查询流水线中的确定性集成及实测执行权衡。

归档探针 checker 报告 1,792 个用例，与早期选择器比较 ID、偏移和总候选数，差异为零。输入分布覆盖大量并列、全部相等、混合正负零、大部分为无穷和极大量级；还覆盖请求探针数大于列表数的情况。这是针对探针原语的有限测试，不是主码 top-c 或原空间精确近邻的证明。

第7.4节给出真实查询一致性检查、阶段计时和端到端对照，也保留了 Vogue 跨进程异常：probe / primary dump 匹配不意味着最终流水线输出必定相同。可选 FP16 路由改变分数精度，属于独立近似；探针高重合度或 recall 不变，都不能使其归入 FP32 等价性主张。

第 6 章

工作负载级精排预算校准

第四、五章确立压缩索引及其 GPU 执行。本章回答 RQ3：一个工作负载需要多少精排工作，选择该预算能否收回校准成本。策略将无标签查询输出与较大预算参考比较，并跨批复用选定的乘数。本章分析接纳规则、有条件的选择定理、实验对照和摊销模型；第七章报告实测结果。

6.1 固定精排预算为何可能浪费

JHQ 先用主码筛选路由候选，再为较小的保留集合读取残差 [15]。对查询 q，令 $U(q)$ 为选定 IVF 列表中的候选。精排乘数决定

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha), |U(q)|\}} U(q). \tag{6.1}$$

这里 C_α 指有效保留候选。实现预留 $c(\alpha)$ 个槽位，候选不足时可用哨兵填充。分析假定至少有 k 个有效候选。增大 α 扩展已访问列表内的精排容量；增大 n_{probe} 改变访问的候选全集。两者均不能自动恢复另一阶段丢失的信息。

较大的固定 α 可能反复精排不改变最终 top-k 的候选；较小值则可能在残差评分前丢弃有用近邻。取舍依赖查询群体、主码重建、列表覆盖及精排分数顺序，因此一个全局默认值未必适合所有数据集—探针数组合。评测网格在 $k = 10$ 时覆盖 $c = 20$ 至 2,000。切换预算复用同一训练索引和常驻残差。降低乘数减少访问的残差行，不释放常驻残差表示。候选缓冲区在容量不足时扩展，之后保留该容量，因此较小预算不代表分配显存按比例下降。

解释性的延迟分解为

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

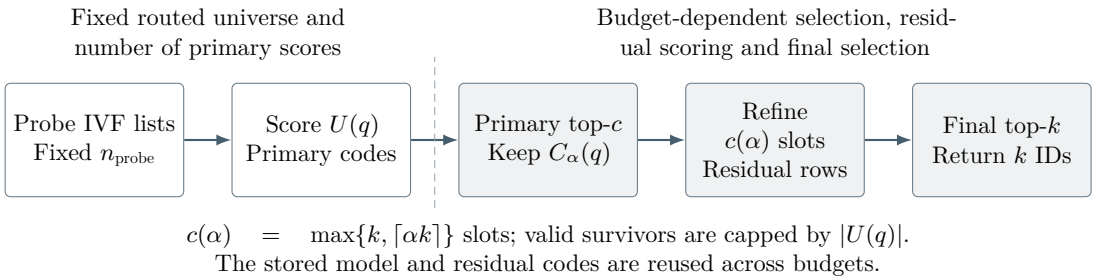


图 6.1. 精排预算改变主码评分之后的工作。固定查询、索引与探针数时，路由候选全集不变。着色阶段依赖保留容量；这是逻辑工作量分解，不是实测阶段耗时。扫描内核还执行随预算变化的接纳和压缩。候选少于 c 时需要填充。

固定批次和路由时，T_{base} 包含传输、旋转、路由和主码分数生成；选择、残差评分及最终选择依赖候选容量（图 6.1）。这是工作量模型，不是拟合的延迟预测器。缓冲容量、launch occupancy 与资源阈值可阶梯变化。小预算时固定成本形成吞吐平台。OpenAI-3072 在 128 探针的十轮交替重测中，$\alpha = 4/2$ 为 142.3 / 142.7 kQPS，未复现归档 145.0 / 128.7 kQPS 的反转（第7.5.5节）。策略最小化接纳预算，不优化有噪声的计时排名。

每个校准试验仍先执行旋转、探针选择和主码评分，再精排。复用索引避免重建或重训练，但不跨预算试验缓存全部查询中间结果。因此，校准成本要计算完整搜索，不能只计两个残差评分工作量之差。

策略为索引、路由配置、k 和查询工作负载选择一个预算，既不训练逐查询预测器，也不为每条到达查询在线决策。扫描分组仍是第5.5节的独立机制；解释预算校准时仍须考虑它实际选择的执行路径。

在自适应 ANN 方法中的定位。 该乘数与 FAISS 的 `IndexRefine::k_factor`、cuVS 精排比例承担相同的候选池规模角色 [6, 9]。FAISS auto-tuning 用 GT 评价准则探索速度—精度工作点 [8]。Li 等从搜索特征学习逐查询终止条件 [19]；DARTH 在执行期间预测 recall，在请求目标处早停 [5]。本方法通过参考输出覆盖率，为无标签工作负载校准一个可复用预算，未训练 recall 预测器。最终评分仍基于压缩重建，而 cuVS 的精排示例重排原始来源向量。这些区别分别定位工作量参数、其选择机制与评分表示。

6.2 无需 GT 的采样输出一致性

令 $\mathcal{R}_\alpha(q)$ 为固定索引在预算 α 下返回的 k 个有效且唯一数据库 ID 的集合，定义参考 $\mathcal{R}_{\text{ref}}(q) = \mathcal{R}_{\alpha_{\text{ref}}}(q)$。降序网格为

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\text{ref}} = A[0] = 200. \tag{6.3}$$

对 S 个校准查询 $\mathcal{Q}_S$，定义

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\text{ref}}(q)|\right), \qquad D_S(\alpha) \le \epsilon. \tag{6.4}$$

评测中的 $\epsilon = 1$ 允许整个样本共缺失一个参考 ID。它忽略结果顺序和距离差异；替换一个 ID 计一次，尽管对称差包含两个 ID。有限网格目标为

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \le \epsilon\}. \tag{6.5}$$

参考预算提供非空回退。目标最小化被接纳的乘数；第6.3节说明二分精确求解的条件。

在 $k = 10$ 时，归一化容忍度 $\epsilon/(Sk)$ 对 $S = 32, 64, 128$ 分别为 0.003125、0.0015625、0.00078125，$S = 500$ 时为 0.0002。因此，增大 S 同时改变覆盖和有效严格度。校准与留出评测使用互斥查询；子样本复用同一有限校准池（第7.5节）。

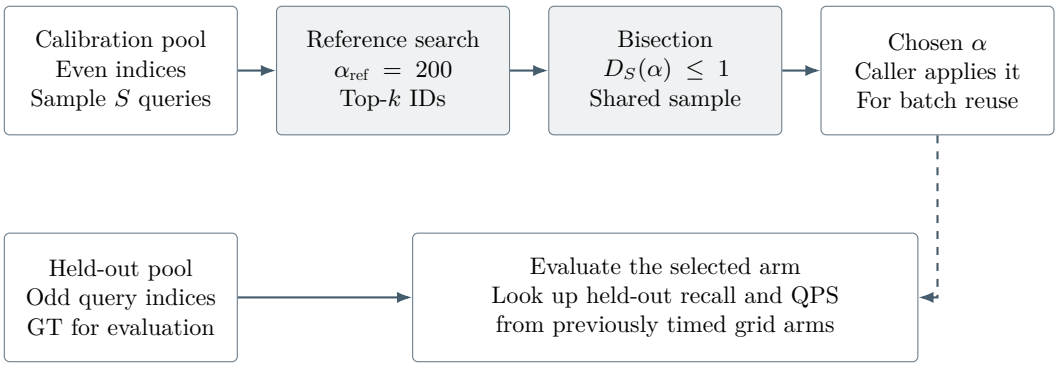


图 6.2. 工作负载级校准及其评测。预算决策使用参考 ID，GT 评价留出质量；所选预算结果从已计时的固定预算中查取。实现边界见第A.3节。

6.2.1 一致性控制什么

在命题 6.1的条件下，不一致数就是主码筛选排除的参考答案数：

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

对有效定长输出集合，令 $r_{\mathcal{Q}_S}$ 为同一批查询的 GT recall。每个缺失参考 ID 至多损失一个 GT 命中，因此

$$r_{\mathcal{Q}_S}(\alpha) \ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}. \tag{6.7}$$

该界仅在样本内、相对参考成立。一致性控制相对参考的额外损失；参考自身质量仍需 GT 评价。第7.8.2节用重建 pilot 量化这一区别。

6.2.2 抽样分辨率与有序检验

令 H_S 为至少缺失一个参考 ID 的查询数。由于 $H_S \le D_S$，接受意味着 $H_S \le 1$。对预先指定预算及 i.i.d. 查询，观察到一次事件时的单侧 95% 精确二项上限满足 [23]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

按查询计，$S = 128$ 和 500 的上限分别为 0.03652、0.009452。将全部 Sk 个结果位看作同参数独立 Bernoulli 是更强假设；相应上限为 0.003701、0.0009484，近似 $4.74/(Sk)$。仅有查询 i.i.d. 不能将查询事件上限再除以 k；两种模型均未为固定池的重复子样本建立置信保证。

当不一致数单调且网格顺序固定时，二分选择被接受的前缀。若逐预算决策是有效水平检验，首个不安全预算的论证可给出无需 Bonferroni 因子的 fixed-sequence 错误控制 [3]。当前一个槽位规则是稳定性阈值；GT 风险保证还需要对目标有效的检验。

有限池扩展。 假设每条查询在目标样本大小的执行制度下具有固定不一致数 $X_q(\alpha)$，且不随子集成员或位置变化。令池大小为 N_{cal}，n_j 为 $X_q = j$ 的查询数。均匀无放回抽样下，$D_S \le 1$ 只可能抽到全零查询，或恰好一个计数为一的查询 [24]：

$$\Pr\{D_S(\alpha) \le 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\text{cal}}}{S}}. \tag{6.9}$$

越界组合数按零计。这是有条件的接纳模型；预测选择风险还需网格接纳结构与选中预算的质量映射。补证用全池记录的 X_q 按实际样本 ID 求和，18,432 个网格计数与所有抽取曲线一致，包括换执行路径的情形（第7.5.5节）。这验证了已测抽取的汇总计数，任意子集预测仍假设逐查询计数稳定。接纳不使用标签；预测 GT 损失越阈还要使用固定预算的留出质量映射。

6.3 预算搜索、嵌套性与并列安全

6.3.1 二分与完整枚举

记 $J = |A|$。实现先计算一次样本参考，再在降序网格中寻找最大的可接纳下标：
1. 置 $l = 0$、$h = J - 1$。下标零为参考回退。
2. 当 $l < h$，测试 $j = \lfloor (l + h + 1)/2 \rfloor$，运行 $A[j]$ 并计算 $D_S(A[j])$。
3. 若 $D_S(A[j]) \le \epsilon$，置 $l = j$；否则置 $h = j - 1$。
4. 返回 $A[l]$；调用方在处理复用批次前应用该值。

八点网格除参考外至多需要三次完整候选预算搜索。调用语义、主机比较复杂度和计时边界见第A.3节。

完整枚举重新计算参考，在完全相同样本上测试其他七个预算，选择最小被接纳乘数。两种计时都含新参考，统一成本边界。即使接纳集合存在空洞，枚举仍评价整个有限网格；二分要保证找到相同最小值，则需要接纳下标构成前缀。

6.3.2 单调性的充分条件

命题 6.1 (嵌套候选保留参考成员)**.** *对单条查询，假定路由固定，且 $\alpha \le \beta \le \alpha_{\mathrm{ref}}$ 时 $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\mathrm{ref}}}$。假定至少有 k 个有效候选、精排分数固定、最终并列使用同一个确定全序。每个结果均为该顺序下精确 top-k，则*

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\mathrm{ref}}| = |C_\alpha \cap \mathcal{R}_{\mathrm{ref}}|, \qquad D_S(\alpha) \ge D_S(\beta). \tag{6.10}$$

后一结论要求条件对每个采样查询均成立。

证明. 参考成员在最大保留集合中至多有 $k-1$ 个前驱。若它存在于较小集合，限制集合不会引入新前驱，因此仍处于该集合精确 top-k。与参考的交集，恰等于保留集合中存在的参考成员。嵌套只能增加这些成员；对查询求和缺失数即得不等式。 □

在同一固定完整 key 下进行精确主码 top-c，可在 c 增大时提供嵌套前提。再固定精排完整 key，接纳便单调，下标形成前缀，二分找到网格中最小可接纳预算。GT recall 和执行速度均不在定理中：近似精排分数可能随候选池扩大而提升错误近邻的名次，运行时间也依赖硬件。

6.3.3 评测分组路径的推论

运行头源码默认精确主码选择。同一次校准抽取内，S、路由和扫描线程数固定，`scheduled_groups(S)` 与 c 无关。改变预算影响缓冲容量和接纳工作，分组评分器仍使用相同 FP32 LUT 项与累加顺序。精排固定 B_r 和 16 lanes，保持每候选的残差归约；最终选择使用 $(D_{\mathrm{H}}, \mathrm{ID})$。

Case	C_1	C_2	C_3	Reference overlap
Nested	$\{b\}$	$\{b,c\}$	$\{a,b,c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b,c\}$	$\{a,b,c\}$	1, 0, 1

图 6.3. 构造的 $k=1$ 例子，精排顺序 $a \prec b \prec c$，参考为 $\{a\}$。嵌套情形保持参考成员；主码分数全并列时，不同并列选择可能产生非嵌套的中间候选池，破坏单调性。这些是解释用集合，不是 GPU 实测输出。

表 6.1. 预算对照及其回答的问题。所有结果均用留出 GT 评价，但只有诊断性 oracle 用它选择预算。一致性策略的参考固定为 200。

对照	选择信息	回答的问题
固定 100	无校准	相对沿用默认值收益多少?
固定 64 及更小网格预算	无校准	简单固定选择能否消除校准需要?
采样二分	S 条无标签查询、槽位一致性	低成本样本能否选好可复用预算?
同样本枚举	相同 S、参考和阈值，遍历网格	二分是否错过最小被接纳网格预算?
全校准池	全部 500 校准查询，同一二分规则	更广覆盖、更严归一化容忍度会如何?
留出 oracle	全网格 GT recall 和实测 QPS	哪个测量预算在诊断质量约束下最快?

推论 6.2 (分组校准)**.** *对评测中的分组配置，在第四、五章的表示与共享界契约下，各预算的主码保留集合嵌套。因此命题 6.1适用，二分与枚举返回同一最小被接纳网格预算。*

证明. 引理 5.1与命题 5.1与5.2给出共同完整键 $(D_{\mathrm{P}}, \mathrm{position})$ 下的精确主码 top-c，其前缀随 c 增大而嵌套。固定的逐候选精排与精确最终全序提供命题 6.1的其余前提。 □

推论覆盖 $S = 32, 64, 128$ 的 2,304 次抽取中的 2,176 次；其余 128 次是 OpenAI-3072 在 $S = 128$ 的距离单 block 路径。补证逐次记录完整网格计数，2,304 条曲线全部单调，决策均与枚举一致（第7.5.5节）。

全池覆盖范围。 十二次 $S = 500$ 调用另计：两个 Vogue 单元分组，十个使用 $g_{\mathrm{scan}} = 0$。补证新增全池枚举，十二条曲线均单调，所选预算均等于枚举得到的最小接纳预算，因此关闭了有限网格的实测缺口。十个距离路径仍不在 推论 6.2中；要给一般嵌套保证，仍可要求共同截断顺序，或所有网格截断（含参考）均无跨界并列。匹配留出路径有助迁移，但不证明一般条件。

基于排名的扩展。 在同样的嵌套与固定分数条件下，令 $\mathrm{rank}_{\mathrm{P}}(x)$ 为参考答案在共同主码 key 下从 1 开始的排名，则

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\mathrm{ref}}(q)} \mathbf{1}\{\mathrm{rank}_{\mathrm{P}}(x) > c(\alpha)\}. \tag{6.11}$$

导出这些排名，可从一次参考搜索恢复整条网格曲线。当前 harness 仍逐预算搜索；排名提取与传输是同一排序条件下的后续工作。

6.4 预算基线及其回答的问题

表 6.1区分预算选择与评价。固定预算共享索引、路由和留出查询；调用与计时细节见第A.3节。

令 $\widehat{r}(\alpha)$、$\widehat{Q}(\alpha)$ 为实测留出 recall 和 QPS，$\widehat{r}^{*}=\max_{\alpha\in A}\widehat{r}(\alpha)$。Oracle 为

$$\alpha_{\mathrm{oracle}}\in\arg\max_{\alpha\in A}\left\{\widehat{Q}(\alpha):\widehat{r}(\alpha)\geq\widehat{r}^{*}-\delta\right\},\qquad\delta=10^{-3}. \tag{6.12}$$

recall 容差 δ 不同于槽位容忍度 ϵ。Oracle 消耗留出标签与计时，是有限网格诊断，不是可部署的无标签决策规则。

全池校准仍无标签，使用二分，不是按 GT 约束遍历全部预算；较大样本还收紧 $\epsilon/(Sk)$。同样本枚举则固定样本覆盖和阈值，隔离搜索过程。Oracle 在质量约束下最大化速度，而一致性规则最小化被接纳乘数。因此，即使二分精确、校准池全部覆盖，也不一定选中 oracle。

固定 64 检查较小的统一默认值是否已能获得收益；固定 200 提供高预算对照。观测质量和所选预算结果见表 7.10与7.11。

6.5 留出风险、校准成本与回本

6.5.1 选择校准规模

全池工作点得到表 7.9中留出结果的支持；采样变体见图 7.6。第6.2节的独立结果位模型提供规划尺度：$k=10$ 时，$4.74/(Sk)\approx10^{-3}$ 提示 S 约为 500，而 128 的分辨率较粗。这一强模型针对参考不一致，不是 GT 损失，也不能将经验建议改写为预先指定的质量保证。

全池与留出批次使用同一扫描制度。十个单元采用距离截断；全池枚举已确认十二个实测单元均选到网格中最小接纳预算（第6.3节）。这一实测补充分组路径定理，但不将其扩展到任意距离并列。

6.5.2 风险与所选预算性能

共做 m 次校准抽样，第 r 次选择 $\widehat{\alpha}_r$，定义留出损失和越阈频率：

$$L_r=\widehat{r}^{*}-\widehat{r}(\widehat{\alpha}_r),\qquad\widehat{p}_{\delta}=\frac{1}{m}\sum_{r=1}^{m}\mathbf{1}\{L_r>\delta\}. \tag{6.13}$$

平均损失、越阈频率和尾部大小回答不同问题。表 7.10与图 7.6报告归档摘要；补证逐抽样最大损失见表 7.13。

所选预算速率为 $\overline{Q}=m^{-1}\sum_r\widehat{Q}(\widehat{\alpha}_r)$，由已计时的固定预算查取。它既不同于中位预算的速率，也不同于校准后查询流的吞吐。质量与速度须一起解读（表 7.12）。

6.5.3 复用成本

对 B_{reuse} 个等大后续批次，模型为

$$T_{\mathrm{policy}}(B_{\mathrm{reuse}})=T_{\mathrm{cal}}+B_{\mathrm{reuse}}T_{\mathrm{chosen}},\qquad T_{\mathrm{fixed}}(B_{\mathrm{reuse}})=B_{\mathrm{reuse}}T_f. \tag{6.14}$$

当 $T_{\mathrm{chosen}}<T_f$，回本批数为

$$B^{*}_{\mathrm{reuse}}=\frac{T_{\mathrm{cal}}}{T_f-T_{\mathrm{chosen}}}. \tag{6.15}$$

严格超过 B^{*}_{reuse} 才有正节省；若选中批次不更快，则没有吞吐回本。对 n 查询批，由 QPS 换算毫秒为 $T=1000n/Q$。选择变化时，$1/\mathbb{E}[Q]\leq\mathbb{E}[1/Q]$，因此使用 $1/\overline{Q}$ 偏乐观。实测调

用边界见第A.3节；四个单元已用直接校准加查询流总成本验证这一模型，并同时报告质量(第7.5.6节)。图 7.7给出当前实验估计。

一个复用成本算例。 OpenAI-1536 在 128 探针时，全池校准选择 $\alpha = 8$，请求 80 个保留候选，固定 100 则为 1,000。两者 recall 均为 0.9280，速率分别为 267,690 和 167,451 QPS。500 查询时，由此得到 $T_{\text{chosen}} = 1.868\,\text{ms}$、$T_f = 2.986\,\text{ms}$；校准为 9.5 ms，故 $B^*_{\text{reuse}} = 8.50$，随后九批有正模型节省。这些是归档预算的算术计算，不是直接计时的查询流成本。

若每 H 批重新校准，会增加每批 T_{cal}/H；漂移检测和自动重校准仍是后续工作。

6.6 策略失效模式与适用范围

复用要求索引、k、路由配置和查询群体足够稳定。改变索引或路由会改变校准目标；批次制度变化也可能经调度改变并列和算术。表 6.2列出适用条件，不是测试程序已经提供的自动保护。

表 6.2. 解释或复用校准预算前应核对的条件。实测净成本及表示检查具有明确有限覆盖；未来漂移处理仍是独立工作。

失效模式	所需证据或条件
参考漏掉真实近邻	一致性之外的留出 GT 质量
样本漏掉困难查询	已声明查询池内的越阈频率与尾损失
非嵌套并列或换路径	共同排序，或配对有限网格枚举
预算计时噪声或任务太短	重复计时与含校准的直接总成本
索引、路由或查询混合改变	复用乘数前重新验证
输出不足或无效	应用集合一致性前检查有效 ID 数与唯一性

对已观察的工作负载族，质量优先时采用全池校准，$S = 128$ 则是校准更便宜、单独测量越阈风险的变体。调用方须显式设置所选 α，并在相同索引、路由和 k 下复用。贡献是无需标签的有限网格决策、针对该决策的有条件精确搜索，以及复用成本模型。第七章以固定预算及诊断 oracle 检验质量与速度，并在四个单元直接计时校准后的复用。

补证已保存逐次样本 ID、所选 α、完整 D_S 曲线及损失、全池枚举、池级 X_q 和 500 批次轨迹。剩余工作包括任意子集上的逐查询不变性、实际主码分数的严格 CPU 参考、Vogue 跨进程差异及未来查询变化。状态维护在 `assessment/ch7-pending-experiments.txt`。

预算主实验及轨迹重跑均使用 v69 算术，OpenAI 包含 TF32。独立 FP32 敏感性测量搜索前沿，没有在 FP32 下重新校准或验证这些预算选择。

第 7 章

实验评估与研究发现

本章通过构建测量、受控执行改动、留出查询预算选择和跨系统搜索，回答论文的四个研究问题。对于 **RQ1**（构建与容量），最大被测索引在单卡上搜索 17.78 百万个向量，常驻设备显存为 20.44 GiB；补证训练加添加重复为 10.54–10.65 s，采样构建峰值 22.58 GiB。对于 **RQ2**（正确且高效的 GPU 执行），两组查找表在 22/24 个配置中领先；批大小 8 时，调度扫描的吞吐量几何平均提高至 2.727×，并保持被测最终标识符集合一致。对于 **RQ3**（预算质量与净成本），全池校准在 12/12 个单元达到质量目标，最大损失 0.0006，所选配置速率相对固定 100 的几何平均为 1.111×。低成本 $S = 128$ 最多有 40/64 次越阈，补证完整决策轨迹及直接校准复用计时关闭有限网格和净成本核对缺口。对于 **RQ4**（竞争区间），召回率 0.97 处，插值比较中 JHQ 在三个 E1 数据集上最快，在 arXiv 上与 IVF-RaBitQ 数值接近；OpenAI 历史结果使用 TF32，FP32 敏感性实验仍领先但幅度缩小；T5 显示不同的小批次区间：batch 8、recall 0.97 时四个数据集均由 CAGRA fp32 领先。

7.1 最终实验清单与对照实验组

实验分组区分了回答 RQ1–RQ4 所需的证据。组件名称描述机制，版本号标识实现来源。记录中的运行源码版本、二进制名称以及源码到结果的映射见附录 A。

7.1.1 证据分组与测量边界

表 7.1 列出了本章使用的实验组。版本号表示不同的实验实现，而不是对同一个不变基准的连续测量。尤其需要区分历史 ADC 论文与后来的 v69/v71 实验：它们的计时结果不同。本章相应的数字以较新的原始记录为准；历史打包访存和精排实验则作为独立消融保留。将这些实验的加速比直接相乘，会混合不同基线，得不到实际测量过的系统加速比。

表 7.1. 实验与证据分组。数量表示测量配置或检查次数，不代表独立索引构建次数。所有实验均使用 RTX 5090，但计时协议有所不同。

实验组	目的与实现	覆盖范围
E1	v69 与 cuVS 图索引、PQ、RaBitQ、暴力搜索比较	1,590 个点；4 个数据集
LUT	v71 查找表分组，$g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$	24 个单元；96 个变体
打包访存	历史字节/机器字访存析因对照	48 个配对单元
精排	v61 标量、加载方式与 lane 数消融	13 个配置
扫描/共享界	v62 全局选择；v63 共享界	64 / 48 个正确性比较
调度	集成后的 v65 执行策略	50 个工作负载—批大小单元
探针选择	v69 已算分数的精确探针选择	1,792 个合成用例
预算	v69 抽样、全校准池与 oracle 预算	12 个单元；每个 S 抽样 64 次
规模	v69 搜索、冷构建与常驻显存	BGE 与 Stella 扩展实验
补证	重复构建、PTX、配对、预算轨迹及精度	30 次构建、12 预算单元、76 精度配对
T4/T5	严格选择参考及小批次前沿	270 个检查；1,808 个计时点

Sections 3.2与3.3定义数据清单、环境与测量协议。主系统比较采用 RTX 5090 上的主机到主机稳态吞吐；构建与预算校准单独测量。

作者的 CPU JHQ artifact 已用于单独归档的 CPU 实验，但未纳入 E1 前沿。十月访存局部性实验在 Vogue 上采用 1,024 个列表和 $[8, 4]$ 级比特，而 E1 为 4,096 个列表及不同残差预算。较早 CPU CSV 还涉及本地重实现，CPU 审计笔记另记录了历史 recall 协议问题。这些记录能回答 CPU 内部问题，但没有提供本章协议匹配的 CPU/GPU 加速倍数。

自适应调用次数与辅助设备端计时见第A.2节。

7.1.2 指标与可复现性

所有实验均采用平方 L2 和式 (3.1) 定义的 Recall@10。精排系数决定候选保留数 $c = \max\{k, \lceil \alpha k \rceil\}$。本章中，$g_{\mathrm{LUT}}$ 表示笛卡尔查找表分组数，g_{scan} 表示并行扫描组数；二者控制不同工作。

本章分析脚本只读取已有测量，不运行新的 GPU 实验。它核对记录数量与正确性字段，重新计算配对比值和匹配召回率结果，并直接从原始记录生成数字表格。配套证据清单记录了所用文件的 SHA-256 哈希。其中的源码检出版本号表示本次核对所使用的仓库快照，并不意味着所有历史二进制程序都由这个提交构建。各次运行的原始日志头和实验报告仍保留其各自构建来源。这样既能复现当前图表，也不会错误地给历史实验重新标注版本。[1]

7.2 现代 GPU ANN 的召回率—QPS 前沿

本节通过定位共同召回率下的运行区间回答 RQ4，同时包含竞争方法更快的区间。

7.2.1 比较范围与前沿构造

图 7.1 将 JHQ-GPU 与 CAGRA fp32、经过标量量化的 CAGRA int8、IVF-RaBitQ、IVF-PQ 和 GPU 暴力搜索进行比较。CAGRA 代表图搜索；PQ 和 RaBitQ 代表使用压缩表示的倒排文件方法 [16, 29, 32]。四个数据集分别为 Vogue（$932{,}328 \times 768$）、arXiv（$2{,}253{,}000 \times 768$），以及各含 999,000 个向量、维度分别为 1,536 和 3,072 的两个 OpenAI 数据集。每个数据集使用单批 1,000 条查询（$b = 1{,}000$），各方法共享同一份 ground-truth 文件。独立 T5 实验在第7.2.5节比较跨系统小批次行为；第7.6节的组件收益仍属于内部消融。

归档调度在全部 105 个 arXiv/OpenAI 运行—预算单元使用 $g_{\mathrm{scan}} = 0$；Vogue 的 27 个单元全部使用 $g_{\mathrm{scan}} = 4$，由 0.338–0.622 的候选数 CV 触发。因此，非 Vogue 前沿使用 query-centric 扫描，Vogue 则还使用分组保留、共享界与合并（第5.5节）。

每种方法贡献其非支配的实测运行点。如果同一方法的另一配置在召回率和吞吐量上均不差、且至少一项严格更好，则该点被剔除。因此，一条前沿组合了不同搜索参数，有时还组合了不同索引配置；它并不一定表示同一个固定索引的变化轨迹。每个数据集上，JHQ 提供 27–39 个测量点，各基线组提供 54–264 个点。E1 解析器得到 1,590 个有效点，没有失败记录，也没有无效标识符或行内重复标识符标记。这个数量表示参数扫描的规模，而不是 1,590 次独立重复实验。

参数覆盖较为充分。CAGRA 改变图出度 32/64、搜索宽度 1–8 和中间 top-k 容量 32–512。RaBitQ 改变 4/8/9-bit 预算、支持的搜索模式以及最高 1,024 的探针数。PQ 改变编码长度、

[1] 报告仓库提供 `scripts/build_ch7.py`、`data/ch7/audit.json`、`data/ch7/matched.csv` 和证据清单 `data/ch7/sources.csv`。脚本需要同级目录中包含归档测量的 `JHQ_GPU` 仓库。

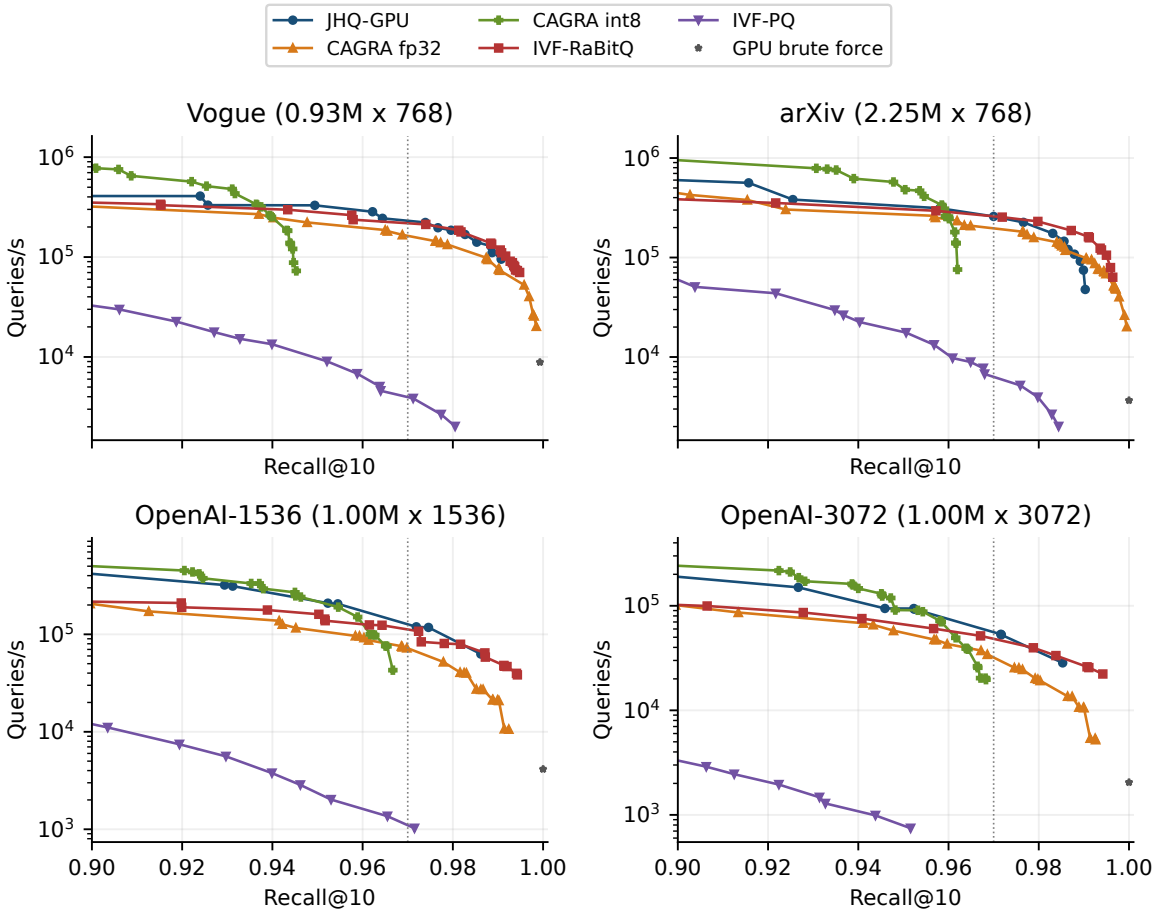


图 7.1. 四个 E1 数据集上实测的主机到主机召回率—QPS 前沿。图中从召回率 0.90 开始显示，并在 0.97 处标出竖线；构造前沿时纳入了包括 CAGRA int8 在内的全部基线组。连线连接实测点，吞吐量轴为对数坐标。

查找精度和探针数。JHQ 同时改变探针数与精排预算，并在初始网格无法充分覆盖召回率时扩展网格。比较结论因而受限于这些已记录的参数网格。某条曲线在目标召回率之前结束，表示该区间没有测量，不能证明该方法无法达到目标。

这些曲线比较的是可达到的吞吐量，而不是相同显存占用下的表现。JHQ 每个向量约存储 $d/8$ 字节主码、d 字节残差码以及元数据。九比特 RaBitQ 的名义编码预算为 $9d/8$ 字节，与 JHQ 接近，但它的 4/8-bit 配置使用更少存储。CAGRA int8 还存储图边；PQ 编码长度可变；CAGRA fp32 保留 FP32 向量。对这些选项取性能包络有助于部署选择，但不能单独证明相同显存预算下的表示优势。显存证据在第7.7节另行讨论。

7.2.2 匹配召回率与不同运行区间的反向结果

表 7.2 使用两种估计方式，量化召回率 0.97 附近的表现。插值估计在相邻前沿点之间对吞吐量取对数后进行线性插值，不向覆盖范围之外外推。实测阈值估计选取达到目标召回率的最快采样点。插值估计将与目标相差不超过 5×10^{-4} 的实测召回率视为匹配；实测阈值估计则要求召回率至少达到目标。插值减小了参数网格间距不一致的影响；实测阈值估计对应真实记录过的配置。两者都不能凭空产生一个召回率恰好等于目标的新测量。

表 7.2. 召回率 0.97 下的比较。每组比值依次为 “插值 / 实测阈值”；大于一表示 JHQ 更快。JHQ kQPS 为插值估计。arXiv 与 RaBitQ 的微小差异随估计方法而改变方向。

数据集	JHQ kQPS	相对 CAGRA fp32	相对 IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

表 7.3. 批大小 1,000 时 Vogue E1 的路径敏感性。两条前沿采用相同运行—预算网格，并按同一 log-QPS 插值规则与 RaBitQ 比较；每个配对运行的最终输出逐位一致。

召回率	$g_{\text{scan}} = 4$ / RaBitQ	$g_{\text{scan}} = 0$ / RaBitQ
0.96	1.250	1.072
0.97	1.059	0.933

相较于 CAGRA fp32，该目标下的插值比值从 arXiv 的 1.317× 到两个 OpenAI 数据集的 1.747× 不等。相较于 IVF-RaBitQ，对应数值分别为 1.059×、0.996×、1.130× 和 1.157×。arXiv 的结果应视为接近平手：插值吞吐量略低，但实测阈值比较略高。如果将其表述为明确获胜，就会让插值约定获得超出测量本身所支持的证据权重。

E1 同批单线程块测量把 RQ2 与这项 RQ4 结果连接起来。以 $g_{\text{scan}} = 0$ 重建 Vogue 前沿，0.97 处相对 RaBitQ 的比值从 1.059 降到 0.933（表 7.3）。因此，这份网格中的有利方向依赖调度后的分组路径。每对保持索引、探针、预算和最终输出不变；它测的是集成路径，未单独隔离共享界的贡献。

在有共同覆盖的比较中，目标召回率 0.90–0.985 范围内，JHQ 相对 CAGRA fp32 的插值优势约为 1.10–2.44×。但是，加入 CAGRA int8 后，低召回率区间的解释发生变化：在 0.90–0.92 附近，其低精度图搜索可以快于 JHQ。在更高召回率下，IVF-RaBitQ 常重新领先，包括若干 0.98 及以上目标。在 arXiv 约 0.99 的召回率处，记录网格中的 CAGRA fp32 也快于 JHQ。因此，可被证据支持的结论是：JHQ 在部分有价值的区间具有竞争力，尤其是中高召回率附近，而不是全面优于现代 GPU ANN 方法。

CAGRA int8 在 Vogue、arXiv、OpenAI-1536 和 OpenAI-3072 上的最高实测召回率分别为 0.9453、0.9620、0.9667 和 0.9683，因此这些网格中没有达到 0.97 的配置。按相同插值规则，目标召回率 0.90–0.92 处，它约比 JHQ 快 1.20–1.98×；在 arXiv 的 0.95 和两个 OpenAI 数据集的 0.94 处也仍然更快。0.97 的结果针对具有测量覆盖的方法，不能推广为低于 0.97 的每个目标均领先。

网格稀疏时，两种估计可能有明显差异。例如，OpenAI-1536 在召回率 0.97 处相对 CAGRA fp32 的比值，插值为 1.747×，而按满足阈值的实测点计算为 2.294×。后一个数值包含了基线下一个超过阈值的测量点所付出的代价，不能将其表述为在完全相同召回率下直接测得的同等幅度优势。配套匹配表保留两种估计，并标明没有覆盖的目标，使读者能够区分真实的运行区间结果与参数采样带来的影响。

7.2.3 OpenAI 数据集上的精度敏感性

补证源码检查确认：E1 实现虽然输入和存储为 FP32，OpenAI 旋转与质心 GEMM 实际采用 TF32 乘法、FP32 累积，源码注释具有误导性（第4.2节）。主 E1 前沿保留该历史实现的

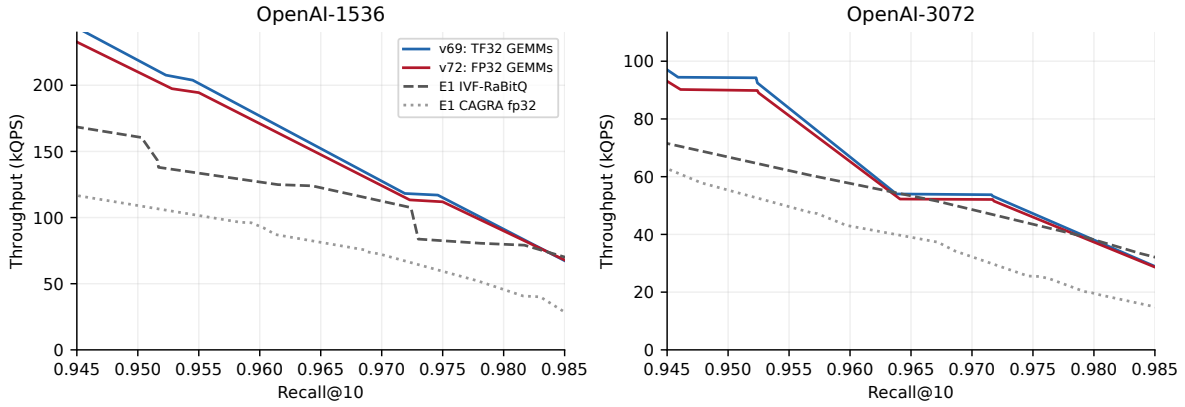


图 7.2. 补证精度前沿。实线为同会话 JHQ 变体，虚线 / 点线为历史 E1 基线；均采用本章前沿及插值规则。

测量，另以 FP32 变体修正 cuBLAS 开关，重跑 37 / 39 个 OpenAI 配置，batch 为 1,000，每配置五轮，在各探针设置间交替变体执行顺序。两者复用同一 TF32 训练模型，改变的是 add / 查询算术，包括数据库编码及路由，而不是独立 FP32 训练模型的对比。表 7.4汇总配对变化。

表 7.4. FP32 敏感性。QPS 比值为相同配置下 FP32/TF32 的几何平均，召回差方向相同。最后两列在 recall 0.97 处比较 FP32 前沿与归档 E1 的 IVF-RaBitQ 及 CAGRA fp32；基线没有重新测量。

数据集	配对数	QPS 比值	ΔRecall	对 RaBitQ	对 CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

相同配置下，FP32 吞吐约低 5%，recall 高 0–0.0005。recall 0.97 处，吞吐比同会话 TF32 前沿约低 3%，仍为归档 RaBitQ 的 1.086× / 1.079×，以及归档 CAGRA fp32 的 1.679× / 1.629×。同会话 v69/RaBitQ 为 1.118 / 1.114，历史值为 1.130 / 1.157。这将精度干预与会话波动分开：领先方向仍在、幅度缩小，跨会话基线比较仍有计时不确定性。两精度变体完整前沿见图 7.2。

7.2.4 结果解释与比较限制

OpenAI 对照来自 Qdrant DBpedia 的 `text-embedding-3-large` 语料族，构成按行对应的嵌入宽度对照 [30, 31]。补证比较全部 1,000 查询及每 97 行一个的 10,299 个 base 行：3,072 维截断并归一化后，与 1,536 维对应向量余弦至少 0.9980，中位 1.000000；GT top-10 平均重合 9.046/10。这强力支持所查导出行顺序一致，但没有导出原始实体 ID 映射。维度翻倍后，历史 0.97 处相对 CAGRA fp32 均为 1.747，RaBitQ 比值从 1.130 变为 1.157；FP32 敏感性也保留相近优势。因此这一配对语料 / 嵌入对照内，维度本身不能解释观测优势。

领先幅度反而与精排需求同向变化：E1 设置的 128 探针预算锚点，OpenAI 为 4–8、Vogue 为 64、arXiv 为 100（后来的全池规则在 arXiv 选择 200）；0.97 处相对 RaBitQ 的比值依次为 1.130–1.157、1.059、0.996。这四个数据集的观测关联符合选择性残差访问的设计；锚点既不是各同召回率前沿点的参数，也没有因果隔离主码质量。后文消融直接检查执行组件。

即便对齐主要计时边界，比较中仍存在限制。JHQ 从 C++ 调用，CAGRA 和 PQ 则通过 Python 绑定调用；即使复用资源，主机端开销仍可能有小幅差异。JHQ 输出回传至可分

页主机内存，基线使用页锁定内存。编译目标不同，无关的主机负载也无法完全控制。对于很小的性能差距，这些因素不可忽略。五轮测量描述的是已构建索引上的重复执行，不能估计独立重建索引或换一台机器后的变化。

7.2.5 小批次改变竞争区间

T5 在四个 E1 数据集、batch 8/32 下比较调度 JHQ、CAGRA fp32/int8 和 IVF-RaBitQ。五轮同步主机到主机调用分别使用 1,000/992 条查询。1,808 个点包含 JHQ 单 block 对照；本实验不含 IVF-PQ 和暴力搜索。图 7.3与表 7.5比较所得到的工作区间。

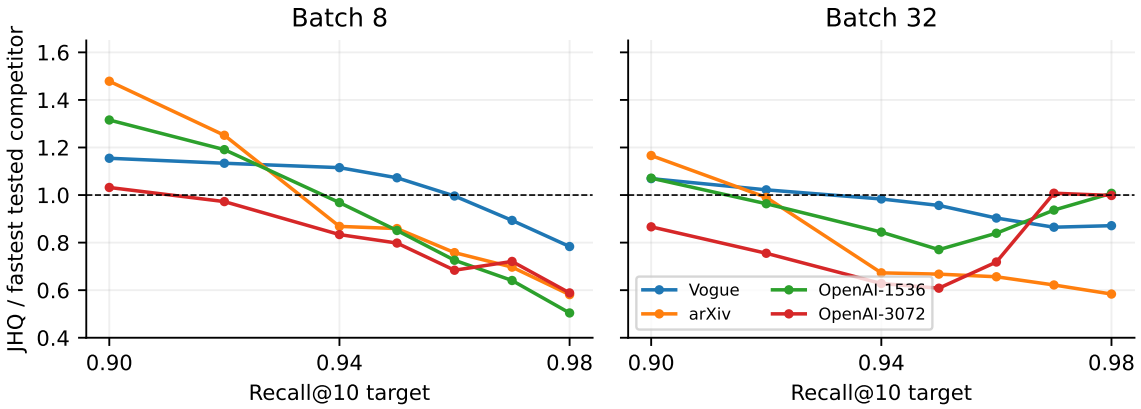


图 7.3. T5 包含达标实测端点的插值估计下，JHQ / 最快对手 QPS。大于一表示 JHQ 更快。严格实测阈值见表 7.5。

表 7.5. T5 QPS 比值，依次为“包含达标端点的插值 / 严格实测阈值”。每种估计独立选择最快对手；更高 recall 的达标端点保留，无外推。

数据集	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

recall 0.97、batch 8 下，两种估计均显示 CAGRA fp32 在四个数据集领先，JHQ 插值比为 0.641–0.893。batch 32 的插值比为 0.622–1.008；OpenAI-3072 在插值下接近平手，但实测阈值有利于 JHQ。recall 0.95 时，Vogue 的 batch-8 领先同样只出现在插值估计。因此，边界排名受网格分辨率影响。

较低 recall 更有利于 JHQ，但必须保留最快前沿点已经超过目标的竞争者。0.90 处，修正后的插值比为 batch 8 的 1.032–1.479 和 batch 32 的 0.866–1.166；严格实测阈值分别给出三个和一个 JHQ 领先。排除这些达标端点会夸大低 recall 优势。调度仍然有用：0.95 处相对单 block 的插值提速为 1.7–3.3×。内部执行提速因此未必转化为跨系统领先。

按数据集、方法与 batch 汇总的基线空调用中位数为 0.012–0.072 ms，该开销仍包含在搜索计时内。CAGRA 使用 Python 绑定，JHQ / RaBitQ 使用 C++，预热策略也有差别。证

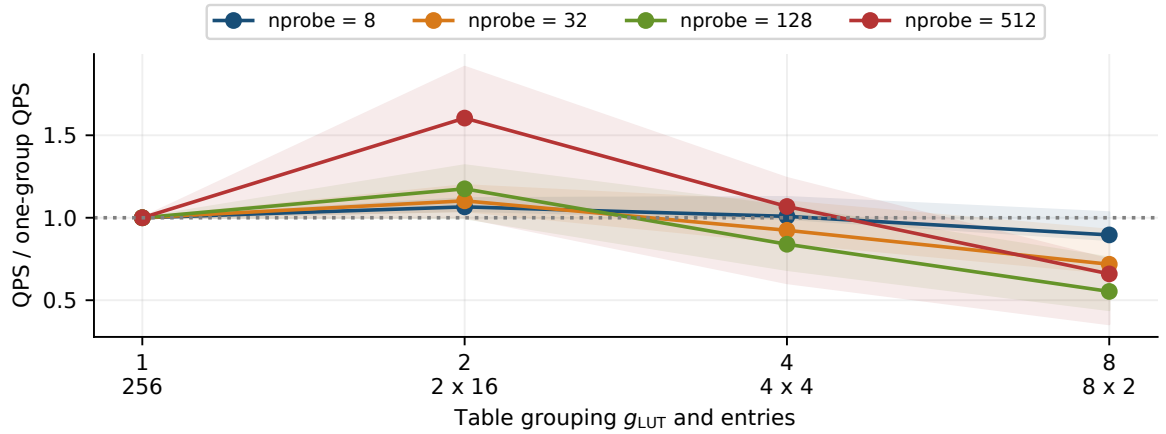


图 7.4. 固定 $\alpha = 100$ 的笛卡尔查找表分组实验（v71）。组数表示 g_{LUT}，不是并行扫描组数。曲线为各数据集相对单组 QPS 比值的中位数，阴影表示范围。

据支持有限网格、单客户端的工作区间，协议见第A.7节。E1 在 batch 1,000 的高 recall 领先不能推广到小批次。

7.3 笛卡尔查找表、打包加载与 lane 协作精排

针对 RQ2，消融区分了保持表示不变的查找表、加载改动与协作式残差评分，并同时测量内核效果和整体应用效果。

T4 还逐候选比较表形式：20.5–29.4% 分数的 FP32 比特不同，最大相对误差 4.64×10^{-7}。Vogue 有 2–4/256 条查询的末端 ID 不同，arXiv / OpenAI 则没有（第A.7节）。

7.3.1 查找表的精确因子化及其吞吐量影响

笛卡尔因子化改变的是已表示主码距离的计算方式，而不是重新训练一个量化器。将一个八比特笛卡尔编码拆为两个四比特分量，可把 256 项查找表替换为 $2 \times 16 = 32$ 项。对于 FP32 表项，每个对应查找表由 1,024 字节降为 128 字节，即缩小八倍。数学上的评分保持不变，但浮点求值顺序仍可能改变末尾比特。因此，仅凭查找表更小，不能断言完整 ANN 输出逐位一致。

图 7.4 比较六个数据集上的笛卡尔查找表分组，探针数为 8/32/128/512、固定 $\alpha = 100$，以及 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$。每个变体有五轮计时；除 Stella 使用 999 条查询外，批大小均为 1,000。在 24 个单元中，两组查找表有 22 个取得最高吞吐量中位数。例外是 arXiv 的 128 探针配置，单组约快 0.23%；以及 OpenAI-3072 的八探针配置，四组约快 0.84%。这些小差异更适合视为接近平手。新版统计是 22/24，而不是历史 ADC 论文中的 23/24。在明确的“两组对单组”配对比较中，两组赢下 23/24 个单元，且各数据集跨四个探针数的几何平均比值均大于一（1.021–1.318）。这支持其在被测工作负载上的选择，无需把共享索引的配置当作独立试验。

两组相对单组的吞吐量比值为 0.998–1.914。四组与八组都只需总计 16 个表项，但查表次数和粒度不同。这构成了有用的对照：在两组设计的基础上进一步缩小表存储，并不必然提高搜索性能。额外的索引计算与累加工作可能超过存储节省带来的收益。测量建立的是净

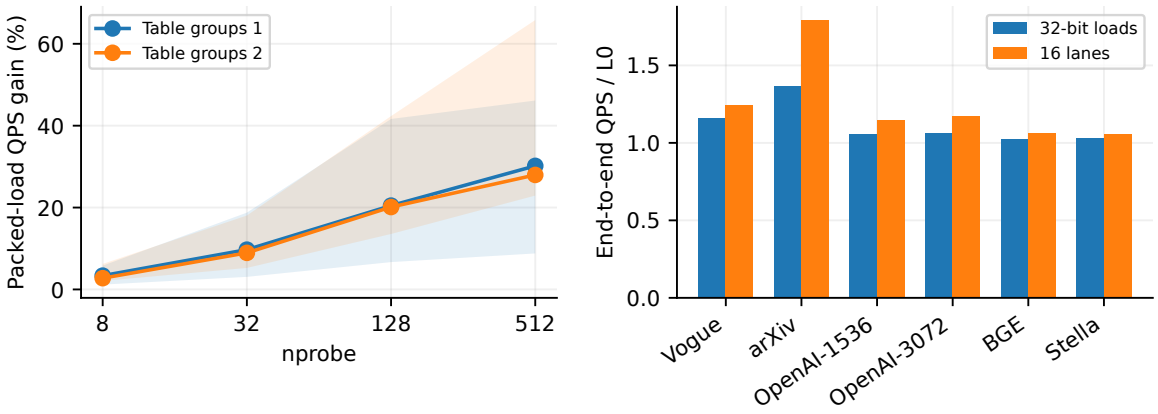


图 7.5. 打包访存与精排对照。左：各查找表分组和探针数下，六个数据集打包加载 QPS 增益的中位数，阴影为最小值至最大值。右：128 探针及各数据集普通预算下，仅改变加载和使用 16 个 lane 的实测端到端收益。两个子图来自不同实验组。

表 7.6. 128 探针下的精排结果。除最后一列为剖析得到的精排内核加速比外，其余比值均为相对 L0 的配对端到端收益。不能将内核加速比与应用整体加速比混用。

数据集	α	L1/L0 吞吐比	L16/L0 吞吐比	内核加速比
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

收益与开销之间的权衡；缺少独立的占用率或指令计数实验，就不能将某个硬件因素认定为唯一原因。任一四变体单元内，报告的最大召回率差为 10^{-4}。

7.3.2　打包访存与协作式残差评分

历史打包实验固定数据集、探针数和查找表分组，比较以字节和机器字为单位的访存。在六个数据集、四个探针数以及 $g_{\mathrm{LUT}} \in \{1, 2\}$ 的 48 对比较中，吞吐量比值为 1.012–1.658，最大召回率差为 10^{-4}。八探针时收益通常较小，512 探针时较大。两种实现都已经采用子空间优先布局，因此，这个实验支持打包访存的效果，而不是“仅靠物理转置就取得了提升”的说法。每个变体只报告一个汇总吞吐率，因而这一较早证据提供的不确定性信息弱于五轮计时的 v71 扫描。

图 7.5与表 7.6 进一步区分了残差精排中更宽加载与 lane 协作的作用。L0 是原始标量路径；L1 改变加载方式，但不将单个候选分给多个 lane；L16 则由 16 个 lane 协作处理一个候选。同一进程在五轮计时中轮换变体顺序，每轮每个变体执行三次搜索。在十二个普通配置上，L16 的端到端吞吐量为 L0 的 1.015–1.789×。另外加入的 arXiv $\alpha = 200$ 配置明确属于高预算对照，达到 2.383×；不能把它并入普通所选预算的收益范围。

在被测用例中，仅改变加载的对照与 L0 逐位一致，但已使 arXiv 吞吐量提高至 1.368×。lane 协作进一步缩短了精排时间。在 arXiv 的 $\alpha = 100$ 配置上，精排内核由 4,277.5 降至 716.8 μs，即 5.97× 的内核加速，但整体吞吐量只提高至 1.789×。OpenAI-3072 的精排内核加速为 8.81×，整体吞吐量则为 1.171×。这与 Amdahl 定律一致：加速一个较小阶段，并不

能消除路由、筛选、最终选择或传输的成本。这也说明了随后扫描与探针实验的动机——继续处理剩余工作。

定量 Amdahl 检验使用基线精排内核时间占主机到主机基线延迟的比例 f，以及内核加速比 s；若其余耗时不变，整体估计为 $1/(1-f+f/s)$。arXiv 和 OpenAI-3072 的比例约为 0.529 和 0.168，对应估计 1.787× 和 1.175×，接近实测 1.789× 和 1.171×。它结合了单独的内核剖析与墙钟测量，只是机制解释的估计。

协作归约改变了浮点加法顺序。被测多 lane 变体的距离差异很小，最大报告相对误差约为 1.6×10^{-6}，但排序边界附近偶有结果变化。因此，证据支持的是数值接近、且具有实测端到端收益的评分，而不是无条件的逐位等价。Sanitizer 检查在较小查询子集上覆盖了记录中的精排和 top-k 内核；这些是有用的实现检查，并不是对任意输入的证明。

7.4 全局 top-c、共享界与探针选择的正确性及消融

第4.6节的早期表示 pilot 另行检查了码本结构、全行修正项及采样返回分数。这些检查补充下文的查询选择比较，两者均不能替代原始输入重编码或完整放置参考。

本节先回答 RQ2 的正确性部分，再考察分组扫描、阈值共享和基数探针选择的成本。

7.4.1 计算分数选择的严格 CPU 参考

补证 T4 使用 GPU 生成的 FP32 分数，在 CPU 上按 $(D_{\mathrm{P}}, \mathrm{position})$ 对每条查询的全部已访问候选排序。诊断导出器遵循打包扫描布局和累加顺序；所有返回有效主码项的导出分数与选择器分数逐位一致。源码对照确认，诊断版本相对拆表实验实现只改变导出部分。Vogue、arXiv、OpenAI-3072 采用 1/8/128 探针、$\alpha\in\{4,100,200\}$、$g_{\mathrm{scan}}\in\{0,1,2,4,8\}$ 及共享界开 / 关，共 270 个配置，每个配置 256 条查询，即 69,120 个查询实例（表 7.7）。

表 7.7. T4 CPU 参考检查。集合及顺序一致使用完整主码键。有效数量检查 $\min(c,n_c)$ 个 survivor；末端一致性按 $(D_{\mathrm{H}},\mathrm{ID})$ 排序已保留的计算精排分数，包含 FP32 比特比较。计数为查询实例，同一查询在配置间重复使用。

路径	查询数	集合 / 顺序精确	纯并列差异	有效数量正确	末端精确
$g_{\mathrm{scan}}\geq1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}}=0$	13,824	13,768 / 8,758	56	13,824	13,824

55,296 个分组实例全部与完整键参考的顺序一致。单 block 的 13,824 个实例全部满足距离全预序，56 个集合差异全部位于截断并列。19,120 个候选不足实例均返回正确数量的有效 survivor；全部 69,120 个末端选择均与保留分数参考一致。这直接检查 引理 5.1与命题 5.1与5.2 的阶段契约。此前 64 个扫描对照及 48 个共享界对照仍提供独立的实现间配对证据；归档 sanitizer 覆盖较小子集。导出与归档边界见第A.7节。

7.4.2 共享界保持选择目标不变

共享界组件使用由 c 个有效候选支撑的完整键界减少准入。过时的完整键读取允许额外工作，但保持选择目标（命题 5.2）。所有主码分数仍被计算；节省发生在选择、压缩整理和合并，而非距离剪枝。

四个工作负载与四种 batch 的 48 个共享界 / 独立界比较，主码和末端输出均一致。五轮配对吞吐比为 0.994–1.562×。arXiv 的 $\alpha=200$、batch 1,000、八组配对剖析中，merge 约由

表 7.8. 已算分数的精确探针选择改动的 v69/v65 端到端吞吐量比。每个工作负载—批大小比较中，每个二进制程序共计时十轮，采用交替运行。大于一表示 v69 更快。

数据集	批大小 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

2,629 降至 471 μs，scan 由 7,968 降至 5,513 μs。共享能减少合适工作负载的中间工作，其他配置也有小幅回退。

未采用的静态配额设计面临优胜候选集中：pilot 中第一候选组包含全局 survivor 的 46–81%。等额配额需要正确性回退，校准的不等额配额仍让主导组处于关键路径。这些观测在该 pilot 范围内促成了共享界设计。

7.4.3 精确探针选择与流水线一致性的边界

当筛选变快后，IVF 探针选择在小批次时间中的占比会提高。基数探针原语（v69）改变了选取最小质心评分的实现，保留此前算分制度；OpenAI 包含 TF32 GEMM（第7.2.3节）。其合成检查覆盖 1,792 个用例，失败数为零。在真实 arXiv、Vogue 和 OpenAI-3072 工作负载上，检查发现全部 1,000 条查询的探针标识符相同，主码 top-c 位置与距离也相同。这比“召回率数字没有变化”更具体，因为它直接检查了路由和筛选输出。

表 7.8 显示端到端吞吐量提高至 1.034–1.167×。批大小为 8 时，剖析得到的探针阶段在 arXiv 上从 56.5 降至 22.6 μs，在 Vogue 上从 52.7 降至 14.7 μs，在 OpenAI-3072 上从 77.8 降至 16.7 μs。大批次下，未修改阶段占整体耗时比例更高，因此整体收益较小符合预期。这些是各自实验组内部的比值，不能把它们与前面的 v62 或精排收益相乘，仿佛所有测量都共享同一个基线。

Vogue 区分了探针原语与跨进程流水线可复现性。在一组、批大小 8 的测试中，两种二进制的探针/主码 dump 对全部 1,000 条查询逐字节相同，但最终召回率为 0.9642 与 0.9644。同一个基线二进制也从一致性运行中的 0.9642，变成之后两次计时运行中的 0.9644。因此，观察到的变化并不需要探针改动才能发生；在 dump 控制的用例中，它出现在已记录主码选择边界之后。残差表示、输入和评分计算仍是可能来源，根因尚未确定。在各被检进程内部，调度与单线程块输出仍逐位一致。附录 A明确了 dump 的覆盖范围，避免将其当作对每条计时路径的检查。

最后，可选的 v70 FP16 实验属于另一项近似。在 arXiv 的 936/1,000 条、Vogue 的 979/1,000 条、OpenAI-3072 的 998/1,000 条查询上，其探针集合与 v69 参考完全相同，平均集合重叠率很高。但高重叠率和很小的召回率变化，并不能使该原语变成精确操作。这些结果不纳入主要 E1 前沿，也不纳入精确探针选择结论。

7.5 留出查询上的预算质量、吞吐量与回本

策略及其单调充分条件见第6章与命题 6.1。本节 RQ3 考察相对可信固定预算的质量和摊销成本。因此，比较同时包含沿用的 $\alpha = 100$ 与更强的统一固定对照 $\alpha = 64$。

7.5.1 无标签选择与使用标签的诊断性 oracle

预算实验在每个数据集—探针数组合中固定一个训练好的索引，比较固定预算、抽样校准、全校准池校准、同样本枚举与离线 oracle。六个数据集均测试 128 和 512 探针。偶数下标查询组成 500 条校准池；奇数下标查询组成互不重叠的 500 条留出池，Stella 例外，留出池为 499 条。从校准池中使用固定随机种子抽取 $S \in \{32, 64, 128\}$ 条查询，每种大小重复抽取 64 次；单次抽样内不放回。重复抽样使用的是同一个有限查询池，并不等于对未来工作负载漂移进行 64 次独立采样。

降序预算网格为 $A = (200, 100, 64, 32, 16, 8, 4, 2)$。校准将候选预算的 top-10 标识符集合与 $\alpha_{\max} = 200$ 的参考结果比较。对于样本 $\mathcal{Q}_S$，接受统计量为

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(10 - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{200}(q)|\right), \qquad D_S(\alpha) \le 1. \tag{7.1}$$

允许的是整个样本合计一个结果槽位不一致，而不是每条查询允许一个，也不是基于 ground truth 的召回率容差。实际规则不接收标签。参考结果本身也是近似结果，因此完全一致也可能复制其错误。

离线 oracle 的作用不同。它查看八个固定预算在留出池上的召回率后，在距最佳实测留出召回率不超过 10^{-3} 的配置中选择最快者。这是诊断性比较标准，不是可直接部署的无标签决策。“全校准池”同样表示在全部 500 条校准查询上应用同一校准搜索，而不是利用留出标签枚举所有预算。这些区分避免将“选到与 oracle 相同预算”误解为质量保证。

7.5.2 全池校准作为质量优先工作点

全池校准每个单元使用全部 500 条校准查询一次。选中预算在 12/12 单元均满足 10^{-3} 留出损失目标，最大观测损失为 0.0006；速率为 GT oracle 的 0.908–1.000，校准为 9.5–60.3 ms。相对固定 100 的几何平均速率比为 1.111×，仅计该对照也达标的十项则为 1.193×。完整比较见表 7.9。相对固定 64，全池选择六项更快、两项相等、四项更慢，其主要优势是十二项质量目标均观测达标。

这些结果支持将全池校准作为该工作负载族的回顾性质量优先工作点；$S = 128$ 是下文评价的较低校准成本替代。第6.2节的二项分辨率讨论说明查询聚集的重要性，不能把全池结果升级为 10^{-3} GT 损失的 95% 保证。

7.5.3 低成本采样及其质量风险

表 7.10 给出了 $S = 128$ 的结果。十二个单元中，所选预算的中位数有十个与 oracle 预算一致；不一致的两个是 arXiv 的 128 探针配置和 OpenAI-3072 的 512 探针配置。所选预算跨越 4–200，显示出明显的工作负载依赖。然而，预算中位数与平均召回率描述的是抽样总体的不同方面，二者不一定对应某一个配置的性能。此外，表中平均召回率由日志打印的最佳召回率减去打印的平均损失重构，所以保留到四位小数的值是近似值。

一个重要反例是 arXiv。在 128 探针下，即使平均损失仅为 0.0009，仍有 40/64 次抽样的召回率损失超过 10^{-3}；512 探针下有 32/64 次超标。BGE 的 512 探针配置有 24/64 次超标，95 分位损失为 0.0014。Stella 超标次数较少，但两个探针数下的 95 分位损失均为 0.0030。因

表 7.9. 全池校准（$S = 500$）。损失相对最佳实测留出 recall；速率为所选固定预算的 kQPS。“校准”每项仅一次全池调用，不是重复查询流计时。留出为 500 条，Stella 为 499。

数据集	探针数	α_{full}	损失	Q_{full}	相对 100	相对 oracle	校准 ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

表 7.10. $S = 128$ 时的留出质量。规则的 α 是 64 次抽样的中位数，oracle 的 α 使用留出标签。$\overline{R}_{\text{rule}}$ 是重构的平均召回率。“超标” 统计相对最佳被测留出召回率损失 $> 10^{-3}$ 的抽样次数；L_{95} 是日志采用较低次序统计量得到的 95 分位损失估计。R_{100} 是固定预算召回率。

数据集	探针数	α：规则/oracle	R_{100}	$\overline{R}_{\text{rule}}$	超标	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

此，平均质量、超标频率与尾部损失幅度必须一起看。OpenAI-3072 在这些抽样中未观察到超标，这支持其在当前查询池上的有利表现，却不是对未见查询的零风险保证。

对于数量较少的抽样，分位数约定也很重要。测试程序报告排序损失中从零起算的 $\lfloor 0.95(64-1) \rfloor = 59$ 项，而不是在次序统计量之间插值。因此，可能同时存在四个非零尾部损失，而打印的 $L_{95} = 0$，OpenAI-1536 就是如此。表格保留日志统计量与准确超标次数；该列为零不等于所有抽样都没有损失。

图 7.6 显示了样本大小的影响。更大的样本通常使选择更保守，但同时收紧了归一化允许量 $1/(10S)$。在全部 2,304 次抽样（$12 \times 3 \times 64$）中，同样本枚举与二分搜索一致，而打印日志显示枚举耗时为其 1.75–2.01 倍。

被测默认路径执行精确主码 top-c，并在全部保留评分上做最终选择；有损 `K_LOCAL=4` 路径需通过 `JHQ_EXACT_TOPCK=0` 显式启用。推论 6.2证明共同完整键分组配置下的参考一致性单调，覆盖 2,304 次抽取中的 2,176 次。剩下 128 次是 $S = 128$ 下 OpenAI-3072 的距离截断并列路径，经验上仍与枚举一致。八个单元校准到评测的换路涉及预算迁移，区别于同次抽取内搜索正确性。

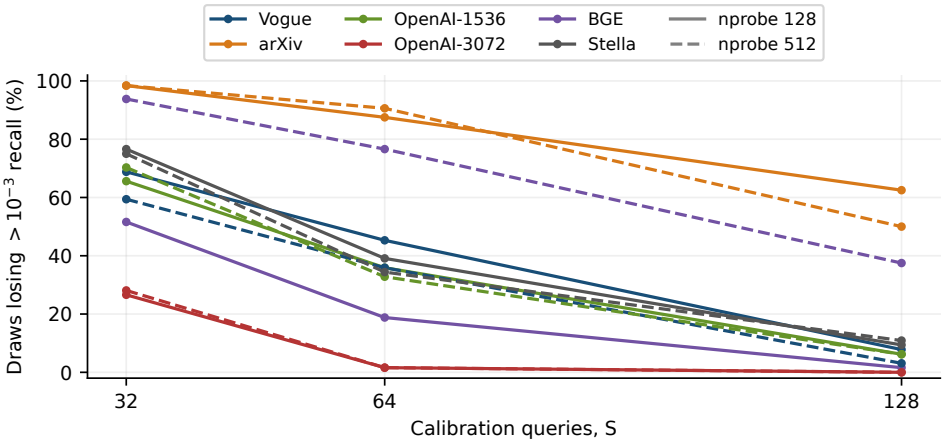


图 7.6. 不同样本大小下的留出风险。纵轴为相对最佳被测预算，留出召回率损失超过 10^{-3} 的校准抽样百分比。由于式 (7.1) 始终保持阈值一，增加 S 也收紧了允许的槽位不一致比例。

表 7.11. 十二个留出单元上的固定预算对照。达标表示按打印精度，距最佳被测召回率不超过 10^{-3}。最大召回率损失使用全部单元，QPS 差额只统计该预算达标的单元。横线表示没有达标单元。

α	达标单元	最大召回率损失	达标单元最大 QPS 差额
2	0/12	0.3100	–
4	2/12	0.1774	2.3%
8	4/12	0.0930	2.1%
16	6/12	0.0408	4.4%
32	7/12	0.0170	9.1%
64	10/12	0.0050	26.6%
100	10/12	0.0016	40.9%
200	12/12	0.0000	57.1%

固定预算也并不完美。表 7.11 按诊断性质量目标评估了全部八个固定选项。固定 $\alpha = 100$ 在十二个单元中有十个达标；$\alpha = 200$ 全部达标，但可能牺牲较多吞吐量。全校准池校准也不等于 oracle：在 Vogue 两个探针数下，它均选择 100，而 oracle 选择 64。即使增加校准样本，基于输出一致性的选择与“质量约束下最快”的选择仍解决不同问题。全校准池耗时为 9.5–60.3 ms。在这十二个单元中，$\alpha = 200$ 达到最高实测召回率，因此作为高预算覆盖对照；12/12 达标不是独立的质量保证。短固定臂计时出现表面的小预算反转；补证交替重复将所引 OpenAI 反转解释为计时变化，并建立小预算平台（第7.5.5节）。最小接纳预算是质量决策，不保证为测得最快的预算。

7.5.4 估计吞吐量与校准经济性

与 E1 不同，这组实验在预热后，对每个固定配置进行三次搜索计时。规则报告的 QPS 是 64 次抽样所选配置的已有实测 QPS 的算术平均，而不是对一个重复执行“校准—搜索”的服务进行新计时。当 $S = 128$ 时，这一所选配置吞吐量估计在十个单元中优于固定 $\alpha = 100$，比值为 1.028–1.691×。arXiv 的比值则为 0.908 和 0.917：平均召回率提高，但吞吐量降低。这两个单元无法通过吞吐量收益回本。

更强的 $\alpha = 64$ 对照缩小了整体收益：所选配置的平均 QPS 在 9/12 个单元中胜出，几何平均比值为 1.054×，范围为 0.821–1.367×。Vogue 几乎不变；BGE 的 512 探针配置只提高 0.6%；OpenAI 的收益范围为 1.052–1.367×。固定 64 和固定 100 都在 10/12 个单元达标，

表 7.12. 校准经济性，$S = 128$。速率单位为 kQPS，规则速率为所选配置平均 QPS。比值分别相对固定 100 与固定 64。“回本批数”相对固定 100，使用 500 条查询，Stella 为 499。“不回本”表示没有正吞吐节省；质量列独立标记固定 100 是否满足留出损失目标。

数据集	探针数	Q_{100}	$\overline{Q}_{\mathrm{rule}}$	相对 100	相对 64	校准 ms	回本批数	固定 100 质量
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	达标
arXiv	128	207.9	188.8	0.908	0.821	4.2	不回本	未达标
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	达标
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	达标
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	达标
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	达标
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	达标
arXiv	512	83.1	76.2	0.917	0.870	9.7	不回本	未达标
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	达标
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	达标
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	达标
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	达标

失败的都是两个 arXiv 单元。校准的价值是适应不同精排需求，而不是始终击败已经合理的固定预算。

arXiv 的超标是相对最佳被测留出召回率，不是相对默认配置。固定 100 在 128 和 512 探针下本来分别损失 0.0012 和 0.0016，而抽样平均损失在两个单元均为 0.0009。因此，即使部分抽样超过相对 oracle 的目标，平均质量仍比默认配置更好。

日志中的校准成本为 3.9–21.3 ms。它包含参考搜索与输出一致性检查，以及测量搜索调用内包含的相关设置，但不包含将抽样向量收集到输入缓冲区的操作。对于相同大小的后续批次，定义

$$B^{*} = \frac{T_{\mathrm{cal}}}{T_{100} - T_{\mathrm{rule}}}, \qquad T_{\mathrm{rule}} < T_{100}. \tag{7.2}$$

使用 $T_{\mathrm{rule}} \approx n/\overline{Q}_{\mathrm{rule}}$ 得到表 7.12中的估计：有利单元相对固定 100 约需 2.4–55.4 个留出池大小的后续批次。相对固定 64，九个有利单元约需 5.7–404.3 批，Vogue 的 512 探针配置没有吞吐量回本。两个 arXiv 对照也未满足质量目标，因此其计时比较是诊断，不是同质量比较。按对照速率换算，$S = 128$ 的最大正回本估计相对固定 100 为 0.54 s、相对固定 64 为 2.72 s，约 27.7K 和 202.2K 查询。全池正收益项按自身校准成本计算，对应最多 1.52/2.25 s、64.9K/78.7K 查询。这些较短的模型时域支持长稳定任务复用；图 7.7保留实际留出查询数。

这种计算偏乐观。一般而言，$1/\mathbb{E}[Q] \le \mathbb{E}[1/Q]$，所以当不同校准抽样选择了速率不同的配置时，对平均 QPS 取倒数会低估平均执行时间。样本收集、未来查询漂移与重新校准开销也会影响摊销。对于长期稳定的工作负载，复用策略可能带来收益；短任务或变化的查询总体则未必如此。当前实验没有测量漂移检测，也没有测量生产环境中的重新校准计划。

作为历史比较，较早的标量精排实现（v57）在 10/12 个单元中报告相对固定 100 的 1.054–2.773× 收益，校准耗时为 5.2–30.8 ms。较新的 lane 精排流水线收益更小，与可节省精排时间减少的解释相容。两次运行共享预算协议和网格，但跨实验比较并未隔离证明精排是变化的唯一原因。

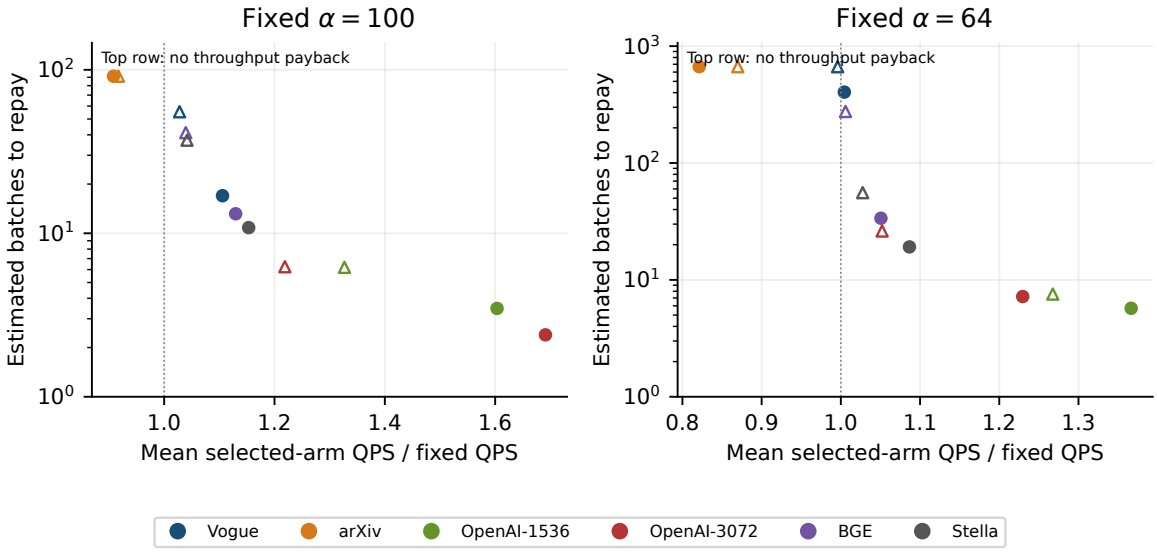


图 7.7. 相对固定预算 100 和 64 的校准回本估计。实心圆表示 128 探针，空心三角表示 512 探针；顶行点没有吞吐量回本。估计对所选配置平均 QPS 取倒数，使用 500 条查询，Stella 为 499。

7.5.5 完整决策轨迹与有限池风险

补证程序保留原网格、训练状态、查询划分与固定 seed 抽样，新增完整日志。十二单元的固定预算 recall 在归档精度下全部复现，全池选择、采样中位选择与越阈计数也全部复现。计时让两个近乎并列单元的 GT oracle 最快臂变化，说明这一诊断也依赖所测速度。

2,304 条采样不一致曲线均单调，选择全部与枚举一致；十二条全池曲线也通过两项检查。进一步，将全池逐查询计数按各次实际样本 ID 求和，18,432 个网格计数全部复现。这比只核对最终预算更强，支持在这些观测中使用式 (6.9)，包括换扫描路径的情形；它验证已记录子集的汇总计数，不证明任意子集的逐查询不变性。表 7.13报告观测尾部大小。

表 7.13. 每个样本量 64 次抽取中的最大留出 recall 损失，相对最佳被测预算。十二个 FULL 选择均与枚举一致；最大值补充之前的均值、p95 与越阈率。

数据集	探测数	全池 α	最大 L_{32}	最大 L_{64}	最大 L_{128}
Vogue	128	100	0.0104	0.0104	0.0022
arXiv	128	200	0.0134	0.0024	0.0024
OpenAI-1536	128	8	0.0018	0.0018	0.0018
OpenAI-3072	128	4	0.0046	0.0046	0.0000
BGE	128	64	0.0100	0.0038	0.0038
Stella	128	16	0.0174	0.0030	0.0030
Vogue	512	100	0.0112	0.0030	0.0030
arXiv	512	200	0.0170	0.0050	0.0050
OpenAI-1536	512	8	0.0272	0.0026	0.0026
OpenAI-3072	512	4	0.0072	0.0072	0.0002
BGE	512	64	0.0294	0.0046	0.0046
Stella	512	16	0.0182	0.0030	0.0030

$S = 128$ 时，arXiv/512 最大损失 0.0050、BGE/512 为 0.0046；$S = 32$ 最大达 0.0294。因此较小平均损失可以伴随少数明显更差的选择。十二个 FULL 单元的最大损失仍为 0.0006。

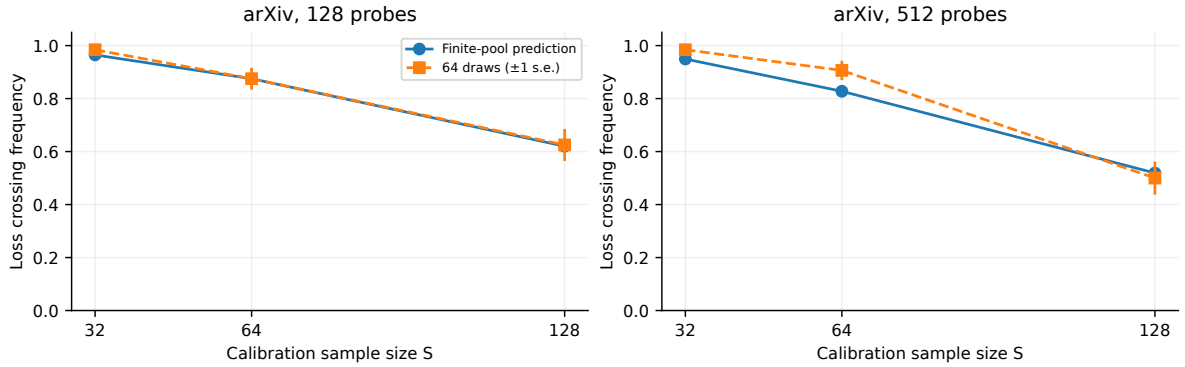


图 7.8. arXiv 损失越阈的有限池预测与观测。误差条是在固定池条件下 64 次独立抽取子集的一倍估计标准误，不是未来工作负载的不确定性。

arXiv/128 在 $S = 32, 64, 128$ 的有限池越阈预测为 0.965、0.875、0.620，实测为 0.984、0.875、0.625（图 7.8）。接纳只用参考输出；将选择概率转为 GT 损失越阈率还要固定预算的留出质量映射，因此这是离线风险解释，不是无标签 GT 保证。36 项比较中最大绝对差为 BGE/512 在 $S = 128$ 的 0.144（预测 0.519、实测 0.375）。有限池计算隔离抽样风险，没有用留出 recall 差拟合总体模型。

十轮交替重测也消除了表面的小预算速度反转。OpenAI-3072/128 在 $\alpha = 4/2$ 时为 142.3 / 142.7 kQPS，原短计时为 145.0 / 128.7；BGE/128 与 Vogue/512 同样趋于平台。这支持将归档反转解释为计时变化，但不代表所有预算和资源阈值上的运行时间均单调。

7.5.6 直接校准加复用测量

四个单元直接执行固定 64、固定 100、采样 $S = 128$ 与 FULL 策略，五轮间轮换策略顺序。每个校准轮先收集样本、校准一次、显式设置所选乘数，再连续计时 500 个留出批次。首批包含可能的 graph 重新捕获，重复使用同一留出池，没有采样新流量。索引构建、此前预热及另行计时的调度校准不在该边界内。表 7.14报告累计测量。

表 7.14. 500 个复用批次的直接总毫秒数，五轮平均；采样与 FULL 包含预算校准，每批 500 查询。质量与计时结果一起讨论。

数据集/探测数	固定 100	固定 64	规则 $S = 128$	全池
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

OpenAI-3072/128 的采样策略每轮均选 4，相对固定 100 将总时间从 2,757 降到 1,769 ms，约节省 36%，测得 recall 同为 0.9252。FULL 选同一预算，含较高校准成本共 1,783 ms。OpenAI-1536/128 的 FULL 为 860 / 固定 100 的 1,491 ms，recall 同为 0.9280；采样策略平均 852 ms、recall 0.9276。

从直接计时的校准和稳态批次拟合，采样策略相对固定 100 / 64 的回本分别为：OpenAI-1536 的 3.6 / 5.5 批、OpenAI-3072 的 3.5 / 8.9 批。小数值是稳态速率拟合，不是直接观测到的小数批次交点；图 7.9绘制含首批设置的实测累计总时间。相对固定 100 / 64，平均累计时间首次正节省在 OpenAI-1536 为 4 / 6 批，OpenAI-3072 为 4 / 9 批。BGE/512 采样收益较小，相对固定 100 为 4,921 / 5,185 ms，平均 recall 为 0.9658 / 0.9670；FULL 恢

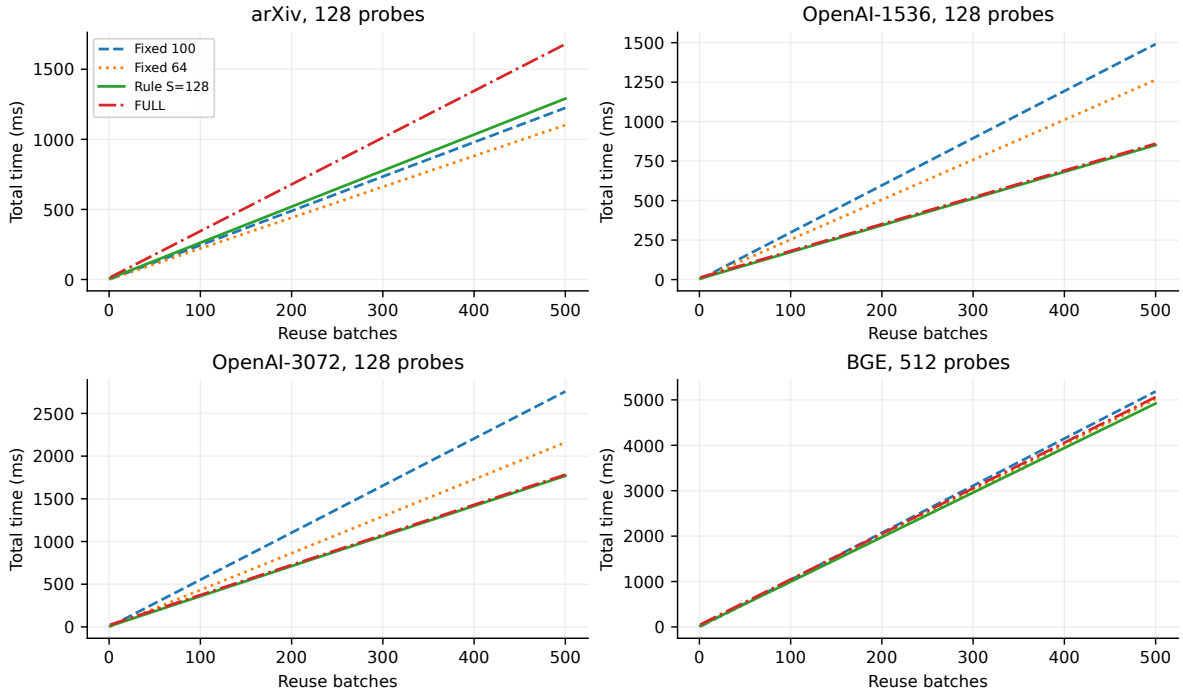


图 7.9. 含预算校准及首批设置的直接平均累计时间，各曲线重复同一固定留出池。不同策略 recall 可不同，因此不能仅按时间作质量匹配排名；配置对应表 7.14。

复至 0.9664，耗时 5,062 ms。arXiv 的 FULL 为提高质量选择 200，慢于固定 100 / 64，而两个固定控制均未达到质量目标。因此必须联合质量和时间判断校准价值。测量支持复用经济性，不要求旧模型回本值在每个单元精确吻合。

7.6 批大小、扫描组调度与工作负载偏斜

本节回答 RQ2 的性能区间问题：何时额外扫描组提供有价值的并行度，何时额外工作成为主导。

7.6.1 并行度与重复选择工作的权衡

扫描实验揭示了两种执行区间。每条查询使用一个线程块时，批大小 8 只在具有 170 个 SM 的设备上提供八个线程块。划分候选能增加可用线程块并行度。在 v62 实验中，批大小 8 使用八组后，端到端吞吐量为原来的 2.06–2.76×。剖析给出了阶段解释：arXiv 扫描约从 849 降至 216 μs，Vogue 从 1,059 降至 246 μs，OpenAI-3072 从 728 降至 155 μs。额外合并分别约需 16.7、9.1 和 3.7 μs。即使支付合并成本，净节省仍很大。

批大小为 1,000 时，独立查询已足以提供较多线程块。增加分组会重复搬入查找表、接纳局部候选并排序，还产生额外合并流量。对于 arXiv 的 $\alpha = 200$ 配置，不共享界的 v62 路径可能降至单线程块基线的约 0.46×。全批次剖析显示，$\alpha = 100$ 的扫描从 2,922 增至 6,079 μs，$\alpha = 200$ 从 3,530 增至 7,652 μs，后者还增加了 2,624 μs 合并时间。这一反向结果对设计至关重要：并行拆分能够解决并发不足，但当并发已经充足时，它引入的额外工作可能成为主导。共享界减少了这些额外工作，却没有消除执行区间的变化。

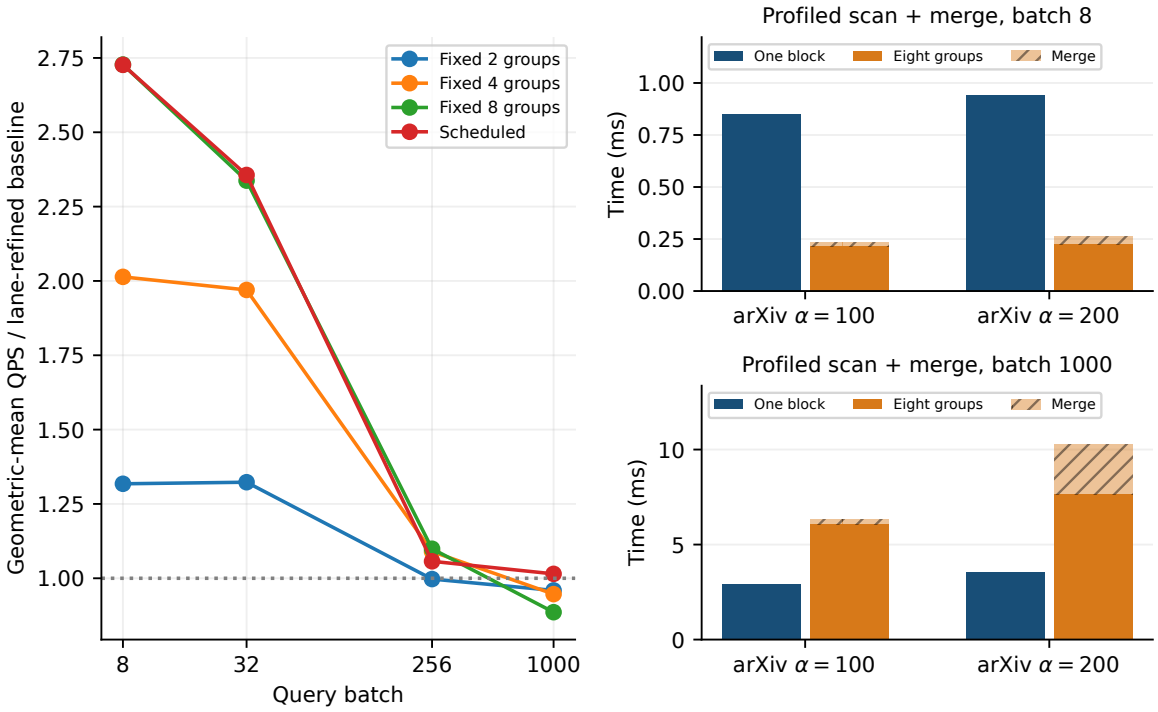


图 7.10. 执行区间。左：共享界固定组数与调度执行，相对 lane 精排单线程块基线的吞吐量比，较小批次有 13 个单元，批大小 1,000 有 11 个。右：arXiv 预算 100 和 200 在批大小 8 与 1,000 下的配对扫描、合并剖析；斜线表示合并时间。右侧来自独立界扫描实验，用于解释一项成本，不代表调度器全部收益。

表 7.15. 集成执行相对 v61 L16 的收益。每项先计算单元内五轮配对加速比的中位数，再跨单元取几何平均。“留出配置”排除了设置规则时使用的四个配置，指的是未用于设定规则的配置，不是预算实验中的留出查询划分。

批大小	单元数	$g_s = 2$	$g_s = 4$	$g_s = 8$	调度	留出配置
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

7.6.2 在更多配置上评估预先固定的规则

集成批调度器（v65）根据批大小、线程块大小以及工作负载偏斜估计选择扫描组数。填充规则将潜在线程常驻数量与设备线程容量的固定 0.6 比例关联；偏斜规则则在每条查询候选数的变异系数至少为 0.3 时，要求至少使用四组。最终取两条规则建议的较大组数；一组情况映射到原始单线程块扫描。偏斜统计只估计一次，使用 512 个数据库向量作为伪查询，不使用 ground-truth 标签或被测查询池。规则常数在四个工作负载配置上确定，之后才测量额外配置。

表 7.15 从原始计时数组重新计算结果。批大小为 8、32、256 和 1,000 时，调度收益的几何平均分别为 2.727×、2.356×、1.057× 和 1.015×。前三种批大小有十三个配置，其中包含额外的 arXiv 高预算配置；批大小 1,000 只有十一个，因为 Stella 的 999 条查询池无法组成一个完整的 1,000 查询批次。对应的留出配置子集分别含九个和七个配置。因此，将这些均值连接起来的图总结的是运行区间，而不是完全平衡的重复测量总体。

表 7.16. JHQ 冷构建与单独测量的常驻显存。训练 / 添加为三次补证正常构建的分项中位数，合计为配对和的中位数。显存来自 v71 两组、128 探针、$\alpha = 100$ 的运行，包含上下文与工作区。“残差占比”表示模型计算的残差码占实测已用总量的比例，而不是每次查询读取的比例。

数据集	训练 s	添加 s	合计 s	已用 GiB	原始 GiB	残差占比
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

固定八组在批大小 8 时的均值接近调度结果，但在批大小 1,000 时降至 0.886×。调度大体保留了小批次收益，并避免上述大批次损失。不过，它仍是启发式规则，不是 oracle。批大小 256 时，在一个 BGE 配置上，“最佳固定吞吐量 / 调度吞吐量”的最差比值为 1.217。候选数 CV 无法描述列表遍历、选择或合并的全部成本。这一失败表明所选特征存在局限，并不说明执行调度没有必要。

Vogue 同时说明偏斜特征的价值与局限。E1 候选数 CV 随探针数在 0.338–0.622 变化，使批大小 1,000 仍采用四组。集成路径相对单线程块的比值为 0.895–1.175；在 $(n_{\text{probe}}, \alpha) = (16, 16)$、$(16, 64)$、$(32, 64)$ 时变慢，比值依次为 0.955、0.895、0.955。候选数离散程度没有编码每条查询的绝对工作量，因此规则可能给这些低探针配置增加开销，同时又支撑表 7.3中 0.97 的跨系统领先。这与 BGE 批大小 256 的 regret 互补：平衡工作和支付分解成本是两项不同的调度决策。

最后，正确性检查独立于计时。在 250 个变体/配置比较、234,320 次逐查询比较中，集成实验未发现非并列主码选择差异，也未发现最终标识符集合差异。在并列情况下，最终顺序可能不同，因而不能将整个实验描述为无条件逐位一致。吞吐量结果针对顺序同步提交，不足以建立开放到达过程下的尾延迟表现。

7.7 冷构建、常驻显存与已验证规模

通过冷构建、常驻显存与已完成的大数据集搜索评估 RQ1；E1 显存比较也为 RQ4 提供依据。

7.7.1 冷构建与生命周期边界

主要搜索扫描复用了已训练状态缓存。加载该状态并添加向量不属于冷构建。独立日志明确在训练状态缓存为空时训练，然后编码并添加数据库；表 7.16给出这些记录中的训练与添加时间。补证重复沿用空训练缓存边界，表 7.16改报每数据集三次正常构建中位数，表 7.17保存范围、较宽启动区间及采样设备峰值。Vogue 训练 298–303 ms，正常合计约 0.56 s，归档 1.33 s 为一次偏慢观测；Stella 为 10.54–10.65 s，接近历史 10.40 s。

训练 / 添加计时在索引对象构造和样本准备之后开始，因此不包含主机 QR 旋转矩阵构造及构造器中的 cuBLAS 预热。这里的“冷”指已学习索引状态。输入加载器可复用内存映射 FP32 数据；报告的训练加添加区间不包含从存储设备读取原始导出文件。调度器的伪查询校准在添加之后进行，也不计入总量。完整部署生命周期还应考虑数据准备、某些接口未计入的传输、校准、序列化和后续查询时间。这些记录是在明确边界内有价值的构建测量，不是完整端到端数据摄取服务的成本。

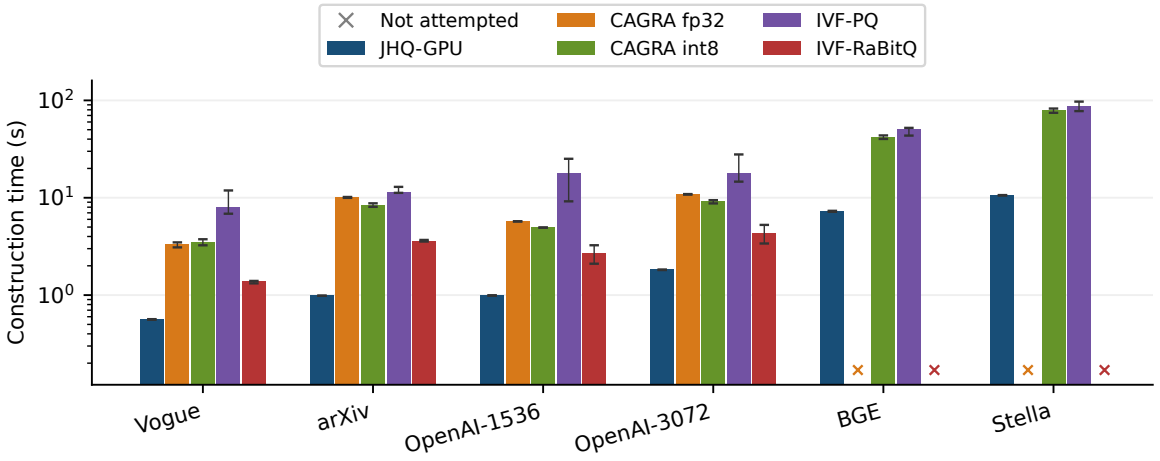


图 7.11. 包括 CAGRA int8 的已记录构建成本。JHQ 为热输入页下三次补证正常冷构建的中位数和范围；其他柱形为成功测量的索引配置之间的中位数，误差线表示最小值至最大值。Int8 包含数据集量化；RaBitQ 不包含所需的序列化往返。大数据集缺失的柱形表示未尝试的配置，不代表实测构建失败。纵轴为对数坐标。

比较图同时给出了其他构建配置与这些冷构建记录。每种方法的范围不是等价重复构建的置信区间。训练样本、索引参数与表示不同，JHQ 现采用热输入页下三次独立正常冷构建，其他方法保留历史配置范围。对于 RaBitQ，记录中的构建函数与序列化/反序列化分开计时，但基准必须完成该往返后才能搜索。只有将柱形明确标为构建函数时间时，才可以不计这一步；它并不表示直到第一次可用搜索的全部时间。

在 Stella 上，CAGRA int8 成功的隔离构建按图出度不同，包含数据集量化后约为 74.55 和 82.45 s；IVF-PQ 三个配置的实测构建中位数为 86.59 s。在报告边界内，JHQ 重复 10.54–10.65 s 的区间明显更短，但不能据此证明它在所有条件下构建最快，也不能证明后续召回率相同。构建速度、可搜索质量与常驻成本是部署时需要分别考虑的维度。

表 7.17. 补证空缓存构建。训练 +add 和启动列为三次正常热输入页运行的范围；首次读取列为本会话首次读取的启动观测。启动计时结束于 add 完成、调度校准 / 搜索之前。峰值为正常运行中 20 ms 设备采样最大值，含上下文。召回范围来自固定 128 探针工作点五次独立构建，包含阶段诊断。

数据集	训练 +add (s)	启动 (s)	首次读取 (s)	峰值 (GiB)	召回范围
Vogue	0.558–0.566	2.0–2.1	4.9	4.11	0.9636–0.9644
arXiv	0.984–0.987	3.1–3.2	9.1	7.83	0.9757–0.9769
OpenAI-1536	0.986–0.996	4.1–4.3	10.0	7.66	0.9313–0.9317
OpenAI-3072	1.809–1.830	7.1–7.2	17.0	11.39	0.9266–0.9268
BGE	7.172–7.341	13.6–15.9	51.4	26.08	0.9174–0.9178
Stella	10.543–10.653	18.2–18.6	129.9	22.58	0.9887–0.9890

较宽启动边界的差别明显：Stella 热输入页 18.2–18.6 s，本会话首次读取 129.9 s。阶段诊断同步各阶段并改为一个流水缓冲区，因此其计时不混入正常构建范围。20 ms 设备采样观察到 BGE 在 add 开头 26.08 GiB、Stella 在 repack 时 22.58 GiB。BGE 三个分配乘积缓冲区共 12 GiB，属于早期流水生命周期，不能直接加到后期 23.05 GiB 的 gather 载荷（式 (4.12)）。因此常驻显存不足以代表构建容量。

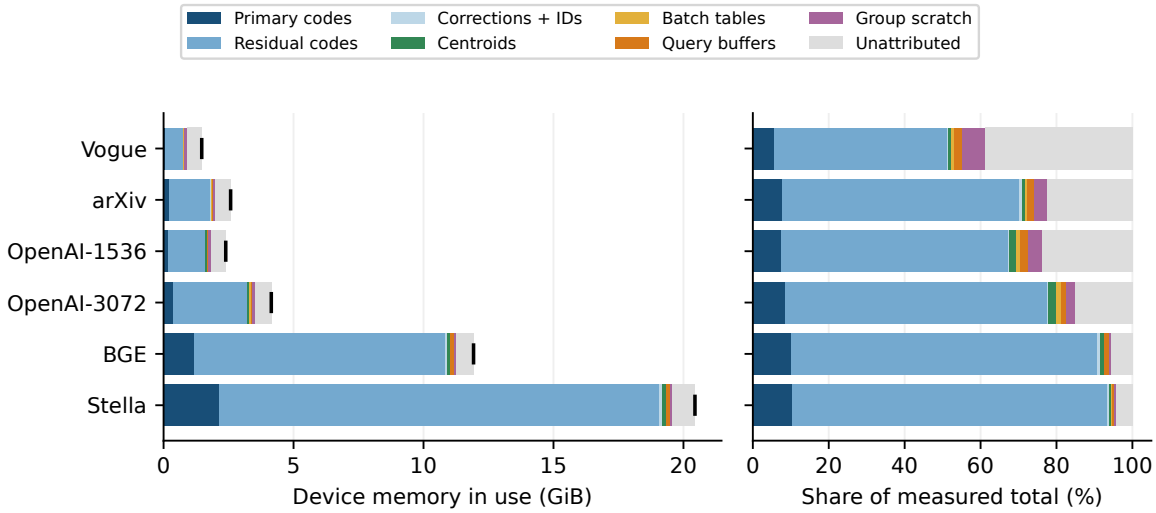


图 7.12. 两组查找表、128 探针、$\alpha = 100$ 下的常驻显存分解。组件大小由模型计算，标记为实测已用总量，斜线区域为未归因部分。总量包含上下文与工作区，另行构建采样见表 7.17，这里的常驻标记不是峰值。

7.7.2 常驻分配与压缩发挥作用的位置

图 7.12 分解常驻显存。标记表示记录时设备已用显存，而不是纯索引分配。堆叠部分用模型计算主码、残差码、修正因子与标识符、质心、每批查找表、候选缓冲区和列表分组临时区。剩余实测分配明确保留为未归因部分；它可能包含运行时上下文、分配器影响以及模型遗漏的分配。将剩余部分全部归给残差码，会高估表示成本。

在大数据集上，残差码是主要占用：BGE 的已用总量为 11.93 GiB，其中约 80.7% 为残差码；Stella 为 20.44 GiB，其中约 82.9% 为残差码。仅对应的原始 FP32 向量数组就分别需要 38.50 和 67.81 GiB。因此，JHQ 通过保留压缩层次表示，使这些已测试的数据能够在单卡常驻搜索。降低 α 会减少每条查询访问、评分的残差码，但不会释放已经存储的残差数组。同样，笛卡尔 LUT 因子化减少的是查询相关查找表存储，不是数据库残差码的精度。

工作区同样重要。在这些 $\alpha = 100$ 测量中，报告的列表分组临时区约为 93.8 MiB，即使实际执行选择单线程块路径，也为批容量 1,024 和最多八组预留。因此，实际组数变小，不意味着分配按比例缩小。另一方面，这些常驻样本没有捕获临时构建峰值。基线测试程序也在测量索引显存之后才初始化部分搜索内存池，因此，不能把构建后的索引增量当作各方法完整进程峰值显存预算。

E1 测量提供了表 7.18中的独立跨系统视图。从 JHQ 进程总量减去基线测得的 507.2 MiB 上下文后，约为 CAGRA fp32 索引分配的 0.29–0.39。JHQ 数字包含工作区，CAGRA 数字则是构建时索引增量。这些是参数扫描范围，不是显存匹配的同召回率运行点，也不是相同峰值预算的比较。同表的 CAGRA int8 分配与 JHQ 处于相近量级，相对 fp32 的压缩是多个设计共有的特征；组件模型见表 4.2。

7.7.3 大数据集可运行性与竞争区间

BGE 含 10,091,524 个向量，Stella 含 17,776,615 个向量，均为 1,024 维、32,768 个 IVF 列表。Stella 查询采样自语料库，因此结果描述的是该基准，而不是独立收集的外部查询分布。图 7.13 包含成功的 CAGRA int8 和 IVF-PQ 测量。对于计算存储需求超过可用设备容量的

表 7.18. E1 分配范围。JHQ 为进程总量减去 507.2 MiB 上下文估计，包含查询工作区；两个 CAGRA 列均为构建时测量的索引分配。范围覆盖参数扫描。

数据集	JHQ 净进程 GiB	CAGRA fp32 索引 GiB	CAGRA int8 索引 GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

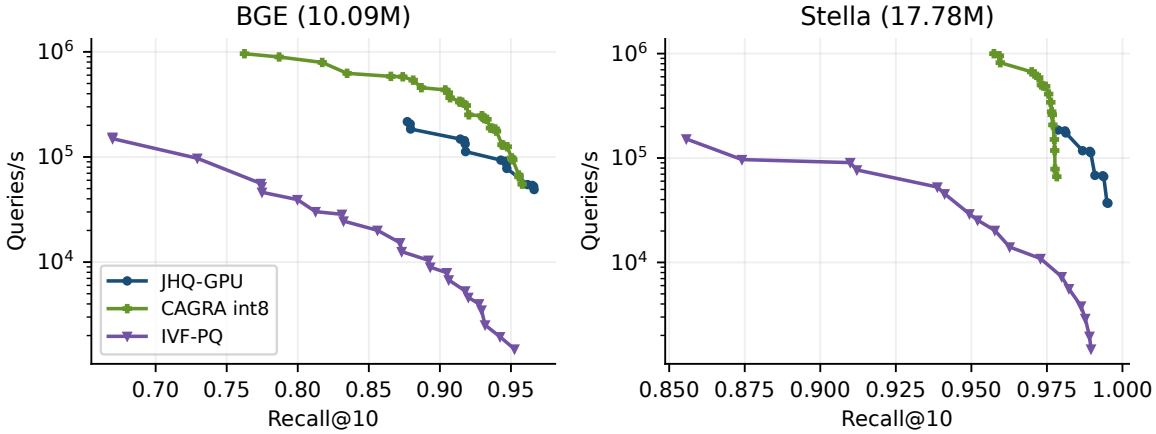


图 7.13. 大数据集上的主机到主机前沿，包含 CAGRA int8 成功的隔离重跑。BGE 使用 1,000 条查询，Stella 使用 999 条。曲线端点是已测试网格的终点，不是方法的内在召回率上限。

全精度 CAGRA、RaBitQ 配置和 FP32 暴力搜索，大规模比较没有尝试运行。这些缺项是预先进行的可运行性排除，不是观察到的显存耗尽错误或实测召回率失败。

成功的隔离重跑显示，CAGRA int8 能在被测配置下装入 Stella。在 BGE 的共同采样目标 0.88/0.90/0.92/0.94/0.95 上，int8 约比 JHQ 快 1.35–2.96×；在 int8 的最高实测召回率 0.9579 处，JHQ 插值 QPS 是 int8 的 1.086×。因此，优势在高召回率末端附近反转。BGE/Stella 上，int8 的最高被测召回率为 0.9579/0.9783，JHQ 为 0.9663/0.9952，PQ 为 0.9525/0.9896。

Stella 上一个具体高端比较为：JHQ 在召回率 0.978679 时达到 185.4 kQPS，int8 在 0.978278 时为 66.3 kQPS，速率比为 2.80×，两者召回率略有不同。这是一对明确的实测点，不是统一的匹配召回率优势：JHQ 很多更高召回率区间没有 int8 覆盖，而且其最高召回率点也以吞吐量换取质量。附录 A记录修正运行的排除规则与测试程序诊断。

结果证明的是已测试规模和有价值的高召回率区间，而不是任意规模的可扩展性。最大已完成搜索是在单 GPU 上处理 17.78 百万个向量；本章没有多 GPU 实验，没有保持数据分布不变的扩展曲线，只有采样构建峰值，没有逐分配精确的容量证明。Stella 搜索使用实际的 999 条查询池。

7.8 数值稳健性、异常与召回率归因

稳健性检查界定 RQ2 的精确性声明、RQ3 的风险评估与 RQ4 中较小的跨系统差距。

7.8.1 质量分辨率与重复执行

当 $k = 10$、查询数为 1,000 时，多找回一个邻居会使总体 Recall@10 改变 10^{-4}；500 条留出查询时，分辨率为 2×10^{-4}。Stella 的 499 查询划分略粗。这种离散分辨率不是不确定性区间，但有助于理解前面的小差异。第四位小数的变化可能只对应一两个结果槽位，而小的平均损失仍可能掩盖若干答案变化较大的查询。

重复调用与重复索引构建回答不同问题。E1 保留了 369 个基线重复调用标识符标记，包括并列顺序变化；这些点仍在前沿中。401 的计数口径及跨协议检查在第A.2节中单独说明。

五轮范围描述固定构建内部的计时变化。另用前沿斜率量化水平 recall 位移的敏感性：在 0.97 邻近的连续 log-QPS 线段上，每 10^{-3} recall 对应约 1.01–3.17% 的速率变化，此处尚未应用接近目标即取实测点的规则。补证五次构建在固定工作点的 recall 范围为：Vogue 0.0008、arXiv 0.0012、OpenAI-1536 0.0004、OpenAI-3072 0.0002。仅作水平敏感性尺度乘以上述局部斜率，对应约数个百分点，而非更大的 OpenAI 相对 CAGRA fp32 差距。未重测完整训练前沿或基线构建，因此它支持局部稳定性判断，不给匹配比值的普遍上界。第7.2.3节则在明确缓存模型边界下直接证明去掉 TF32 后 OpenAI 优势仍在；arXiv 与 LUT 接近平手的结论仍保持较窄解释。

在各数据集 RaBitQ 配置间，“五轮最大值减最小值 / QPS 中位数”的中位数分别为：Vogue 3.79%、arXiv 3.34%、OpenAI-1536 2.87%、OpenAI-3072 5.25%。这些描述性波动与 Vogue 小幅领先的量级接近，而不是匹配比值的置信区间。原始调用轨迹也包含长调用，在原本较快的配置中偶有 30–60 ms 观测。敏感性检验对 1,000 条查询的一次调用，按 $10^6/\,\mathrm{median(call\ latency\ in\ ms)}$ 重算 QPS，以减小长调用的影响。召回率 0.97 处，得到的 JHQ/RaBitQ 比值为 1.063、0.995、1.118 和 1.100。三个有利方向与 arXiv 接近平手的格局保持，因此结果不只是长调用造成的。

7.8.2 精确性必须限定到具体阶段

几项不同声明必须区分。笛卡尔因子化在代数上保持已表示评分；打包加载保持逻辑编码值；采用完整键的局部保留和合并恢复已访问候选中的精确主码 top-c；共享界保持该选择目标；已算分数的探针原语通过了合成与真实输入的等价检查。以上任何一项，都不等于在原始向量空间找到了精确最近邻。

lane 归约会改变舍入；主码截断边界的并列可能改变被精排的候选；最终选择对全部保留的精排评分进行精确选择，并按 ID 打破并列；相对原空间邻居的剩余误差可以来自此前阶段或已表示评分。归档中的 Vogue 跨版本差异说明，完整流水线的一致性需要独立检查。类似地，FP16 路由实验表明，几乎不变的平均召回率也不足以证明探针精确等价。因此，主要正确性结论都限定到实际测试过的阶段与输入覆盖。

归档重建 pilot 测量了相对全探针重建评分的剩余差距。七个工作负载—预算变体各用 128 条查询，全部 probe 候选的 FP64 评分相对所选预算的 GPU 输出，recall 增益为 0–0.002344。在 arXiv 的 128 探针配置，$\alpha = 200$ 与全 IVF 重建答案集合在全部 128 条查询上一致；100 时 recall 为 0.9734375，全 IVF 为 0.9750，即在联合路由 / 重建缺口 0.0250 之外，再有 0.0015625 的预算缺口（第6.2节）。这些是 pilot 特定的总体差异，七个变体包含同一数据集—探针工作负载的两个预算。FP64 重建评分也不同于 GPU 运算，因而不能在每个变体中单独隔离纯预算效应。进一步分开路由与表示，需要逐阶段 GT 覆盖检查。

7.8.3 研究发现的适用范围

表 7.19 收束 RQ1–RQ4 的证据链。结果支持少量面向表示的执行模式与工作负载相关精排工作量，并分别提供正确性、速度和质量证据。

表 7.19. 本章证据支持的四个研究问题答案。

研究问题	证据	答案	边界
RQ1	冷构建、常驻组件、大规模搜索	单卡 17.78M 向量；20.44 GiB；重复训练加添加 10.54–10.65 s	采样峰值 22.58 GiB；无多 GPU 扩展
RQ2	严格 CPU 参考及配对执行检查	55,296 个完整键实例顺序一致；batch 8 调度为 2.727×	T4 有限网格；FP32 表形式可改变末端 ID；大 batch 收益更小
RQ3	互斥查询、固定 64/100、全池、oracle、枚举	全池 12/12 质量达标，最大损失 0.0006	回顾性工作点；$S = 128$ 最多 40/64 越阈；四单元直接测量净成本
RQ4	E1、T5 前沿、两种 recall 估计与显存	E1：0.97 处三个领先；T5：batch 8、0.97 时 CAGRA 领先；分配低于 fp32 CAGRA、与 int8 相近	OpenAI E1 使用 TF32，FP32 领先更小；有限不等网格与接口成本；batch 32 边界依估计而变

结论适用于平方 L2 的 top-10 搜索、记录查询池与被测 RTX 5090 环境。跨设备行为、变化查询、在线更新与开放到达服务延迟尚未测试。这些边界与正面结果共同呈现，而不是取代正面发现。

第 8 章

讨论、局限与未来研究

测量支持结构化编码、选择性精排与工作负载相关执行的一种具体组合。更广泛的价值在于解释：表面上更便宜的操作，何时能变成更快的搜索。本章提出三项解释，分析两条未能支持普遍采用的路线，再把局限转化为可检验的问题。沿用第3.6节对实测效果、原因解释与未来扩展的区分。

8.1 三项跨层认识

8.1.1 有价值的表示暴露有价值的复用单位

Cartesian 码本允许把每份 primary 表按坐标精确拆成两张 16 项表。新版扫描中，该选择在 22/24 个单元领先；四组与八组都只存每子空间 16 项，却执行不同次数的查找（图 7.4）。因此，表示让紧凑评分器成为可能，而表大小不能单独决定最佳实现。真正复用的单位是查询相关坐标组：其距离构造一次，再被多个候选使用。

共享内存可容纳性解释了重要的资源转变：所测配置中，完整表加扫描 scratch 超过可用空间，两组表可以放下。继续缩表时，额外查找与累加开始竞争。packed word 同样在逻辑字节量不变时改变指令和寻址工作。因此应联合考虑状态、指令与复用；没有计数器时，无法进一步区分 cache、bank、寄存器和 issue。两个例外 LUT 单元及下文 Mailman 案例反驳普遍的“最小表最好”规则。两组默认来自实测取舍，不是跨设备最优结论。

这种机会比任意 PQ 更窄：这里的乘积结构存在于每个 primary 子码本内部。学习式 PQ 子空间的质心通常相互耦合，拆标签位不等于拆距离。反过来，为降低重建误差而改码本，也可能失去该执行优势。表示应结合评分器和残差层次可达的 recall–成本前沿评价，这把第 5 章的数学兼容性检查连接到第 7 章的端到端问题。

8.1.2 有效并行受重复状态与选择成本限制

相对 lane 精排参考，集成调度在 batch 8 的吞吐几何平均为 2.727×，batch 1,000 只有 1.015×（图 7.10）。小 batch 缺乏查询级并行时，把单查询分给多 block 可以补足工作；额外组也引入局部缓冲、重复 query / table staging 及合并。原 grid 已有足够 block 时，这些成本可吞掉收益。因此最大 occupancy 本身不能作为优化目标。

负载偏斜提供另一种分组理由。E1 大 batch 只有 Vogue 走分组，候选数 CV 为 0.338–0.622；0.97 处相对 RaBitQ 的比值，从单 block 的 0.933 变为调度路径的 1.059（表 7.3）。配对运行把执行机制连接到竞争区间，但测的是调度、保留、共享界与合并的整体，单独识别共享界需要自己的消融。浅 probe 的部分 Vogue 设置会变慢，BGE batch 256 的调度 regret 达到 1.217。CV 描述不均衡，却没有纳入绝对工作量、staging 和 merge 成本。

工程上应保留 query-centric 路径，并同时依据可用并行度与单查询预计工作选择分组。候选数、survivor 容量和局部缓冲成本是更丰富决策的候选输入。拟合选择器还需留出工作

负载与校准成本核算，目前观察未验证这类选择器。现有启发式的价值也在于其收益与失败单元可以分别复查。

8.1.3 执行变快会改变避免精排的价值

降低预算省去残差读取、计算与最终选择，路由和广泛 primary 评分仍保留。可省比例取决于这些操作原本占多少延迟。更快的 lane 精排可能降低减少候选的回报，更快 primary 扫描则可能提高精排的相对占比。二者都不会改变丢弃特定候选的质量后果。

这是成本模型推论，不是对每项历史 / 新版预算差异的因果解释。ADC 旧校准时间和收益属于不同执行记录，同时改变多个条件。因果测试应固定预算决策与候选集合，只替换精排内核。选中臂 QPS 还独立于校准计时，真正服务节省需要足够复用，四单元直接轨迹现已测量该边界（第6.5节）。

新版 FULL 在全部 12 个单元满足所测 10^{-3} 相对损失目标，提供实测质量优先工作点。小样本降低校准成本，却可能接受有害预算。独立结果位模型的分辨率计算支持约 500 条的样本尺度，较强独立性和排序条件仍明确保留，补证十二个 FULL 枚举均已一致。工程决策需要依据质量要求和预计复用来投入校准工作，单独优化选择器或只报告有利的平均选中臂吞吐都不足以完成该决策。

8.2 两项深入的负结果案例

8.2.1 Mailman 式共享求和：算术复用与散射成本

primary 距离的符号码形式提示：共享位模式的候选可以复用求和。Mailman 提供有限字母矩阵–向量乘法的一般思路 [20]。历史 pilot 固定 Vogue、arXiv、OpenAI-3072 候选流，在 128 probe、batch 8/1,000 上测试 GPU 实现。基线复刻 packed 两表评分循环；替代方案包括原子桶累加、私有直方图及 4/6/8 组宽，数学分数和输入编码不变。

六个单元中最快替代方案只有基线评分吞吐的 0.187–0.581 （图 8.1）。阶段 profile 把大多数插桩周期定位到桶累加。list 内偏置模式造成重复桶目标；私有直方图避免原子操作，却增加状态与共享内存读改写。缩小组宽、绕过主要桶可以改善部分变体，但都未追上基线。这支持停止这些实现与输入上的路线，不说明所有有限字母复用都无效：胜出的两表基线已经沿另一方向复用坐标和。

pilot 只写出 primary 分数，没有返回最终近邻。FP32/FP64 检查揭示 cutoff 附近数值差异，不等于完整流水线等价。以实测评分比 s 和估计可替换延迟占比 f，乐观替换模型为 $1/[(1-f)+f/s]$；由于未集成，其预测减速只是模型。辅助无损字典路线也在结构统计阶段停止，有限字节 / 工作节省没有支持 GPU 实现。两条路线均未向主前沿贡献系统实测点。

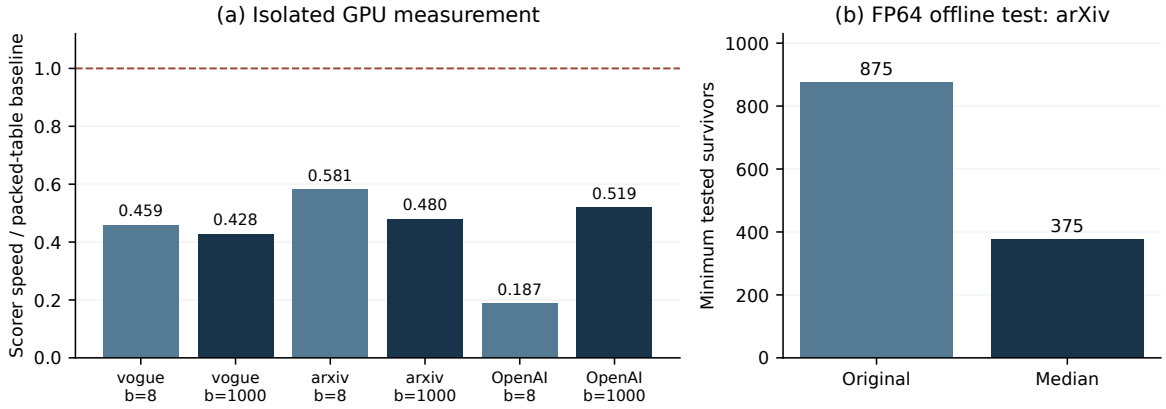


图 8.1. 两种不同证据边界。左：RTX 5090、128 probe、Vogue、arXiv、OpenAI-3072 和两种 batch 下，最快 Mailman 评分器 / packed 表评分器速度，全部小于 1。右：arXiv 的 400 查询 FP64 离线测试划分中，满足原 JHQ recall 目标的最小 survivor 数。候选减少不是 GPU 提速；两图均由论文 audit 脚本从归档记录重新生成。

8.2.2 面向排序的编码：局部收益不等于通用替代

排序导向研究在相同名义码长下，改变 1-bit 阈值或打分，尝试以更少 survivor 保留有用近邻。主比较在导出候选上用 FP64 回放，划分为 400 校准、200 验证、400 最终测试。方案包括似然权重、中位数阈值、学习排序损失、线性 MMSE 解码与 list 相关阈值。较低的重建 / 内积 MSE 可能使极少数最近候选的顺序变差；改变 primary 电平还会改变残差参照点，因此不同目标暴露不同失败模式。

arXiv 测试集中，中位数阈值把达到原目标所需的最小被测 survivor 数从 875 降至 375。按同一严格标准，成功未推广到另外两个工作负载。共享旋转下的高斯直觉也不能消除真实语料均值偏置，因为旋转保留均值向量的范数。居中编码可以修复部分失配，但必须同时评价 primary 排序与重建分数上限。因此停止的研究支持工作负载相关改进，而非普遍排序最优的新量化器。

后续仅 median 方案的 GPU 跟进是另一层证据：把改变后的阈值、电平及重新训练的残差导入相同搜索流水线，分别扫预算，再按实测 recall 阈值比较。在 arXiv 原 $\alpha = 100$ 目标，batch 8 和 1,000 的比值约为 1.40 和 1.21。Vogue median 前沿最高 0.9630，未达到原工作点 0.9644；OpenAI-3072 基本持平。这修正了“从未做 GPU 测试”的笼统表述，同时保留不采用通用替代的决定。跟进使用独立索引状态、查询池和三轮协议，不能把其吞吐插入 E1。

8.3 内部与外部有效性威胁

内部有效性最强的是同进程配对干预；版本、训练状态或接口同时变化时最弱。表 8.1列出可反转各解释的因素。第 3 章的共享输入、明确分数契约、计时数组和版本清单构成主要控制。重复调用估计条件计时波动，不估计独立重训索引的分布；五次补证构建量化了固定工作点的 recall 变化，而非整条重训前沿分布；精度敏感性在同一缓存模型下改变 add / 查询算术。

表 8.1. 可能反转或缩小结论的威胁。

受影响主张	替代解释或反转	已有控制与剩余验证
执行收益	分数、并列或索引改变产生表面提速	配对状态与输出检查；T4 严格 CPU 排序及表形式数值诊断；覆盖有限，候选 dump 已不保留
预算质量	有限池抽样或漂移掩盖有害查询	留出 GT、完整轨迹、子集和核对及 FULL 枚举；未来查询及任意子集不变性未测
竞争差距	网格密度、慢调用或 API 开销改变排名	两种 recall 估计、保留慢调用、int8 及 TF32/FP32 敏感性；T5 已显示小 batch 反转，跨设备比较仍未测
容量 / 构建	瞬时分配超过常驻样本	阶段字节、重复空缓存构建及 20 ms 设备峰值；更短暂分配与完整重训前沿未测

外部有效性限于一张 GPU、平方 L2 top-10、六份准备后的嵌入数据、E1 的 batch 1,000 及 T5 的 8/32。T5 高 recall 排名反转说明 E1 排名不能跨 batch 外推。低目标比较必须保留已达标前沿端点；实测阈值结果揭示受网格影响的排名。按位置移除查询不认证没有近重复嵌入，这些导出也不是带相关性标签的检索基准。数据库更新、并发流量、部署延迟目标和变化的查询总体未测试。压缩索引常驻结果不构成十亿规模或多设备容量证据。这些局限给出把所观测工作区间迁移到部署前需要补的验证。

8.4 可检验的后续研究方向

8.4.1 带风险控制和复用成本的输出一致性策略

后续策略可在有效的选择后置信界下，最小化校准与预计复用时间，并认证逐查询的参考一致性损失。输入为匹配执行路径的逐 query 不一致曲线和固定参考，变量包括样本量及 survivor 预算。绝对 GT 目标还需要独立标签验证，或参考与 GT 之间的额外假设。平稳性、逐查询抽样与共同分数 / 排序契约必须明确，漂移触发器应使旧证书失效并重新认证。

最低实验应在留出池比较固定样本规则、秩曲线复用和成本策略，记录总耗时与尾部一致性损失。旧四单元离线回放的序贯控制器已因额外搜索 launch 吞掉样本节省而失败，单纯重复序贯测试不足以形成改进。新的问题是：复用中间信息与质量证书能否联合改善总成本。违反风险限制，或比相同质量的固定样本对照没有省时，即失败。

8.4.2 考虑容量的残差放置

大型索引中常驻残差占主导。可研究 primary 常驻、残差 block 在显式显存上限内缓存，miss 从主机内存取回。固定编码、路由、survivor 集合与精排算术，隔离放置与表示质量。最低实验改变显存上限、batch 和查询局部性，测 hit、传输字节、重叠、尾延迟和吞吐。只有在可接受的总时间前沿上扩展容量才算有效；原本可常驻规模上更慢的 cache 不足以支持方案。这是未实现的存储扩展，与已有残差下界 pilot 不同；降低 α 也不实现它，因为完整数组依然分配。

8.5 有条件的部署建议

压缩常驻层次能放入设备、且目标 recall 位于已测竞争区间时，JHQ-GPU 值得评估。E1 支持 batch 1,000 的高 recall 吞吐；T5 则显示 recall 0.97、batch 8 下，四个数据集均由 CAGRA fp32 领先。batch 32 的 OpenAI-3072 在插值下接近平手、实测阈值有利于 JHQ，其余三个有利于 CAGRA。低 recall 优势同样依赖数据集和网格。分组调度能改善 JHQ 内部执行，但不保证跨系统领先（第7.2.5节）。

FULL 是质量优先校准的实测起点，之后应使用部署查询验证并直接计时复用总成本。应在实际 batch 制度下联合比较质量、总时间和常驻显存。单客户端同步实验尚未证明并发服务尾延迟、在线更新性能或总体 recall 保证。

第 9 章

结论

JHQ-GPU 展示如何把结构化量化层次转化为有用的 GPU 搜索流水线，以及为什么执行与精排工作量需要联合评估。四个研究问题得到具体且有边界的答案。

9.1 四个研究问题的答案

RQ1：构建与容量。 GPU 训练、batch 编码、IVF 放置及 packed 数组形成可搜索常驻层次。最大被测数据库为 17.78 百万向量、常驻 20.44 GiB，补证 train+add 为 10.54–10.65 s（第7.7节）。Stella 采样构建峰值 22.58 GiB，BGE 在 add 入口达到 26.08 GiB。独立重复构建和启动区间更完整地说明了已测容量边界。

RQ2：正确高效的执行。 继承 Cartesian 编码允许实数精确拆表，完整键保留 / 合并及共享界论证说明分组选择条件。两组表在 22/24 单元领先；相对 lane 精排参考，集成调度在 batch 8 的吞吐几何平均为 2.727×，batch 1,000 只有 1.015× （sections 7.3与7.6）。有效并行因 batch 和工作分布而变；T4 严格 CPU 参考与全部 55,296 个分组实例完整键顺序一致，56 个单 block 集合差异全部是截断并列（第7.4.1节）。

RQ3：精排预算与质量。 无标签参考一致性无需学习预测器即可选择工作负载 survivor 工作量。FULL 在 12/12 单元满足所测相对 recall 损失目标，最大损失 0.0006；$S = 128$ 一个单元则有 40/64 次越阈（第7.5节）。参考分析解释有条件的预算搜索，不给总体 GT 保证。10 个 FULL 单元的一般完整键嵌套仍有条件，但十二个 FULL 有限网格选择均与枚举一致。四个单元的直接校准加复用测量支持合适工作负载的收益，并保留困难工作负载的质量 / 时间权衡。

RQ4：竞争区间。 0.97 recall 的大 batch E1 插值比较中，JHQ 在三个数据集第一，arXiv 与 IVF-RaBitQ 接近平手（第7.2节）；Vogue 领先依赖分组路径。OpenAI E1 使用 TF32；同会话 FP32 敏感性在该目标仍领先归档 RaBitQ 7.9–8.6%，为 CAGRA fp32 的 1.63–1.68×。低 recall 由 CAGRA int8 领先，部分高目标由 IVF-RaBitQ 领先。T5 收窄该区间：recall 0.97、batch 8 下，四个数据集均由 CAGRA fp32 领先；batch 32 的边界依赖 recall 估计（第7.2.5节）。排名因此依赖 batch、质量和网格，显存另行报告；跨设备排名仍未测。

9.2 系统原则与展望

最终原则是在明确质量目标下联合评估表示、执行与工作量。表更小可能工作更多，分组更多可能重复工作，精排更少可能牺牲最重要近邻。负结果解释数学复用和更低重建误差为何未必改善搜索前沿。下一步应检验复用校准信息能否以更低总成本提供有效风险控制，以及

显式残差放置能否在可接受尾延迟下扩展容量（第8.4节）。这些是假设，来自已测局限，不是当前系统已经提供的能力。

参考文献

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[5] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3(4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[6] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[7] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[8] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[9] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[10] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[11] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL https://arxiv.org/abs/2409.09913.

[12] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html.

[13] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf.

[14] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL https://proceedings.mlr.press/v119/guo20h.html.

[15] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[16] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL https://arxiv.org/abs/1702.08734.

[18] Karthik V., Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbor search using a single GPU, 2025. URL https://arxiv.org/abs/2401.11324v4. Version 4.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL https://www.cs.yale.edu/publications/techreports/tr1402.pdf.

[21] Rui Luo and Mengxuan Zhang. JHQ-GPU: Representation-driven GPU execution and workload-adaptive hierarchical refinement, 2026. Project manuscript artifact; its historical measurement versions are distinguished in this thesis.

[22] National Institute of Standards and Technology. *Dataplot: Sign Test*, 2023. URL `https://www.itl.nist.gov/div898/software/dataplot/refman1/auxillar/signtest.htm`. Updated 11 December 2023; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL `https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm`. One-sided exact binomial limits; accessed 28 October 2026.

[24] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL `https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf`. Sampling without replacement; accessed 28 October 2026.

[25] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html`. Section 1.4.1.1; accessed 28 October 2026.

[26] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html`. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[27] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html`. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[28] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL `https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html`. PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[29] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[30] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M`. Accessed 28 October 2026.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M`. Accessed 28 October 2026.

[32] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[33] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/macrocosm/arxiv_abstracts`. Accessed 28 October 2026; exported embedding preparation is project-specific.

[34] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding`. Corpus split; accessed 28 October 2026.

[35] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3`. Italian configuration; accessed 28 October 2026.

[36] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL `https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html`.

[37] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL `https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf`.

附录 A

Reproducibility and Experiment Manifest

A.1 Source and Build Environment

Chapter 7 is an offline analysis of archived measurements in the sibling `JHQ_GPU` repository. The analysis checkout is recorded in `data/ch7/audit.json`; this is distinct from each experimental run's source head. 表 A.1 maps component names to implementations, recorded provenance and executable names. A run-head commit identifies source visible to a run, not a cryptographic identity of its binary. The archives do not supply a complete collection of executable SHA-256 hashes or immutable trained-state hashes. Missing identities are left explicit rather than reconstructed from the current checkout.

Chapter 4's source and construction-record audit is recorded separately in `data/ch4/audit.json`. It distinguishes the inspected source, modelled bytes, archived train/add inter-

表 A.1. Component and run provenance. "Report" distinguishes a version stated in an experiment report from a raw run-header commit. The exact E1 build state is not inferred from a baseline's metadata.

Component	Recorded source evidence	Executable or harness	Run/state source
E1 final pipeline	Environment logs include `a31c22d`; extensions have their own metadata	`demo_jhq_v69_batch`; `bench_cuvs.py`; `bench_rabitq_sweep`	E1 `RAW_RESULTS/`; shared trained-state cache
Lane refinement (v61)	Raw header `b37efa4`	`demo_jhq_v61_ab`	`warp_refine/raw/run.log`; `arms6_cache`
Grouped scan (v62)	Report: `862543a`; run script `e0d0518`	`demo_jhq_v62_batch`	`pipeline_v62/raw/`; recorded compiler flags
Shared bound (v63)	Implementation and build flags archived; no run-head claim inferred here	`demo_jhq_v63_batch`	`pipeline_campaign/exp2/v63_raw/`
Batch scheduling (v65)	Raw header `d425ab8`	`demo_jhq_v65_batch`	`exp47/raw/run.log`; `arms6_cache`
Radix probe (v69)	Comparative runs and source archived; binary SHA not recorded	`demo_jhq_v65_batch`, `demo_jhq_v69_batch`	`practical_opt/v69_run.sh`; `v69_results/dumps.sha256`
Budget / table sweep	Raw header `d5e1526`	`demo_jhq_v69_budget_arms`; `demo_jhq_v71_gN_batch`, $N \in \{1, 2, 4, 8\}$	`adc_v69/RAW/`, `RAW_LUT/`; `arms6_cache`
Large int8 rerun	Stella degree-64 metadata `ad433a4`	`bench_cuvs.py`	`adc_v69/RAW_BIG/`; isolated processes
Historical budget	Raw header `bf31b41` (v57 family)	`demo_jhq_budget_arms.cu` harness	`paper_adc2026/data/budget_arms.log`

vals and resident samples. In particular, object construction, host QR and constructor library warm-up precede the reported train/add timers.

The principal environment is RTX 5090, driver 595.71.05 and CUDA 13.0; E1 uses cuVS 26.08.01. Release JHQ targets compute_80 PTX, JIT-compiled on the device. Native-library baselines and C++/Python interface boundaries differ. E1 improves comparability by reusing resources, preallocating outputs, using pinned input staging and timing int8 query quantization. JHQ and baseline output buffers still have different host memory types. These details limit interpretations of small margins.

The adaptive call budget uses a warm-up interval containing the first call. In the Python baseline harness that interval also includes initial result conversion and validation before the decision; it is not a pure steady-state latency measurement. The 274 reduced-call points therefore include a RaBitQ point above 507 kQPS. Actual calls and five round arrays remain in the JSONL records. The default exact-selection setting was also inspected at the budget run's historical source head, not merely in the current checkout: `JHQ_EXACT_TOPCK=0` opts into the old lossy primary-retention implementation. Final selection scans all retained refined scores and breaks ties by database ID.

A.2 E1 Measurement Details and Review Diagnostics

E1 normally times 24 calls per round. The baseline harness reduces this count when a warm-up interval exceeds its per-call threshold; 274 points use fewer calls, including some fast steady-state points. Reported QPS is the median of five round throughputs, each based on total queries divided by elapsed time, rather than the reciprocal of median individual latency. Device-only baseline timing is auxiliary: JHQ has no equivalent device-query API for the main comparison.

The parser's 369 false repeat-call identifier flags comprise 170 RaBitQ, 80 IVF-PQ, 68 CAGRA int8, 50 CAGRA fp32 and one brute-force point. They can include tie-order changes, without implying different ID sets or invalid outputs; no point is discarded solely for the flag. The raw records retain first/last recall and set/order diagnostics. JHQ does not populate the same flag. The earlier count of 401 is the union of these 369 flags and 367 B-versus-A mismatches, with 32 adding only the latter. Cross-protocol disagreement is distinct from repeat-call nondeterminism.

The second-review audit is `scripts/audit_ch7_round2.py`; its input identities and calculations are in `data/ch7/round2-audit.json`. It reconstructs the paired Vogue one-block frontier, counts actual scan paths and computes continuous local frontier slopes. A single cold/cache recall comparison is a sensitivity scale, not an empirical retraining distribution. The later vector-row check supports alignment of all query rows and a 10,299-row base sample (第A.6节); original entity-ID mappings remain absent.

A binary paired sign test counts the signs of paired differences [22]. Under hypothetical independent fair signs, the one-sided tail for at least 22 positives out of 24 is 1.7941×10^{-5}; the two-sided value is twice that. Here 22/24 instead counts wins among four LUT arms.

The explicit two-arm comparison gives 23/24, and probe configurations also share datasets and indexes. The chapter therefore reports the paired counts and per-dataset means rather than adopting that nominal significance test. Likewise compute_80 PTX is translated into native target code by the driver [28]; without a matched compilation ablation the sign of any throughput effect remains unmeasured.

A.3 Budget Harness Contract and Diagnostic Coverage

Chapter 6's policy and review audits are reproducible with `scripts/audit_ch6.py`; `data/ch6/audit.json` distinguishes source-derived scan paths, statistical illustrations and archived measurements. The budget harness's fallback grid string starts at 100, although its header comment lists 200. The campaign script explicitly sets the eight-point grid beginning at 200, so its recorded reference is unaffected. Full-pool calibration is one call per cell; selected-arm rates reuse the previously timed fixed arms.

The harness partitions query indices by parity: 500 even-indexed queries calibrate and 500 odd-indexed queries evaluate, or 499 for Stella. There are 64 draws for each $S = 32, 64, 128$. Partial seeded shuffling samples without replacement using `rand() % remaining`; the analytic finite-pool formula models uniform subsets. The initial warm-up traverses the entire query file, including held-out queries, but its outputs and GT do not enter the agreement decision. Scan-schedule calibration uses separate database pseudoqueries.

Each fixed arm receives one warm-up and three host-to-host timed calls. Calibration gathers the selected vectors before starting its timer, then runs a reference and candidate searches with graph capture bypassed. The host agreement comparison costs $O(Sk^2)$ per tested budget; sample gathering is $O(Sd)$ and reference-ID storage is $O(Sk)$. The timer includes search-call setup and host comparisons, while sample gathering and separate scan calibration are excluded. The logged percentile uses zero-based sorted index $\lfloor 0.95(m-1) \rfloor$, with $m = 64$.

The helper returns a grid index, leaving the index at the last tested multiplier. A caller must invoke `set_alpha(A[index])` before future searches; selected-arm benchmark results instead use lookup. Enumeration repeats the reference plus all seven other budgets, so both timers include a fresh reference. In the original campaign it is requested only for the subsampled calls: the FULL call omits `enum_out`. Thus the 2,304 matched sample decisions exclude the twelve FULL decisions. At $S = 500$, two Vogue cells use four groups and ten cells use the distance-only path. Matched calibration/evaluation paths remove a transfer difference, but do not by themselves supply a common cutoff order. The supplementary harness adds and passes all twelve FULL enumeration checks (第A.6节).

Finite-pool prediction requires more than a single 500-query score dump. For each intended S, the per-query disagreement counts must be invariant to subset membership and position under that execution regime. A FULL $g_{\text{scan}} = 0$ dump cannot establish counts for grouped subsamples. Under these conditions and monotone acceptance, arXiv's recorded fixed-arm quality map makes selecting an unacceptable arm equivalent to accepting 100: 200

is its only qualifying budget. This mapping uses held-out quality labels; acceptance counts alone do not predict unknown GT loss. The original archived log lacks per-query curves and per-draw choices; the supplementary traces supply them and reproduce all sampled grid counts from full-pool per-query subset sums. This finite check does not establish per-query invariance on arbitrary subsets.

A.4 Data and Trained States

The six database shapes and actual query counts are stated in 表 3.2. The comparisons use the same exported vectors and squared-L2 ground truth per dataset. The budget split uses even query indices for calibration and odd indices for evaluation; Stella has 500/499 queries. Search sweeps load the recorded `arms6_cache` trained states, whereas cold-build records explicitly start without trained state. The chapter's file ledger hashes analysis inputs, not the full underlying vector files or every cache object; it does not certify unrecorded data or binary identities.

For the table-grouping sweep, `evidence/ch5/v71/` supplies selected source extracted from the recorded run head `d5e1526`, its harness, build definitions, run parameters and original records. Its manifest hashes 96 timed JSON records, four gate-check JSON records, their text outputs and build/progress logs, together with the selected source. 图 7.4 is derived from the timed records using `scripts/build_ch7.py`. This review bundle closes the local source-to-result inspection path and is included in the report repository; it supplies neither measured binary identity nor the datasets and trained states needed for GPU reproduction.

The Vogue probe comparison stores matching SHA-256 hashes for the one-group, batch-8 probe/primary dumps, covering all 1,000 queries. Its baseline identity run reports recall 0.9642; the same baseline binary's two subsequent timing runs report 0.9644. The dump controls a specific routing/screening boundary. It does not store a complete cross-process residual-code, query-rotation or score trace and therefore does not identify the numerical root cause downstream.

Two exclusions affect the scale evidence. Initial large-baseline builds retained an earlier index through a closure; successful isolated reruns supersede those attempts, including Stella CAGRA int8. Separately, Stella's 999-query pool causes the full-batch harness to skip batch 1,000. Top-level corrected large-search records use batch 999; superseded skipped-search files supply no throughput points. Cold-build logs are used for train/add time even where their search list is empty.

A.5 Result-to-Figure Mapping

表 A.2 maps the empirical figures to their measurement families. The Chapter 7 empirical plots and main numeric tables are generated by `scripts/build_ch7.py`; the additional path table is generated by `scripts/audit_ch7_round2.py`. `data/ch7/sources.csv` gives consumed input paths relative to `JHQ_GPU` and their hashes. Historical v57 and newer budget comparisons

表 A.2. Chapter 7 result-to-figure mapping. Paths below are relative to `JHQ_GPU`; detailed file identities are in the hash ledger.

Figure	Raw source
7.1	`experiments/comp8800_poster_e1/RAW_RESULTS/`; `analyze.py` frontier and interpolation definitions
7.4	`experiments/adc_v69/RAW_LUT/v71_g*.json`
7.5	`paper_adc2026/data/layout6.log`; `experiments/warp_refine/raw/ab/` and `raw/summary.json`
7.6, 7.7	`experiments/adc_v69/RAW/budget_v69.log`; actual 500/499 query counts
7.10	`experiments/pipeline_campaign/exp47/raw/bench/`; `experiments/pipeline_v62/raw/perf/stages_summary.json`
7.11	`experiments/thesis_evidence/RAW_A1/` normal repeats; baseline index records, with int8 dataset quantization
7.12	`adc_v69/RAW_LUT/v71_g2_*_np128.json`; modelled component sizes in the chapter analysis script
7.13	`experiments/adc_v69/RAW_BIG/` corrected top-level records

retain separate versions and timing protocols. The memory decomposition combines measured totals with an explicit component model; its unaccounted remainder remains visible.

To regenerate the report, run `make ch7-data` and then `make pdf`. The optional Chinese reading copy uses `make pdf-zh`; its table translator reuses numeric data from the English tables. Regeneration is an offline consistency check, not a new GPU measurement. The original vectors, trained states, matching GPU software environment and executable build would be needed for an independent benchmark rerun.

A.6 Supplementary Evidence of 28 October

The supplementary records are under `JHQ_GPU/experiments/thesis_evidence/`. Running `scripts/audit_supplement.py` recomputes the figures and bilingual tables, with a ledger at `data/supplement/audit.json`. Its input ledger hashes raw records, analysis/run scripts, exported PTX and toolkit SASS, and the relevant source. Original E1 and ADC campaigns retain their own records. The supplementary campaign uses RTX 5090, driver 595.71.05 and CUDA 13.0; cross-session baseline comparisons are explicitly labelled.

Construction and representation. A1 uses an empty trained-state cache per run: first read, three normal warm-input repetitions, and a synchronised phase diagnostic, for each of six datasets. Process-start timestamps end at the printed add completion; schedule calibration and the first search follow. Device memory is sampled every 20 ms, with a 20 ms allowance at the output boundary. It is device used memory including context, not an allocation-exact process trace. The phase run forces one pipeline buffer and synchronisation, so its times and peak remain separate. Five-build recall is measured at each dataset's fixed 128-probe point, not over five complete frontiers. C1 structurally checks four complete databases and recodes 200K/100K/20K/100K rows for Vogue/arXiv/OpenAI-3072/Stella. Norm checks apply FP64 to the stored matrix, not to a dump of actual GPU rotation outputs.

Machine-code identity. The inspected v69 executable has SHA-256 (line break for display):

```
5371616ed99ab2b2dd323688eb3cf7a2
3dafb9f3f6a3904cf44475f65cf63ceb
```

A2 extracts its `sm_80` PTX and reassembles it with toolkit 13.0.88 `ptxas` for `sm_120`. The bound read is `LDG.E.64.STRONG.SYS`; publication is `REDG.E.MIN.64.STRONG.GPU`. This validates the extracted PTX contract and corroborates it with toolkit machine code. The driver JIT's actual loaded SASS is not archived.

Row correspondence and precision. A3 checks every query and a stride-97 base sample by truncating and renormalising the wider OpenAI embedding; it is vector correspondence, not an original entity-ID export. D alternates v69/v72 order by probe setting, using the original E1 grid and five timing rounds per configuration. Both variants reuse the TF32-trained cache. The FP32 switch affects add/query GEMMs; the experiment does not retrain FP32 models or remeasure E1 baselines. Source patches are archived as local commits `d924b30`, `b445c96`, `51d5502`, with server counterparts documented in the experiment report; patch correspondence is distinct from executable identity.

Budget trace and direct timing. B preserves the original `srand(12345)` sampling sequence for 2,304 draw records and adds twelve FULL enumerations. Its JSONL records contain sample IDs, complete disagreement curves, selected arms, losses, full-pool per-query counts and direct execution traces. All 18,432 sampled grid counts equal sums of the full-pool counts over the recorded sample IDs; this is an aggregate check on measured subsets. The 36 finite-pool predictions need no GT for acceptance; the conversion to GT-loss crossing probability uses the held-out fixed-arm quality map. Direct reuse timing instead uses a separate `mt19937(20261028)` sequence, five rounds with rotated policy order, and 500 repetitions of the same held-out batch. It includes sample gathering, calibration, explicit budget installation and first-batch graph setup. Earlier construction, schedule calibration and warm-up are excluded. Fractional break-even values fit measured calibration and steady batches; cumulative totals provide the direct observation. First schedule calibration calls take 6.00–27.77 ms, separate from 0.81–7.04 ms pseudoquery gathering; five repeated calls give medians of 2.72–18.13 ms.

The later T4 campaign completes the finite-grid strict CPU selection reference (第A.7节). The controlled same-model Vogue cross-process diagnosis (C3) remains unperformed; T4 checks within-run selection rather than that cross-process anomaly. Remaining contracts are tracked in `assessment/ch7-pending-experiments.txt`.

A.7 Strict Selection and Small-Batch Follow-ups

T4 uses `examples/demo_jhq_topc_reference.cu` and `jhq_v73_topcref/search.cu`. Normalised source comparison of all ten CUDA source/header files with `jhq_v71_lutgroups/` finds no execution change after removing the two debug-export blocks. The dump follows the packed word layout and calls the same `jhq_lut_sum` in subspace order. CPU sorting consumes

these GPU-produced primary scores, and the retained refined scores for final selection. The zero score-mismatch count compares exported scores against returned valid primary entries; it is not separate instrumentation of every rejected candidate. The padding counter verifies the number of valid entries, rather than the bytes of every unused sentinel. Cutoff ties occur in 352 grouped and 88 one-block query instances. Cases include natural short candidate pools; deliberately constructed empty/adversarial lists remain a separate test.

The table-form comparison fixes 128 probes, $\alpha = 100$, 256 queries, shared bounds enabled, $g_{\text{scan}} \in \{0, 2\}$ and $g_{\text{LUT}} \in \{1, 2, 4, 8\}$. It aligns candidate positions before comparing FP32 score bits. Full-table versus two-way maximum absolute error is 4.77×10^{-7}; the maximum over all forms is 5.97×10^{-7}. On Vogue, the grouped four-way table changes four final answers despite unchanged primary survivor sets: primary-score rounding can propagate into the refined score. Thus this evidence establishes small numerical error and measures its ranking effects, rather than FP32 identity. Temporary `.topc` dumps were deleted by the campaign. The report rechecks retained `RAW_T4/` JSONL tallies against stdout, and consumes `lut_compare.json`; it does not rerun those deleted comparisons.

T5's launcher is `t5_smallbatch.sh`. Baseline harnesses are `bench_cuvs_smallbatch.py` for CAGRA and `bench_rabitq_smallbatch.cu` for RaBitQ, all under `experiments/thesis_evidence/`. JHQ/CAGRA reuse their E1 grids; RaBitQ uses 4/9-bit codes, two modes and eight probe counts. Search timing includes staging, transfers, int8 query quantisation when applicable, and synchronisation. JHQ captures its graph with an untimed call; baseline warm-up traverses the used query pool. Five round throughputs are reduced by their median. The report consumes numeric JSON, avoiding rounded stdout at strict recall thresholds. Null timing is a separate diagnostic and is not subtracted. It does not equalise Python/C++ binding overhead or establish a service latency guarantee.

`scripts/audit_t4_t5.py` generates the matched-recall table and figure, normalised point registry and hashed input ledger under `data/t4-t5/`. Its interpolation estimate follows the archived near-target/log-QPS rule, retaining a qualifying measured endpoint below the minimum frontier recall. The original T5 summary omitted such competitors: OpenAI-3072 at recall 0.90 changes from about 2.08 to 1.032 at batch 8, and from 1.26 to 0.866 at batch 32. Strict measured thresholds are reported separately, revealing the effect of finite grid spacing. These estimates leave the historical E1 campaign unchanged.

附录 B

补充结果与正确性检查

本附录记录支持第 3–8 章的参数覆盖、逐阶段验证及次要结果。各测量家族保留各自版本与计时边界，来源映射见附录A。

B.1 完整参数记录

E1 比较有 1,590 个有效实测配置，覆盖见表 B.1。逐点完整参数 JSON、recall、主机到主机 QPS 及源文件名保存在 `data/appendix/e1-configurations.csv`。`scripts/build_appendix.py` 生成该记录，输入身份见`data/appendix/audit.json`。下列集合表示观测值，不假定其笛卡尔积全部测量过。

表 B.1. 包含覆盖扩展的 E1 精确配置数。计数为参数点，不是独立索引构建。

方法	Vogue	arXiv	OpenAI-1536	OpenAI-3072	总计
JHQ-GPU v69	27	29	37	39	132
CAGRA fp32	64	64	64	64	256
CAGRA int8	64	64	64	64	256
IVF-RaBitQ	132	132	264	198	726
IVF-PQ	54	54	54	54	216
GPU brute force	1	1	1	1	4
总计	342	344	484	420	1590

JHQ-GPU。 E1 均用 $B = B_r = 8$、$M = d/8$、$k = 10$ 和 batch 1,000；Vogue 为 4,096 个 IVF 列表，其他三个为 8,192。Vogue 观测探针 16、32、64、96、128、192、256、384、512，$\alpha \in \{16, 32, 64, 128\}$，共 27 个实际组合。arXiv 再包含探针 1,024，$\alpha \in \{25, 50, 100, 200\}$，共 29 个组合。两个 OpenAI 均用探针 32、64、128、256、512、1,024；倍率集合分别为 $\{4, 8, 16, 32, 64, 128, 256, 512\}$ 和 $\{2, 4, 8, 16, 32, 64, 128, 256\}$，共 37 / 39 个组合。所报调度路径在 Vogue 用四组，其余用单 block。历史 v69 的 OpenAI 旋转 / 质心 GEMM 为 TF32；v72 FP32 敏感性另成家族（第7.2.3节）。

CAGRA fp32 与 int8。 每种表示每数据集 64 点：图度数 32 / 64、中间度数 128、`ivf_pq` 构建，`itopk_size` 为 32、64、96、128、192、256、384、512，搜索宽度 1、2、4、8。int8 数据缩放记录 quantile 0.99，主机搜索边界包含查询量化。

IVF-PQ。 八位子码在 768 维使用 192、384、768 字节编码，OpenAI 使用 384、768、1,536 字节。每种长度测试 fp16 / fp32 LUT 及探针 16、32、64、96、128、192、256、384、512，每数据集 54 点。记录的精排比例为一，列表数与 JHQ 相同；编码长度和显存预算不同。

IVF-RaBitQ 与暴力搜索。 RaBitQ 包含每维 4、8、9 位、探针 16、32、64、96、128、192、256、384、512、768、1,024。768 维与 OpenAI-1536 含 LUT16、LUT32、QUANT4、QUANT8，OpenAI-3072 含 LUT32、QUANT4、QUANT8；另用 4,096 列表 OpenAI 索引扩展默认 8,192 列表覆盖。精确组合见参数记录。每数据集还含一个 FP32 平方 L2 暴力搜索点。缺少某种模式或更高 recall 点不等于实测失败。

辅助网格。 LUT 实验固定 $\alpha = 100$，六数据集探针 8、32、128、512，$g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$，共 24 单元、96 计时臂。预算实验在相同六数据集用探针 128 / 512、$A = (200, 100, 64, 32, 16, 8, 4, 2)$，每个 $S \in \{32, 64, 128\}$ 抽取 64 次、$\epsilon = 1$；FULL 另用全部 500 校准查询。留出为 500 条，Stella 为 499 条。直接复用为四单元、四策略、五轮，每轮 500 个重复留出批次。边界见sections A.3与A.6。

B.2 正确性与数值证据

表 B.2区分数值、算法与内存证据。第五章排序论证针对满足所列契约的计算分数；原始向量 recall 另行评测。

表 B.2. 验证覆盖及实际核对对象。

边界	已有证据与范围
表示	四库全行 offset / ID 审计，原始输入采样 FP64 重编码、存储矩阵几何，已识别 TF32 偏差
Cartesian LUT	实数因子化；96 个计时臂；T4 覆盖四种表形式的逐候选 FP32 误差与排名诊断
全局选择	T4：270 配置、69,120 查询实例；55,296 个分组实例与严格 CPU 完整键排序的顺序一致
共享界	48 组配对；提取标量 64 位 PTX 及 toolkit SASS 支持完整 key；未捕获驱动实际 SASS
调度	250 个变体 / 配置及 234,320 次查询比较，区分 ID 集合与并列顺序
探针选择	1,792 合成用例零失败；真实查询 ID / offset 检查，针对已计算路由分数
预算搜索	2,304 采样曲线单调、12 个 FULL 枚举一致；18,432 网格计数可由全池子集和复现

较小表示审计库在按 ID 关联的一遍 / 两遍构建中主码和残差字节相同，correction 差至多 8.94×10^{-8}，列表分配有 1、1、40 行变化；Stella 验证自然两遍路径（第4.6.2节）。T4 已提供计算分数选择的严格 CPU 参考（第A.7节）；同模型 Vogue 跨进程受控诊断仍未执行：归档进程即使 probe / primary dump 一致，最终 recall 仍有 0.9642 / 0.9644。进程内选择检查并未定位这项独立异常。

B.3 次要结果与证据状态

不利运行区间。 调度器最大记录 regret 为 batch 256 的 1.217×；Vogue E1 中调度 / 单 block 吞吐为 0.895–1.175。使 0.97 recall 领先的偏斜规则也会在低探针单元增加开销（第7.6节），这些不利结果与内部调度收益一起保留。T5 跨系统比较中，recall 0.97、batch 8 时四个数据集均由 CAGRA fp32 领先；batch 32 的边界依赖 recall 估计（第7.2.5节）。

表示替代方案。 Mailman 式 GPU 评分器为独立 kernel 测量，端到端替换只是乐观模型。面向排序的主研究为 FP64 离线回放；另一个 median 编码 GPU 后续实验在记录目标上，arXiv

的 batch 8 为 1.402×、batch 1,000 为 1.208×，Vogue 未达严格目标，OpenAI-3072 接近平手。阈值、级值和残差训练同时改变，不能只归因于阈值（第8.2节）。

校准与精度。 有限池预测在查询计数稳定的条件下解释抽样风险，GT 损失映射使用留出固定臂质量。直接复用测量四个重复池的累计成本；精度干预在同一 TF32 训练缓存上改变 add / 查询 GEMM。完整 FP32 重训、未来查询漂移、跨设备排名及在线更新不属于这些实验。

B.4 材料获取

报告仓库提供参数记录、派生表、图源、分析脚本，以及`evidence/ch5/v71/` 下的 209 文件 v71 源码 / 测量包，可用`python3evidence/ch5/v71/check.py` 离线核对。更多原始记录位于同级 `JHQ_GPU` 项目的清单路径，完整向量、训练状态与所有可执行文件没有打包在本报告仓库。离线复算需要所列输入，GPU 复现还需要相应构建和环境。

附录 C

贡献与工具使用记录

C.1 研究角色与继承工作

Rui Luo 为 COMP8800 项目报告人及论文作者，Mengxuan Zhang 博士为导师。相关项目手稿将两人列为共同作者 [21]。本论文扩展正确性论证、较新测量、补证审计及失败设计讨论。作者名单表示共同工作，不把每个材料都归为个人独立完成。

Jiabao Han、Mengxuan Zhang 和 Goce Trajcevski 的 JHQ 方法提供表示基础 [15]。IVF、PQ、既有 GPU 选择 / 合并技术与基线库保留所引作者归属。本项目 GPU 构建 / 执行系统、工作负载预算校准及综合评测的对应见 表 1.1；源码历史、运行记录和分析清单说明材料来源。

C.2 生成式 AI 协助

表 C.1记录项目对话、源码及实验归档所支持的活动。工具按产品名标识，不依据这些记录推断具体模型版本。

表 C.1. 生成式 AI 协助及相应证据记录。

工具	活动与保留记录
OpenAI Codex	中英文写作修改、LaTeX、图及分析脚本；读取源码 / 日志、离线数值审计、PDF 编译。各章证据说明及`scripts/` 保留对应工作。
ChatGPT	用户提供的框架与章节审阅，涉及证据、范围和实验设计。`assessment/` 记录接受及纠正的建议。
Anthropic Claude	实现 / 实验协助及审阅；补证包含构建、表示 / 精度审计、预算轨迹、严格选择及小批次基线，记录位于 `JHQ_GPU` 的 `experiments/thesis_evidence/`。
共同手稿	独立披露 Claude / ChatGPT 在实现、程序、图和写作中的协助，其版本化材料保留相应范围。

C.3 核对、可追溯性与责任

审阅意见已对照源码和测量核查；修正包括选择器范围、预算迁移条件、OpenAI TF32 算术及构建显存生命周期。记录区分实测、数值恒等式、有条件证明、成本模型和未执行检查。

参考文献标注继承方法，材料清单标识代码与测量，审阅记录说明辅助修改。这些材料描述保留的工作流程，不声称导出了每次交互，也不代替个人签署。Rui Luo 承担提交论文、来源使用及采用材料解释的责任。